INDEX

SI.NO	PROGRAMS(ESP32, ARDUINO UNO, STM32)
1	Program to blink an LED.
2	Program to turn ON the LED when a push button is pressed.
3	Program to turn ON the LED when a touch sensor is touched.
4	Program to display the percentage of light intensity detected by the LDR sensor.
5	Program to print temperature and pressure readings from the BMP180 sensor in the Serial Monitor.
6	Program to print temperature and humidity readings from the DHT11 sensor in the Serial Monitor.
7	Program to print gyroscope (X, Y, Z axes) and accelerometer (X, Y, Z axes) values from the MPU6050 sensor in the Serial Monitor.
8	Program to print the distance between the ultrasonic sensor and an object in the Serial Monitor.
9	Program to print the gas concentration detected by the MQ135 sensor in the Serial Monitor.
10	Program to Detect the object by IR sensor and print the value in Serial Monitor.
11	Program to turn on the Relay using a Push button.
12	Program to display temperature and humidity readings on the TFT display.
13	Program to turn on the LED using Bluetooth communication.
14	Program to turn on the LED using WiFi communication in BKYNK Cloud.
15	Program to send data from the microcontroller to a Serial Port utility application using RS232 (UART protocol).
16	Program to control LED using Serial Port utility application by using RS485(UART Protocol).
17	Interfacing CAN Controller with BMP180 to transmit “BMP180 data” from Point-Point using Arduino Uno.
18	Program to send the DHT11 data from Microcontroller to Mobile by using 4G Module(UART Protocol).

19	MQTT on desktop version
20	Lora P2P
21	LoRaWAN OTA DHT11

SI.NO	PICO-PROGRAMS
1	Program to Blink ON-BOARD LED.
2	Interfacing Push Button to control External LED.
3	Program to print DHT11 data in shell using Pico W.
4	Program print “Hello world” in TFT display using Pico W.
5	Program to print BMP180 data in Shell.
6	Program to print MPU6050 data in Shell.
7	Program to control LED using BLYNK IOT platform.
8	Program to send DHT11 data to THINGSPEAK.

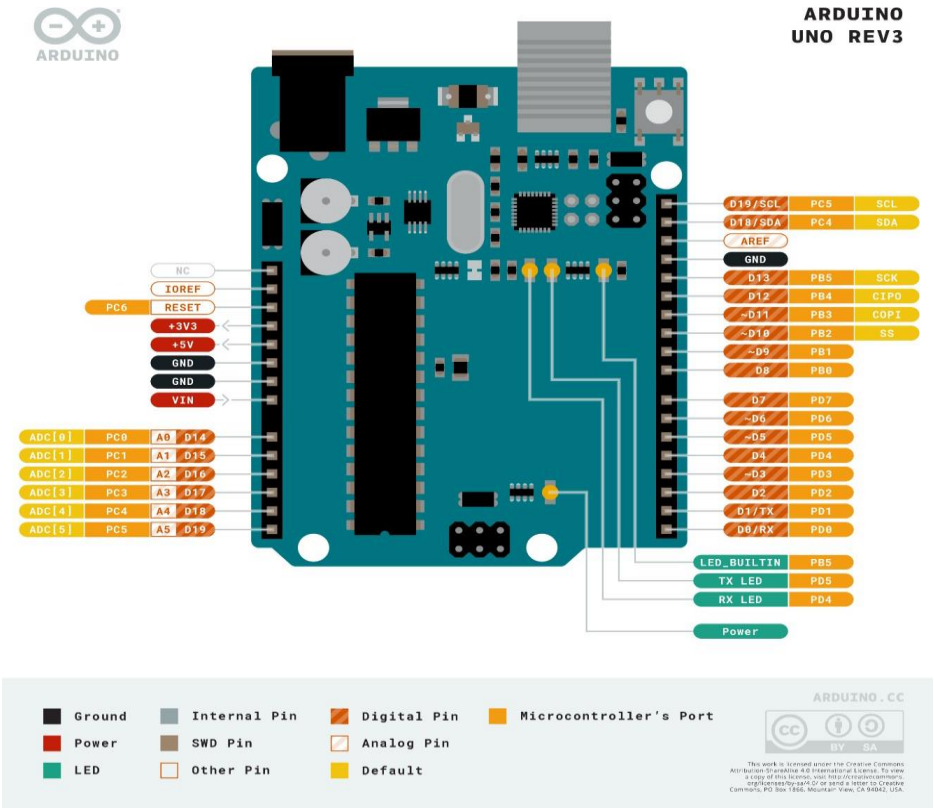
Microcontroller: Arduino UNO

Introduction of Arduino UNO:

The Arduino UNO is the best board to get started with electronics and coding. If this is your first experience tinkering with the platform, the UNO is the most robust board you can start playing with. The UNO is the most used and documented board of the whole Arduino family.

Arduino UNO is a microcontroller board based on the ATmega328P. It has 14 digital input/output pins (of which 6 can be used as PWM outputs), 6 analog inputs, a 16 MHz ceramic resonator, a USB connection, a power jack, an ICSP header and a reset button. It contains everything needed to support the microcontroller; simply connect it to a computer with a USB cable or power it with a AC-to-DC adapter or battery to getstarted. You can tinker with your UNO withoutworrying too much about doing something wrong, worst case scenario you can replace the chip for a few dollars and start over again.

GPIO Pin Outs:



Microcontroller: ESP32

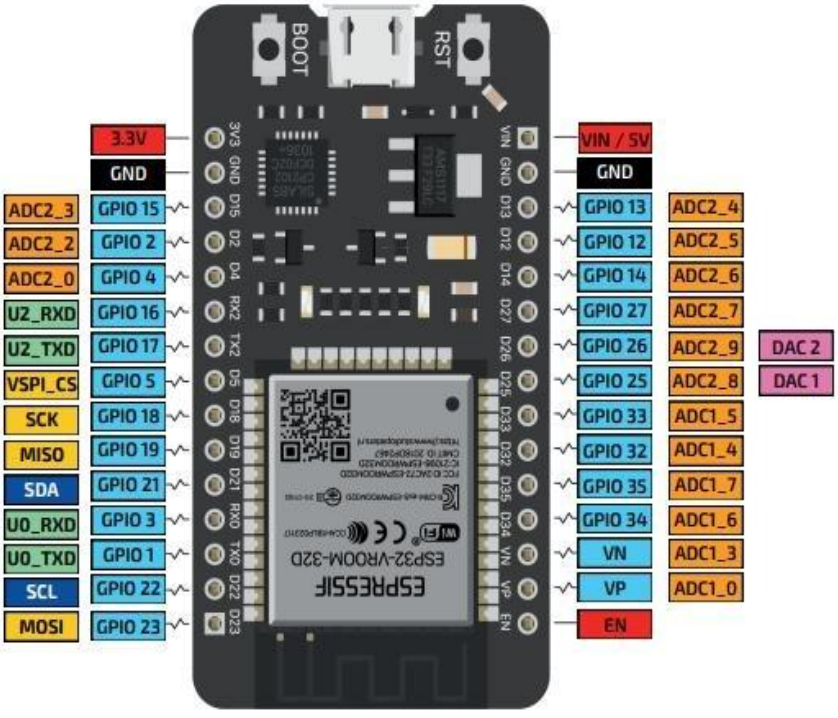
Introduction of ESP32:

The ESP32 is a dual-core system with two Harvard Architecture Xtensa LX6 CPUs. All embedded memory, external memory and peripherals are located on the data bus and/or the instruction bus of these CPUs.

GPIO Pins:

The ESP32 peripherals include:

- 18 Analog-to-Digital Converter (ADC) channels
- 3 SPI interfaces
- 3 UART interfaces
- 2 I2C interfaces
- 16 PWM output channels
- 2 Digital-to-Analog Converters (DAC)
- 2 I2S interfaces
- 10 Capacitive sensing GPIOs



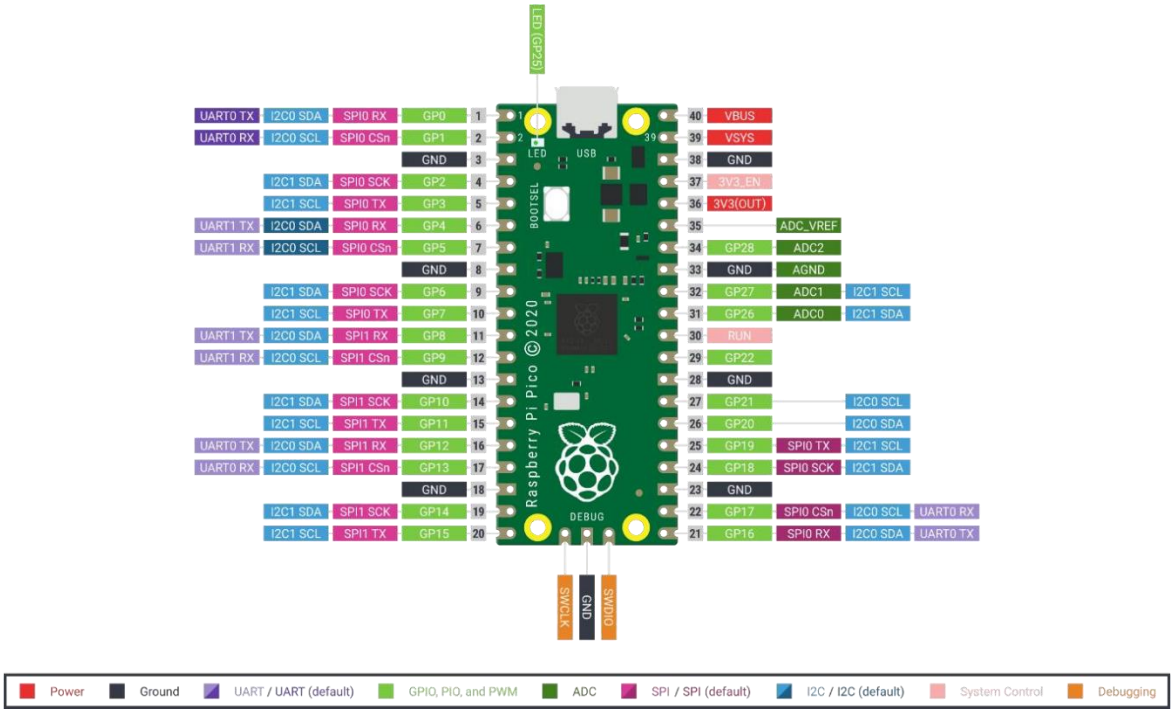
Microcontroller: Raspberry Pi Pico

Introduction of Pico:

No one expected that Raspberry Pi, the most popular single-board computer maker in the world, would suddenly release a microcontroller of its own. What's more surprising is that Raspberry Pi Pico does not base its design on the common ESP32or SAMD21, but instead a brand new microcontroller chip: the RP2040 microcontroller. The RP2040 microcontroller is a microcontroller chip independently designed by Raspberry Pi, and is powered by a dual-core ARM Cortex-M0+ processorthat runs up to 133Mhz.

GPIO Pins:

In addition to the powerful new chip, the board of Raspberry Pi Pico exposes 26 multifunction GPIO pins, including 2 * SPI, 2 * I2C, 2 * UART, 3 * 12-bit ADC, and 16 controllable PWM channels. I will explain the functions of these pins in later chapters.



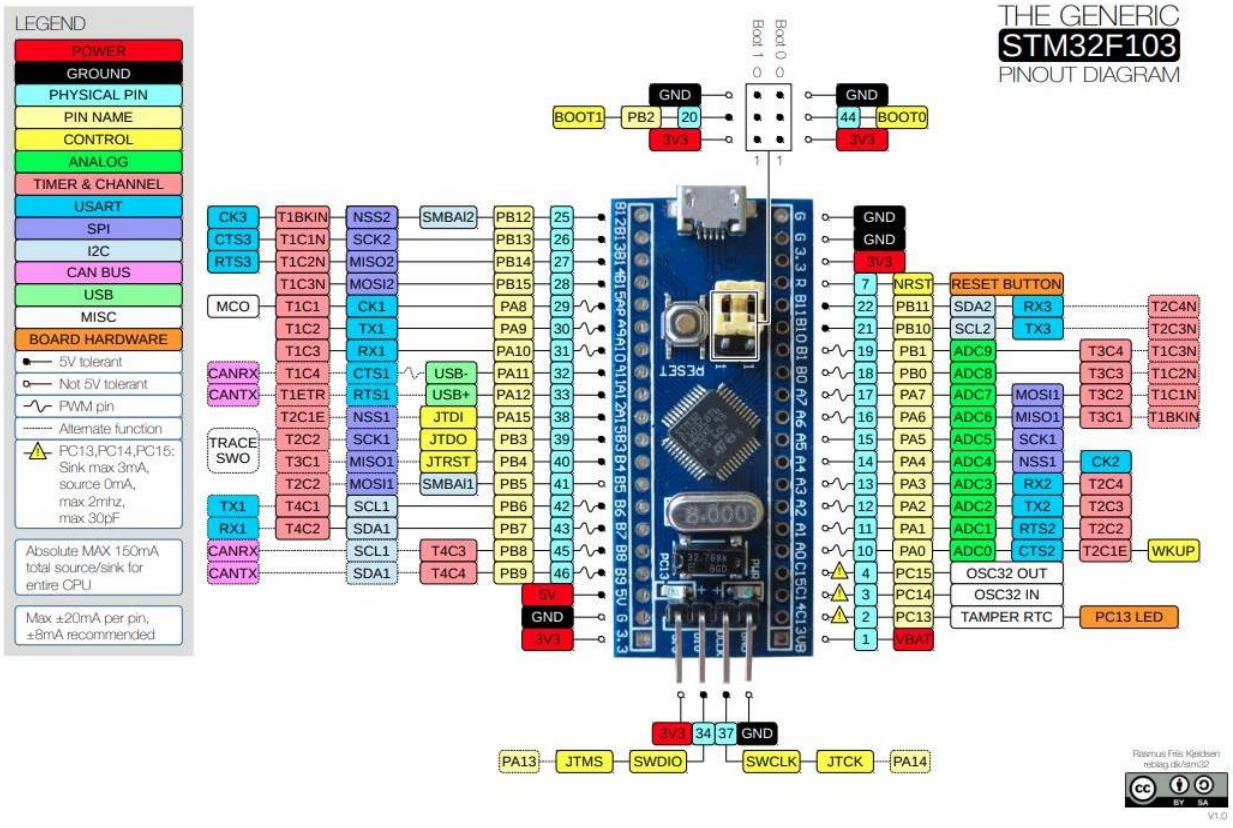
Microcontroller: STM32

Introduction of STM32:

The STM32 family of 32-bit microcontrollers based on the Arm® Cortex®-M processor is designed to offer new degrees of freedom to MCU users. It offers products combining very high performance, real-time capabilities, digital signal processing, low-power / low-voltage operation, and connectivity, while maintaining full integration and ease of development.

The unparalleled range of STM32 microcontrollers, based on an industry-standard core, comes with a vast choice of tools and software to support project development, making this family of products ideal for both small projects and end-to-end platforms.

- Supplying 3.3V directly to the 3.3V pin.



Sensors:

Sensors	Description
	**DHT11:** The DHT11 is a commonly used Temperature and humidity sensor that comes with a dedicated NTC to measure temperature and an 8-bit microcontroller to output the values of temperature and humidity as serial data.
	**MPU6050**: MPU6050 sensor module is complete 6-axis Motion Tracking Device. It combines 3 -axis Gyroscope, 3 -axis Accelerometer and Digital Motion Processor all in small package. Also, it has additional feature of on-chip Temperature sensor. It has I2C bus interface to communicate with the microcontrollers.
	**BMP180** (Barometric Pressure and Altitude Sensor): The BMP280 Breakout has been designed to be used in indoor/outdoor navigation, weather forecasting, home automation, and even personal health and wellness monitoring. And to measure barometric pressure, and temperature readings all without taking up too much space.
	**TOUCH Sensor:** A touch sensor is a type of sensor that captures and records physical touch or proximity. Provides direct mode i.e. toggle mode by pad option.
	**Ultrasonic Sensor** An ultrasonic sensor is an instrument that measures the distance to an object using ultrasonic sound waves. An ultrasonic sensor uses a transducer to send and receive ultrasonic pulses that relay back information about an object's proximity.
	**LDR Sensor:** The LDR is a special type of resistor that works on the photoconductivity principle means that resistance changes according to the intensity of light. Its resistance decreases with an increase in the intensity of light.

	**IR Sensor:** IR sensor is a device that uses infrared technology to detect objects or changes in the environment. IR sensors can detect a wide range of physical properties such as temperature, motion, and proximity.
	**MQ135 Gas Sensor:** The MQ 135 sensor can be implemented to detect smoke, benzene, vapors, and other hazardous gases. It can detect various harmful gases.
	**5V Active Buzzer:** Buzzer meaning electronic component that generates sound through the transmission of electrical signals. Its primary function is to provide an audible alert or notification and typically operates at 5V.
	**RGB LED:** SMD RGB LED technology allows the colour of light to be changed by using three diodes: red, blue and green.
	**Push Button:** A push button switch is a mechanical device used to control an electrical circuit in which the operator manually presses a button to actuate an internal switching mechanism.
	**10K POT:** This adjustable or Variable resistor are PCB mountable and has 3 terminals. The voltage between the terminal varies as the preset is rotated. The Variable resistors are used for variation voltage as per the need in a circuit.
	**5V Realy:** The primary purpose of a relay is to protect the electrical system from too high of a voltage or current, allowing the safe operation of any equipment it connects to.

Wired Communication Modules

Modules	Description
	**RS232 to TTL Serial Interface Module:** This RS232 to TTL Serial Interface Module is a board with the MAX3232 transceiver integrated circuit (IC). It facilitates serial communication between TTL and RS232 ports by providing the necessary electrical signal conversion.
	**MAX485 TTL To RS485 Module:** The MAX485 TTL To RS485 Module converts TTL signal to RS485 for long range, high data rate error prone differential communication. Digital communications networks implementing the EIA-485 standard can be used effectively over long distances and in electrically noisy environments. Multiple receivers may be connected to such a network in a linear, multi- drop configuration. These characteristics make such networks useful in industrial environments and similar applications.
	**MCP2515 CAN Bus Module:** It contains CAN Controller MCP2515 and TJA1050 which is a high-speed CAN trans-receiver. This module can be easily interfaced to any microcontroller via SPI Interface. It can also be interfaced to Arduino and Raspberry PI.

Wireless Communication Modules:

Modules	Description
	**LoRa Transceiver:** RFM series wireless communication module is designed to work at 865 - 867 MHz ISM band global free application. It is designed for low-power, high-bandwidth wireless digital communications module. The modules have a low cost, small, stable, product consistency and other characteristics, the highest spatial rate of up to 300Kbps, can be widely used in daily life and activities of the occasion requires a wireless connection can also be used for industrial control, access control, attendance, surveillance and security industries.

Display:

Modules	Descriptions

	**TFT LCD Display:** This lovely little display breakout is the best way to add a small, colorful and bright display to any project. Since the display uses 4 -wire SPI to communicate and has its own pixel-addressable frame buffer, it can be used with every kind of microcontroller. Even a very small one with low memory and few pins available!

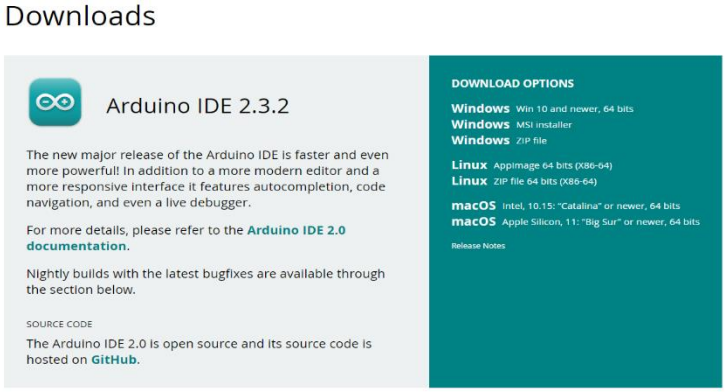
Introduction to Software

Installing Arduino IDE

The first thing you need is the Arduino IDE. If your computer doesn't have Arduino IDE installed, then visit the official Arduino download page and download the installation file for your preferred operating system.

Note: Download the current latest Arduino IDE software from below link.

Link: https://www.arduino.cc/en/software

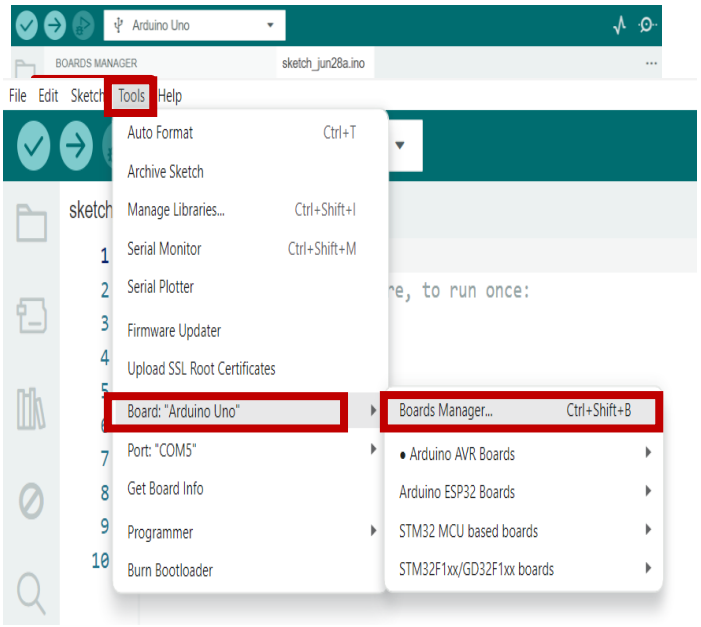


Installing the Microcontroller Board in Arduino IDE

1. ESP 32

There's an add-on for the Arduino IDE that allows you to program the ESP32 using the

Arduino IDE and its programming language. We'll show you how to install the ESP32 board in Arduino IDE.



1. Open the Boards Manager. Go to TooLs > Board > Boards Manager…

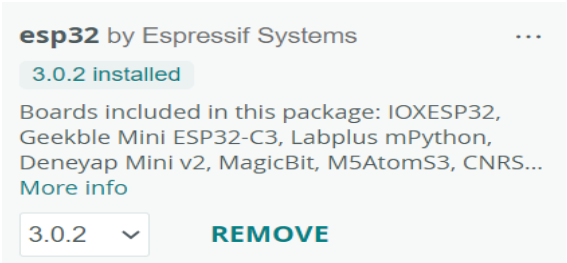
2. Search for ESP32 and press install button for the "ESP32 by Espressif Systems":

3. That's it. It should be installed after a few seconds.

Testing the Installation

Plug the ESP32 board to your computer. With your Arduino IDE open, follow these steps:

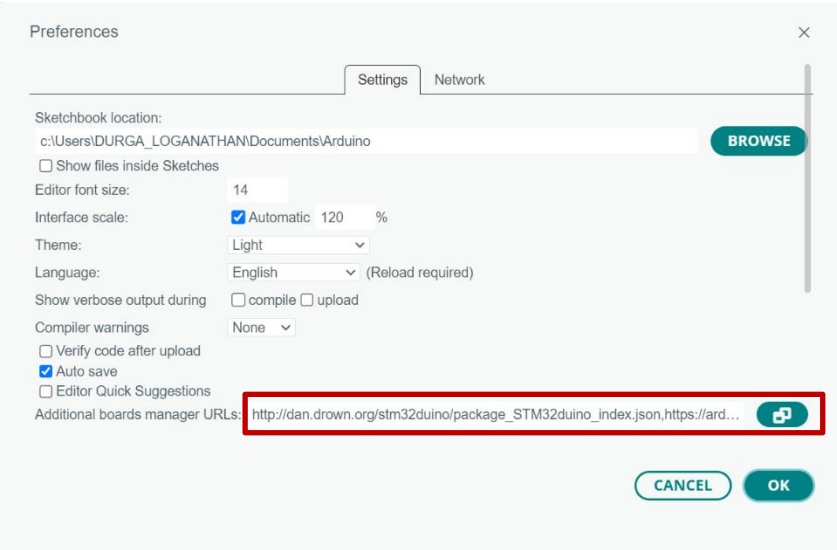
1. Select your Board in TooLs > Board menu (in my case it's the DOIT ESP32 DEVKIT V1).
2. Select the Port (if you don't see the COM Port in your Arduino IDE, you need to install the CP210x USB to UART Bridge VCP Drivers).



2. STM 32 Blu – Pill

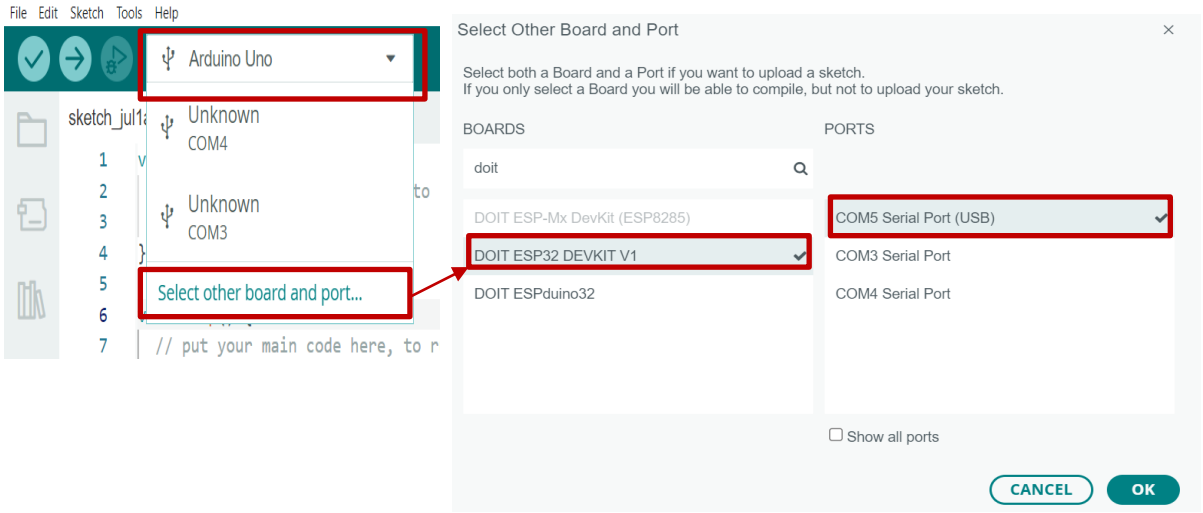
Configuring Arduino IDE to Program STM32F103C8T6 Blue Pill

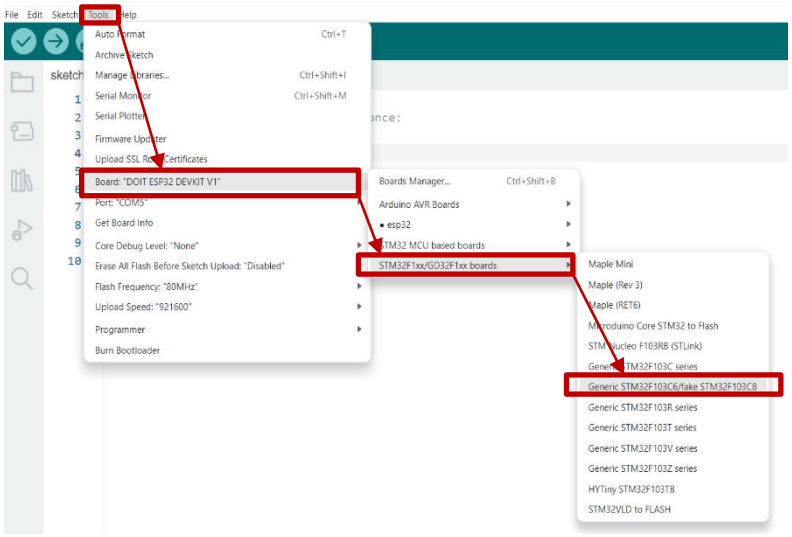
I am sure you already have Arduino IDE installed on your PC (or Laptop). If not, then install it first. After than open your Arduino IDE and select File -> Preferences. You will find a tab called "Additional Boards Manager URLs". Copy the following Link and paste it there.



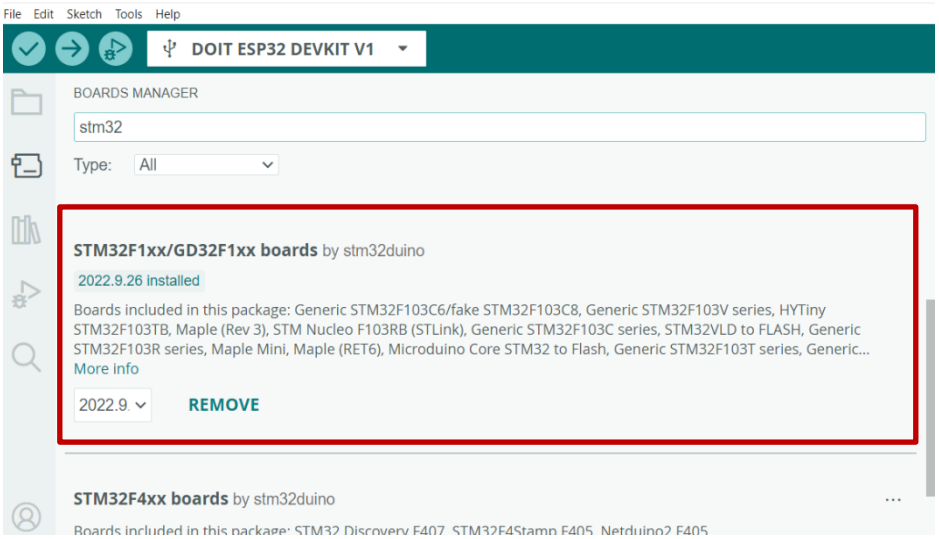
Link: http://dan.drown.org/stm32duino/package_STM32duino_index.json
Note: If you already have some URLs in this section, you can add more by separating them with comma. If you have worked with ESP8266 Boards, then you might be familiar with this process already. After adding the URL, click on OK.

Now, go to Tools -> Board -> Board Manager… option and search for "stm32". You will get a result Like "STM32 Cores by STMicroelectronics". Install the latest version.





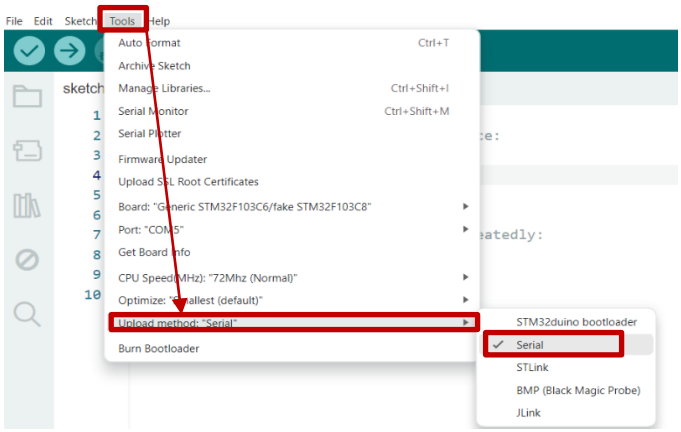
Now you can select the board from Tools -> Board -> Generic STM32F1 series. Once you select this board, a bunch of options will appear below for customizing your board type. The first important option is “Board part number”. Make sure that “BluePill F103C8” is selected.



The other important options are “U(S)ART support”, make it as “Enabled (generic ‘Serial’)” and “Upload method”, make its as “ Serial”. You can leave the remaining options as their default values.

Connections:

STM32 Blue Pill	TTL Conveter
5V	VCC
GND	GND
A9	RX
A10	TX



Installing Thonny IDE for Raspberry Pi PICO:

To build interesting projects with Raspberry Pi Pico, you need to learn how to program it first. Raspberry Pi Pico supports two types of programming languages: C++ and MicroPython.

C++ is a general-purpose language developed on the basis of the C language. It is often used in the development of ESP32 and Arduino, so most microcontroller users are familiar with this language. However, to make things simple in this course, we will use another language to program.

Pico: MicroPython.

Install Thonny

The installation of Thonny is very simple. Since Python 3.7 is built into Thonny, all you need is a simple installer and you're ready to start learning to program. First, click Thonny.org to enter the



download page, and select the installation package of Thonny to download in the upper right corner of the page according to your operating system. After downloading the installation package, double-click to open it and follow the on- screen instructions to install it.

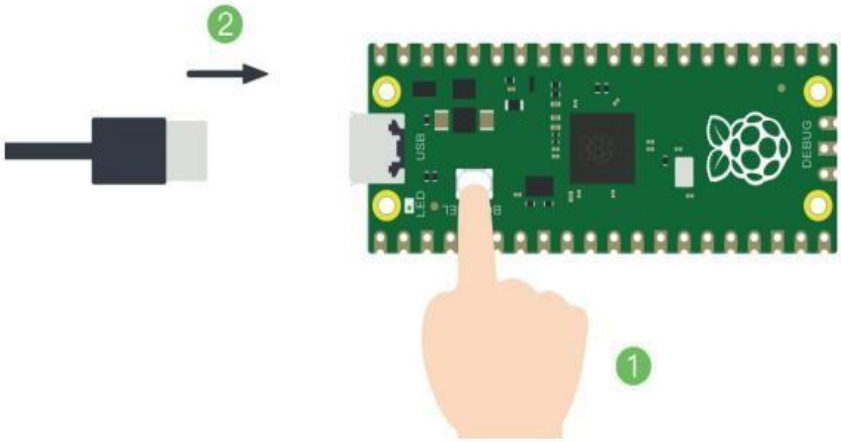
After the installation, open Thonny. You should be directly greeted by the main interface.

Install MicroPython for Pico:

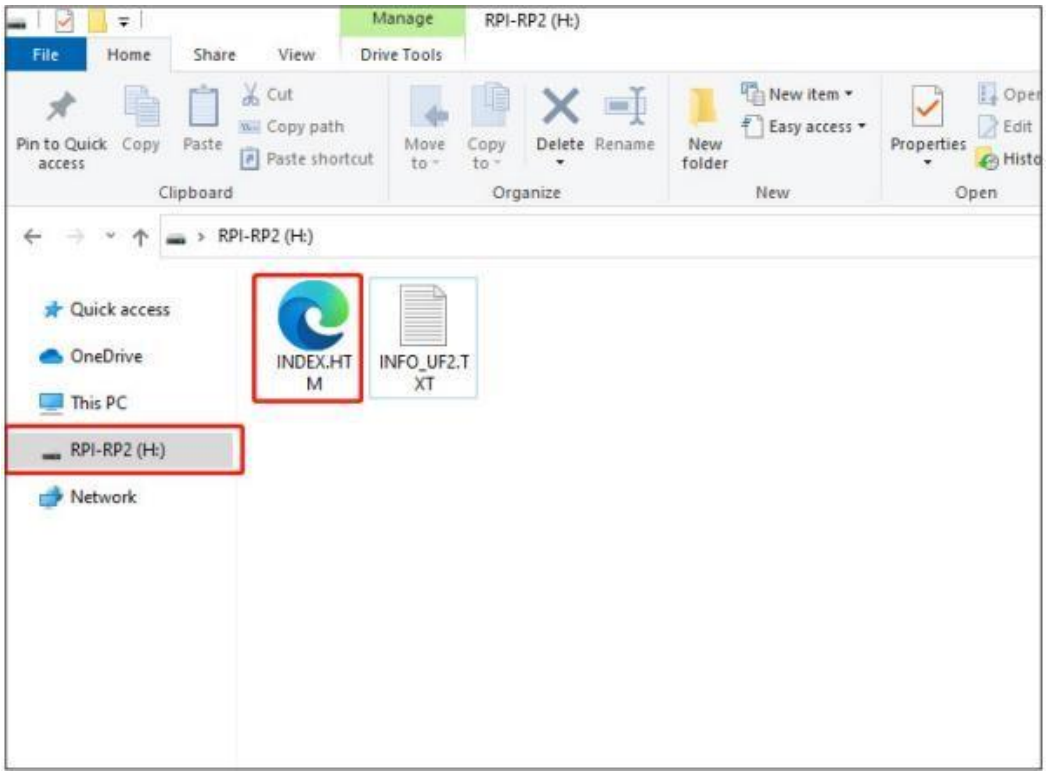
In addition to installing Thonny on the computer, we also need to install the MicroPython firmware on the Pico itself. The MicroPython firmware download link is preloaded inside Pico, so we can directly use this link to enter the download interface.

First, hold down the “BOOTSEL" button on Pico; then, while still holding it down,

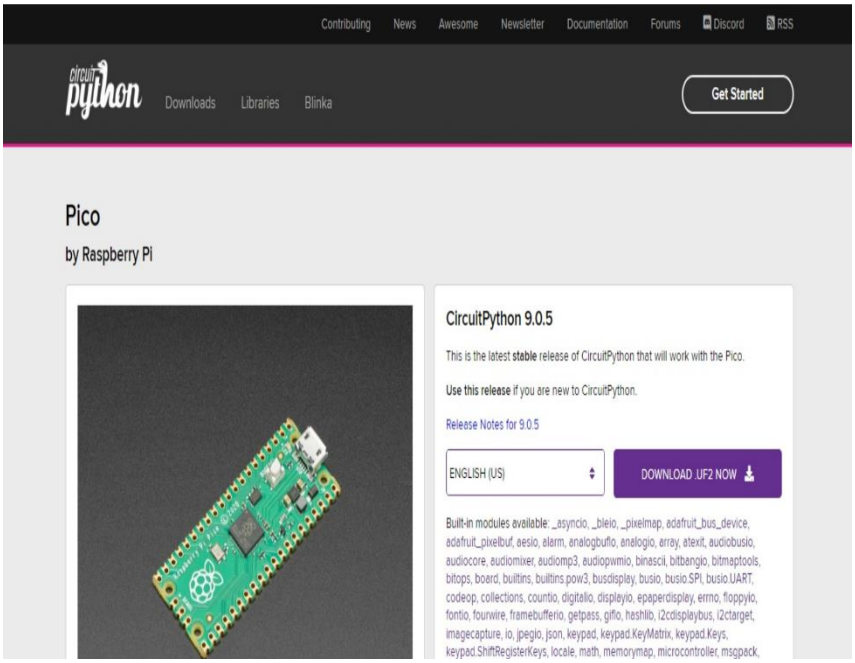
connect Pico to a computer using a USB cable.



At this point, a drive called RPI-RP2 should show up as a removable device on your computer. Navigate to it to find this file: INDEX.HTM.



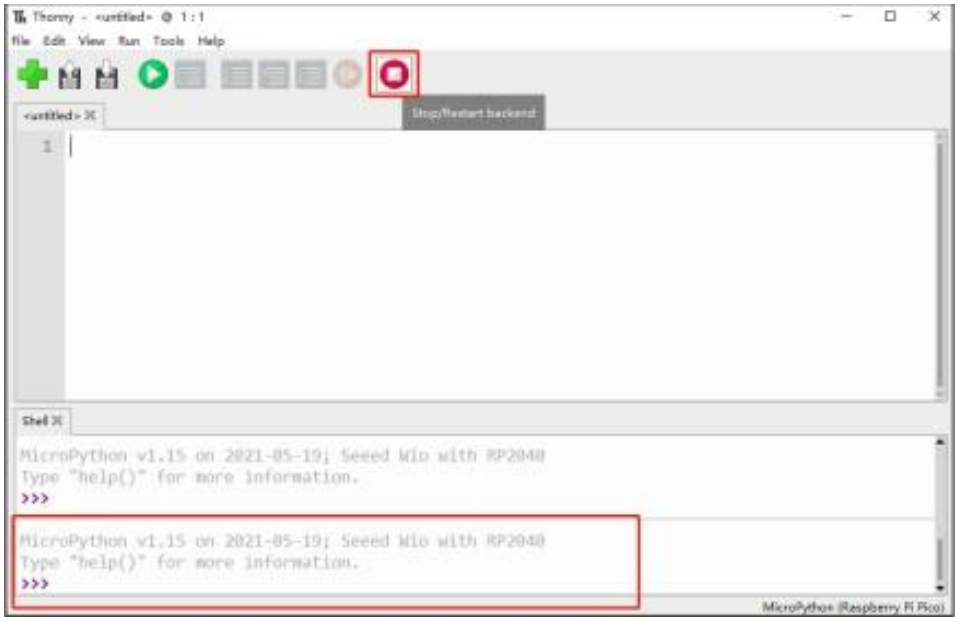
Open INDEX.HTM to be taken to the download page. Drop down and click on the MicroPython tab where you can find information about Pico and MicroPython. Click on the “Download UF2.file" button in this tab to download the MicroPython firmware.



Finally, drag the downloaded MicroPython firmware into the RPI-RP2 directory to complete the firmware installation.

First, connect Pico to the computer using a USB cable; then, open Thonny, and click on

the “restart back-end process" button on the toolbar. If you successfully connect Pico to your computer, you will see the MicroPython version information and device name returned by Pico in the Shell area.



After connected, click on the Shell area and type the instruction:

1. Program to blink an LED.

PIN CONFIGURATIONS	
Microcontrollers	**LED**
ESP32 – GPIO15	Any one in RGB LED
Arduino UNO – D1	Any one in RGB LED
STM32 – PC13	Any one in RGB LED

CODE

```
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(15, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(15, HIGH);  // turn the LED on (HIGH is the voltage level)
  delay(1000);                       // wait for a second
  digitalWrite(15, LOW);   // turn the LED off by making the voltage LOW
  delay(1000);                       // wait for a second
}
```

2. Program to turn on the LED when Push button is Pressed.

PIN CONFIGURATIONS			
Microcontrollers	**Arduino UNO**	**STM32**	**ESP 32**
Push Button Pin (SW1/SW2)	D2	PA15	GPIO 4
RGB LED (C1/C2/C3)	D3	PC13	GPIO13

CODE

```
const int buttonPin = 4; // the number of the pushbutton pin
const int ledPin = 13;      // the number of the LED pin
int buttonState = 0;  // variable for reading the pushbutton status
```

```
void setup() {
// initialize the LED pin as an output:
pinMode(ledPin,OUTPUT);
   // initialize the pushbutton pin as an input:
   pinMode(buttonPin, INPUT);
 }
 void loop() {
   // read the state of the pushbutton value:
   buttonState = digitalRead(buttonPin);
   // check if the pushbutton is pressed. If it is, the buttonState is HIGH:
   if (buttonState == HIGH) {
     digitalWrite(ledPin, LOW); // turn LED off:
   }
   else {
     digitalWrite(ledPin,HIGH); // turn LED on:
   }
}
```

3. Program to turn on the LED when a touch sensor is touched.

PIN CONFIGURATIONS			
	ESP 32	**Arduino UNO**	**STM32**
Touch Sensor output pin	GPIO 4	D4	PA15
RGB LED (C1/C2/C3)	GPIO13	D5	PC13

CODE

```
const int touchpin = 4;
const int ledPin = 13;
int touchState = 0;

void setup() {
   pinMode(ledPin,OUTPUT);
   pinMode(touchpin, INPUT);
 }
 void loop() {
```

```
  touchState = digitalRead(touchPin);
  if (touchState == HIGH) {
    digitalWrite(ledPin, LOW); // turn LED off:
  }
  else {
    digitalWrite(ledPin,HIGH); // turn LED on
  }
}
```

4. Program to display the percentage of light intensity detected by LDR sensor.

PIN CONFIGURATIONS			
	ESP 32	**Arduino UNO**	**STM32**
LDR	GPIO 15	A1	PA8

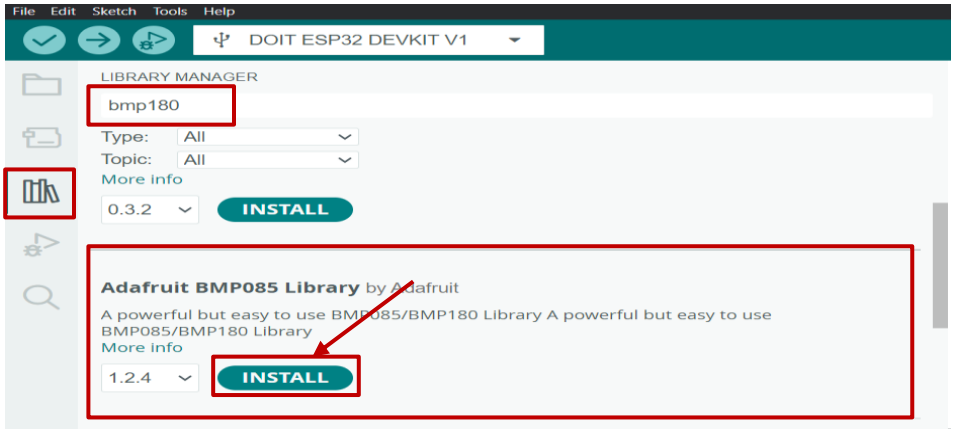
CODE

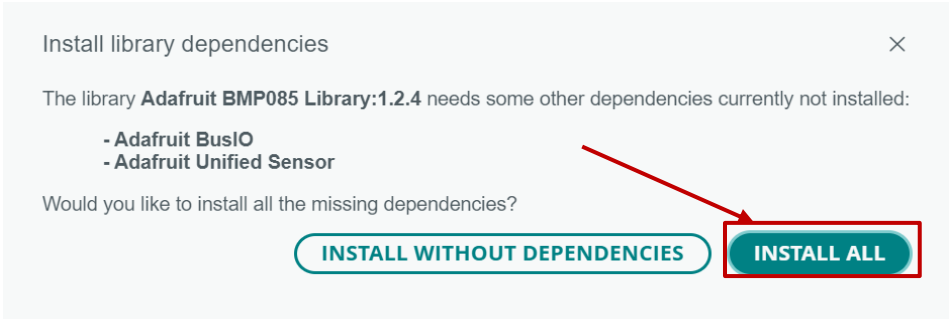
```
#define ldr 15

void setup() {
  // put your setup code here, to run once:
  pinMode(ldr,INPUT);
  Serial.begin(9600);
}
void loop() {
  // put your main code here, to run repeatedly:
  int val=analogRead(ldr);
  Serial.print("Value = ");
  Serial.println(val);
  float per = map(val,0,4095,0,100);
  Serial.print("Percentage : ");
  Serial.println(per);
  delay(1000);
}
```

5. **Program to print temperature and pressure readings from the BMP180 sensor in the Serial Monitor.**

Library Installation





PIN CONFIGURATIONS			
BMP	**ESP 32**	**Arduino UNO**	**STM32**
SDA	GPIO 21	A4	PB7
SCL	GPIO22	A5	PB6

CODE

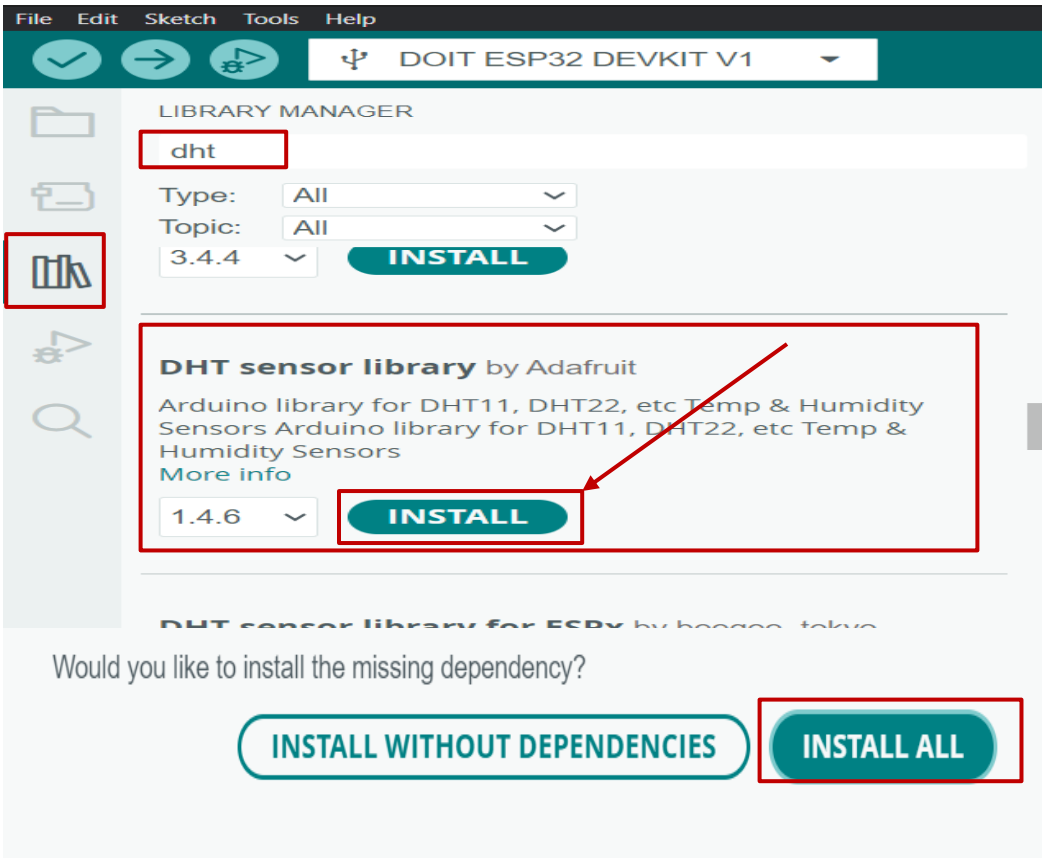
#include <Adafruit_BMP085.h>
Adafruit_BMP085 bmp;
void setup() {
  Serial.begin(9600);
  if (!bmp.begin()) {

```
    Serial.println("Could not find a valid BMP085 sensor, check wiring!");
    while (1) {}
  }
}
void loop() {
  Serial.print("Temperature = ");
  Serial.print(bmp.readTemperature());Serial.println(" *C");
  Serial.print("Pressure = ");
  Serial.print(bmp.readPressure()); Serial.println(" Pa");  Serial.println();
  delay(2000);
}
```

6. Program to print temperature and humidity readings from the DHT11 sensor in the Serial Monitor.

Library Installation:

7.



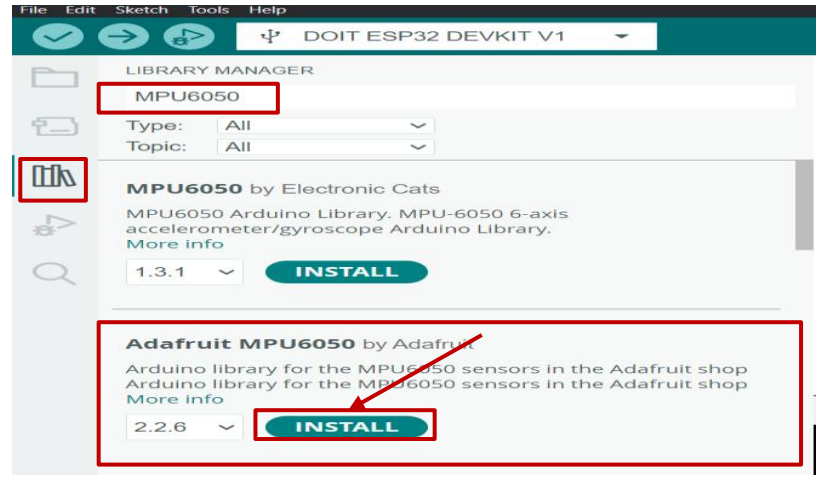
PIN CONFIGURATIONS			
DHT11	**ESP 32**	**Arduino UNO**	**STM32**
Signal/ output pin	GPIO 4	A0	PA8

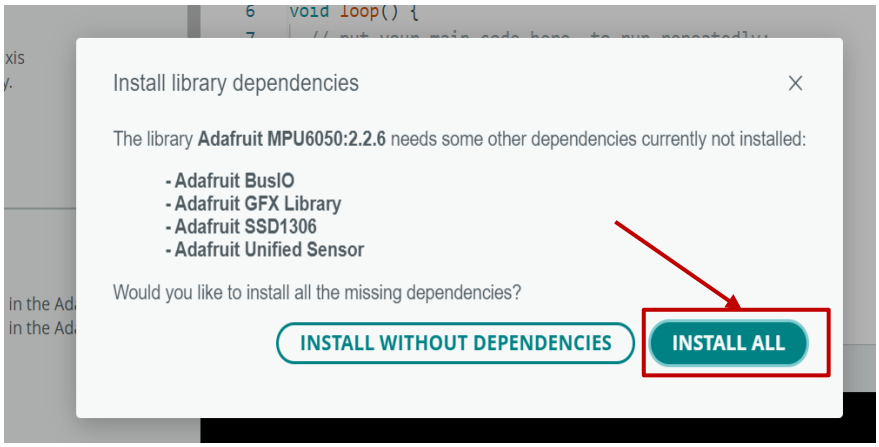
CODE

```
#include "DHT.h"
#define DHT11PIN 4
DHT dht(DHT11PIN, DHT11);
void setup()
{
  Serial.begin(9600);  /* Start the DHT11 Sensor */
  dht.begin();
}
void loop()
{
  float humi = dht.readHumidity();
  float temp = dht.readTemperature();
  Serial.print("Temperature: ");
  Serial.print(temp);
  Serial.println(" C ");
  Serial.print("Humidity: ");
  Serial.println(humi);
  delay(1000);
}
```

7. **Program to print the Gyroscope (X,Y and Z axes) and Accelometer (X,Y and Z axes) values by MPU6050 sensor in Serial Monitor.**

Library Installation





PIN CONFIGURATIONS			
MPU6050	**ESP 32**	**Arduino UNO**	**STM32**
SDA	GPIO 21	A4	PB7
SCL	GPIO22	A5	PB6

CODE

```
// Basic demo for accelerometer readings from Adafruit MPU6050

#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <Wire.h>

Adafruit_MPU6050 mpu;

void setup(void) {
  Serial.begin(115200);
  while (!Serial)
    delay(10); // will pause Zero, Leonardo, etc until serial console opens

  Serial.println("Adafruit MPU6050 test!");

  // Try to initialize!
  if (!mpu.begin()) {
    Serial.println("Failed to find MPU6050 chip");
    while (1) {
      delay(10);
    }
  }
  Serial.println("MPU6050 Found!");

  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
  Serial.print("Accelerometer range set to: ");
  switch (mpu.getAccelerometerRange()) {
  case MPU6050_RANGE_2_G:
    Serial.println("+-2G");
    break;
  case MPU6050_RANGE_4_G:
    Serial.println("+-4G");
    break;
  case MPU6050_RANGE_8_G:
    Serial.println("+-8G");
    break;
  case MPU6050_RANGE_16_G:
```

```
  Serial.println("+-16G");
  break;
 }
 mpu.setGyroRange(MPU6050_RANGE_500_DEG);
 Serial.print("Gyro range set to: ");
 switch (mpu.getGyroRange()) {
 case MPU6050_RANGE_250_DEG:
  Serial.println("+- 250 deg/s");
  break;
 case MPU6050_RANGE_500_DEG:
  Serial.println("+- 500 deg/s");
  break;
 case MPU6050_RANGE_1000_DEG:
  Serial.println("+- 1000 deg/s");
  break;
 case MPU6050_RANGE_2000_DEG:
  Serial.println("+- 2000 deg/s");
  break;
 }

 mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
 Serial.print("Filter bandwidth set to: ");
 switch (mpu.getFilterBandwidth()) {
 case MPU6050_BAND_260_HZ:
  Serial.println("260 Hz");
  break;
 case MPU6050_BAND_184_HZ:
  Serial.println("184 Hz");
  break;
 case MPU6050_BAND_94_HZ:
  Serial.println("94 Hz");
  break;
 case MPU6050_BAND_44_HZ:
  Serial.println("44 Hz");
  break;
 case MPU6050_BAND_21_HZ:
  Serial.println("21 Hz");
  break;
 case MPU6050_BAND_10_HZ:
```

```
    Serial.println("10 Hz");
    break;
  case MPU6050_BAND_5_HZ:
    Serial.println("5 Hz");
    break;
  }

  Serial.println("");
  delay(100);
}

void loop() {

  /* Get new sensor events with the readings */
  sensors_event_t a, g, temp;
  mpu.getEvent(&a, &g, &temp);

  /* Print out the values */
  Serial.print("Acceleration X: ");
  Serial.print(a.acceleration.x);
  Serial.print(", Y: ");
  Serial.print(a.acceleration.y);
  Serial.print(", Z: ");
  Serial.print(a.acceleration.z);
  Serial.println(" m/s^2");

  Serial.print("Rotation X: ");
  Serial.print(g.gyro.x);
  Serial.print(", Y: ");
  Serial.print(g.gyro.y);
  Serial.print(", Z: ");
  Serial.print(g.gyro.z);
  Serial.println(" rad/s");

  Serial.print("Temperature: ");
  Serial.print(temp.temperature);
  Serial.println(" degC");

  Serial.println("");
```

```
  delay(500);
}
```

8. Program to print the distance between the sensor and the object by Ultrasonic sensor in Serial Monitor.

PIN CONFIGURATIONS			
Ultrasonic	**ESP 32**	**Arduino UNO**	**STM32**
Echo	GPIO 21	A2	PA8
Trig	GPIO 22	A3	PB5

CODE

```
#define TRIG 22
#define ECHO 21

//define sound speed in cm/uS
#define SOUND_SPEED 0.0343
#define CM_TO_INCH 0.393701

long duration;
float distanceCm;
float distanceInch;

void setup() {
  Serial.begin(9600); // Starts the serial communication
  pinMode(TRIG, OUTPUT); // Sets the trigPin as an Output
  pinMode(ECHO, INPUT); // Sets the echoPin as an Input
}

void loop() {
  // Clears the trigPin
  digitalWrite(TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH);   // Sets the trigPin on HIGH state for 10 ms
  delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  duration = pulseIn(ECHO, HIGH); // Reads the sound wave travel time in ms
  distanceCm = duration * SOUND_SPEED/2; // Calculate the distance
  distanceInch = distanceCm * CM_TO_INCH; // Convert to inches
  Serial.print("Distance (cm): ");
```

```
  Serial.println(distanceCm);
  Serial.print("Distance (inch): ");
  Serial.println(distanceInch);
  delay(1000);
}
```

9. Program to print the gas concentration detected by the MQ135 sensor in the Serial Monitor.

PIN CONFIGURATIONS			
MQ135	**ESP32**	**Arduino UNO**	**STM32**
A0	GPIO 15	A0	PB9

CODE

```
void setup() {
  // initialize serial communication at 9600 bits per second:
  Serial.begin(9600);
}

// the loop routine runs over and over again forever:
void loop() {
  // read the input on analog pin 0:
  int sensorValue = analogRead(15);
  // print out the value you read:
  Serial.println(sensorValue);
  delay(1000);  // delay in between reads for stability
}
```

10. Program to Detect the object by IR sensor and print the value in Serial Monitor.

PIN CONFIGURATIONS			
IR Sensor	**ESP32**	**Arduino UNO**	**STM32**
Output pin	GPIO4	D9	PA7

CODE

```
#define IR 4
void setup() {
  // put your setup code here, to run once:
  pinMode(IR,INPUT);
  Serial.begin(9600);
}

void loop() {
  // put your main code here, to run repeatedly:
  int ir = digitalRead(IR);
if(ir == 1)
Serial.println("Object Not Detected");
else
Serial.println("Object Detected");
delay(1000);
}
```

11. Program to turn on the Relay using Push Button.

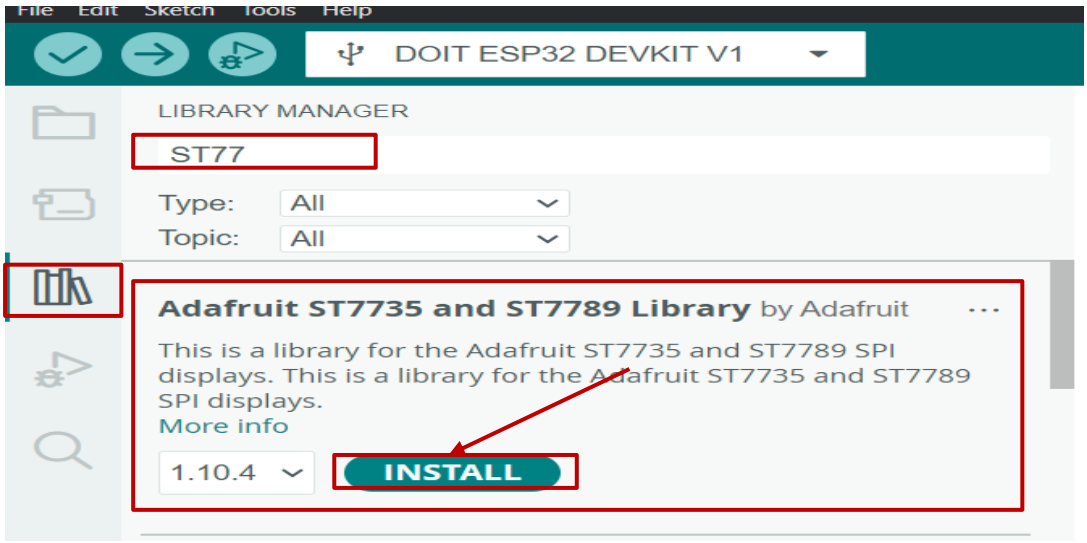
PIN CONFIGURATIONS			
Relay1/Relay2	**ESP32**	**Arduino UNO**	**STM32**
Relaypin 1/Relaypin 2	GPIO27	A0	PA7
Push Button 1/Push Button 2	GPIO22	A1	PA6

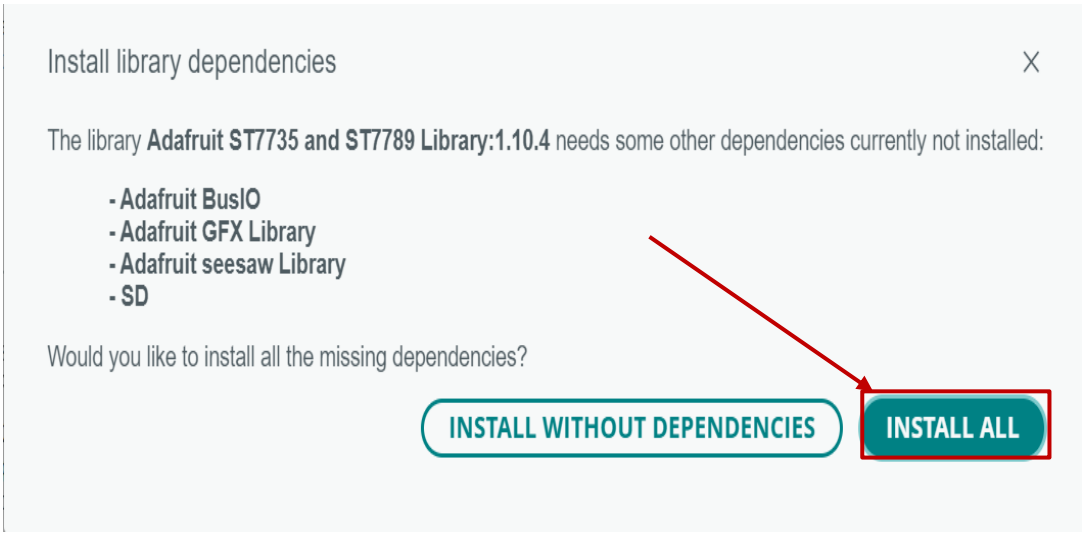
CODE

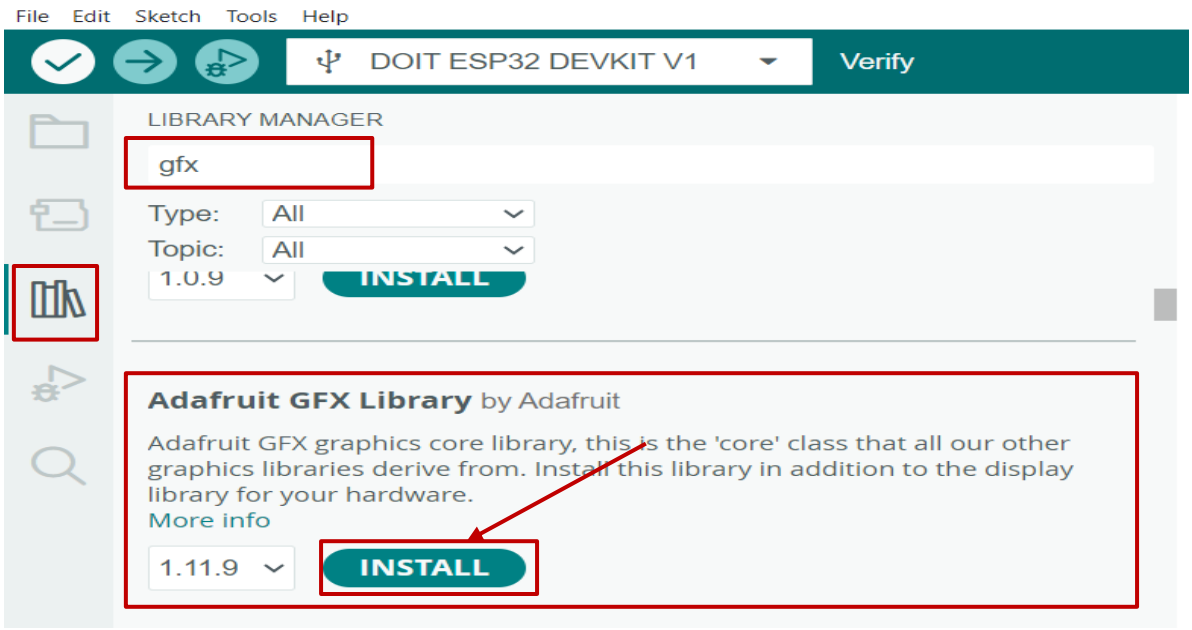
```
#define BUTTON_PIN 22 // In ESP32 pin GPIO22 connected to button's pin
#define RELAY_PIN 27 //In ESP32 pin GPIO27 connected to relay's pin
void setup() {
Serial.begin(9600); // initialize serial
pinMode(BUTTON_PIN, INPUT_PULLUP); pinMode(RELAY_PIN,
OUTPUT); // set ESP32 pin to output mode
}
void loop() {
int buttonState = digitalRead(BUTTON_PIN); // read new state
if (buttonState == LOW) {
Serial.println("The button is being pressed");
digitalWrite(RELAY_PIN, HIGH); // turn on
}
else if (buttonState == HIGH) {
Serial.println("The button is unpressed");
digitalWrite(RELAY_PIN, LOW); // turn off
}
}
```

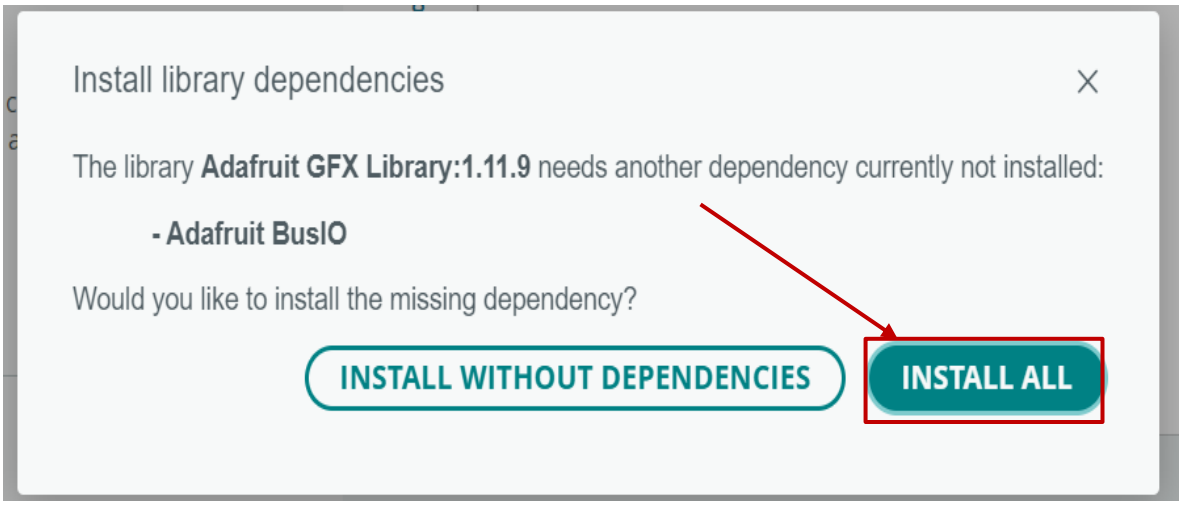
12. Program to display temperature and humidity values on the TFT Display.

Library Installation:









PIN CONFIGURATIONS			
TFT Display	**ESP32**	**Arduino UNO**	**STM32**
LED	5V	5V	3.3V
SDA	GPIO21	A4	PA5
SCL	GPIO22	A5	PA7
A0	GPIO13	D9	PA2
RES	GPIO12	D8	PA3
CS	GPIO14	D10	PA4

CODE

```
#include "Arduino.h"
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_ST7735.h>
#include <SPI.h>
#include <DHT.h>

#define DHTPIN 4
#define DHTTYPE DHT11
#define TFT_CS     14
#define TFT_RST    12
#define TFT_DC     13  //A0
#define TFT_SCLK   SCL   //SCK //CLK //22
#define TFT_MOSI   SDA   //SDA //DIN //21

DHT dht(DHTPIN, DHTTYPE);
Adafruit_ST7735 tft = Adafruit_ST7735(TFT_CS, TFT_DC,
TFT_MOSI, TFT_SCLK, TFT_RST);

float t;
float h;

void setup() {

  Serial.begin(9600);
  dht.begin();
  tft.initR(INITR_BLACKTAB);                /* initialize a ST7735S chip,
black tab */
  tft.setTextWrap(false);                  /* Allow text to run off right edge
*/
  tft.fillScreen(ST7735_YELLOW);
}
```

```
void loop() {
  Dhtread();
  Display();
  delay(100);
}
void Dhtread(void)
{
  delay(1000);
  h = dht.readHumidity();
  t = dht.readTemperature();
  Serial.print("Humidity : ");
  Serial.print(h);
  Serial.print("%  Temperature : ");
  Serial.println(t);
  }

void Display(void) {
  tft.fillScreen(0x0000);  /* BLACK -- Background */
  tft.setRotation(1);
  //tft.serRotation(2); //for Vertical Display
  tft.setCursor(10, 20);
  tft.setTextColor(ST7735_YELLOW);
  tft.setTextSize(2);
  tft.println("Temp C:  ");

  tft.setCursor(110, 20);
  tft.setTextColor(ST7735_YELLOW);
  tft.setTextSize(2);
  tft.println( t);

  tft.setCursor(10, 50);
  tft.setTextColor(ST7735_YELLOW);
  tft.setTextSize(2);
  tft.println("Hum %:  ");

  tft.setCursor(90, 50);
  tft.setTextColor(ST7735_YELLOW);
  tft.setTextSize(2);
  tft.println( h);
```

```
  delay(1000);
}
```

13. Program to turn on the LED using Bluetooth Communication.

PIN CONFIGURATIONS			
	ESP32	**Arduino UNO**	**STM32**
RGB	GPIO 2	D10	P13

CODE

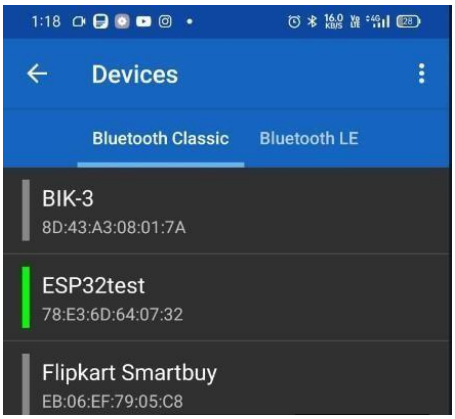
```
#include "BluetoothSerial.h"
#define ledPin 2
BluetoothSerial SerialBT;
String message = "";
char incomingChar;
void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(115200);
  SerialBT.begin("ESP32"); //Bluetooth device name
  Serial.println("The device started, now you can pair it with bluetooth!");
}
void loop() {
 if (SerialBT.available()){
    char incomingChar = SerialBT.read();
    if (incomingChar != '\n'){
      message += String(incomingChar);
    }
    else{
      message = "";
      }
      Serial.write(incomingChar);
  }
  if (message =="on"){
   digitalWrite(ledPin, HIGH);
  } else if (message =="off"){
   digitalWrite(ledPin, LOW);
  }
  delay(20);
}
```

To Experience the Outcome

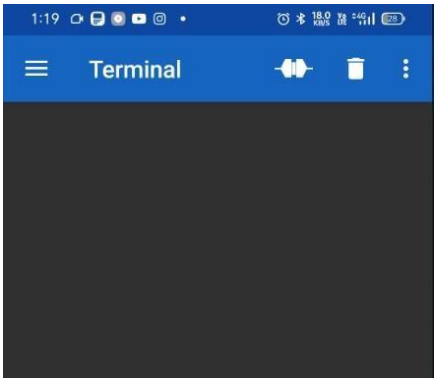
- Download & install the apk file of “BluetoothSerial” form Google Play store.



- Open the Mobile Application and Turn ON the Bluetooth in yourMobile toScan List the ESP32 Devices

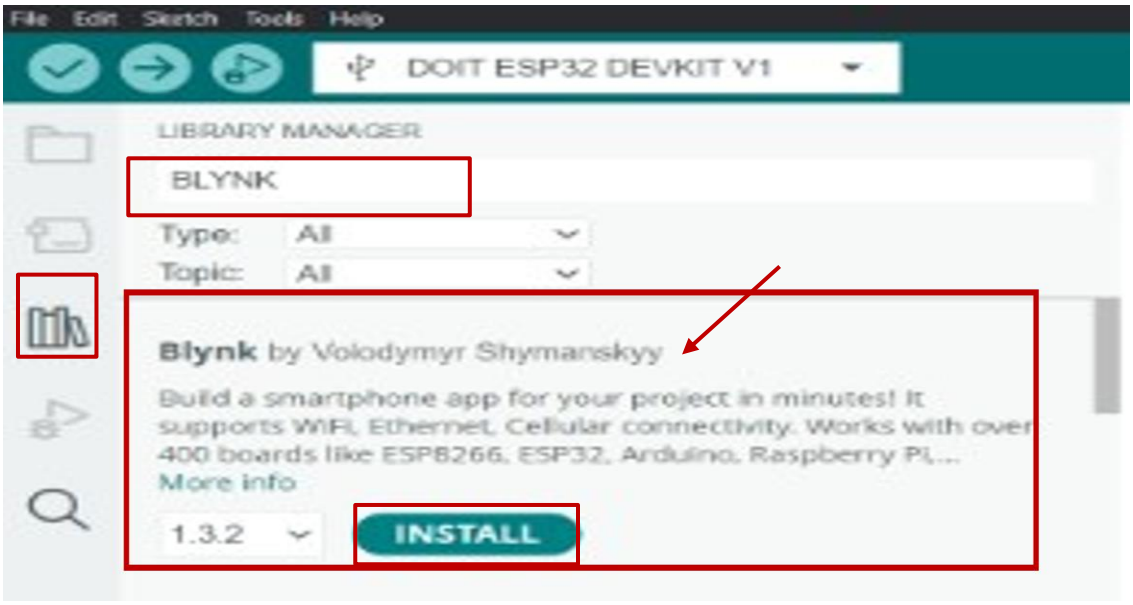


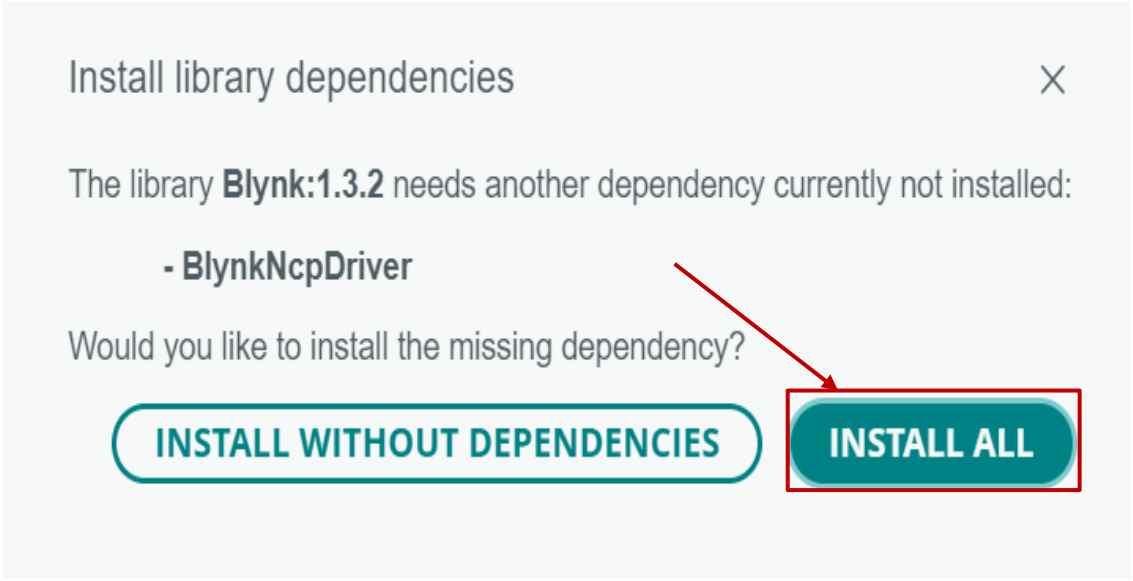
- Make Sure it is Connected



14. Program to turn on the LED using WiFi Communication.

Library Installation.





PIN CONFIGURATIONS			
RGB	**ESP32**	**Arduino UNO**	**STM32**
R	GPIO 2	D11	PC13

CODE

```
#define BLYNK_TEMPLATE_ID "Enter Blyk ID"
#define BLYNK_DEVICE_NAME "LED CONTROL"
#define BLYNK_AUTH_TOKEN "Enter Auth Token"

#define BLYNK_PRINT Serial
#include <WiFi.h>
#include  <WiFiClient.h>
 #include <BlynkSimpleEsp32.h>

char auth[] = BLYNK_AUTH_TOKEN;
char ssid[] = "Type Your SSID"; // Enter your Wifi Username
char pass[] = "Type your Pass"; // Enter your Wifi password

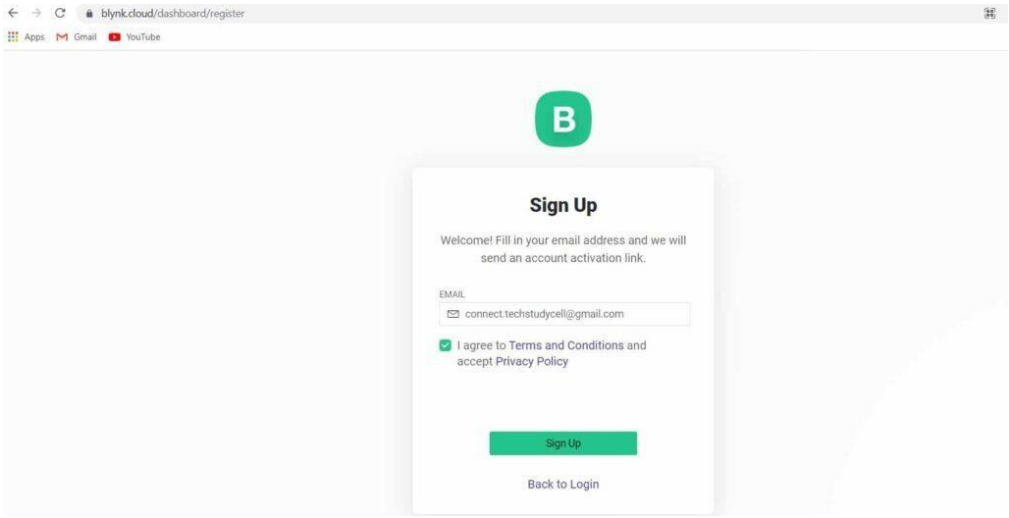
int ledpin = 2; void setup()
{
  Serial.begin(9600);
  Blynk.begin(auth, ssid, pass);
  pinMode(ledpin,OUTPUT);
}

void loop()
{
  Blynk.run();
}
```

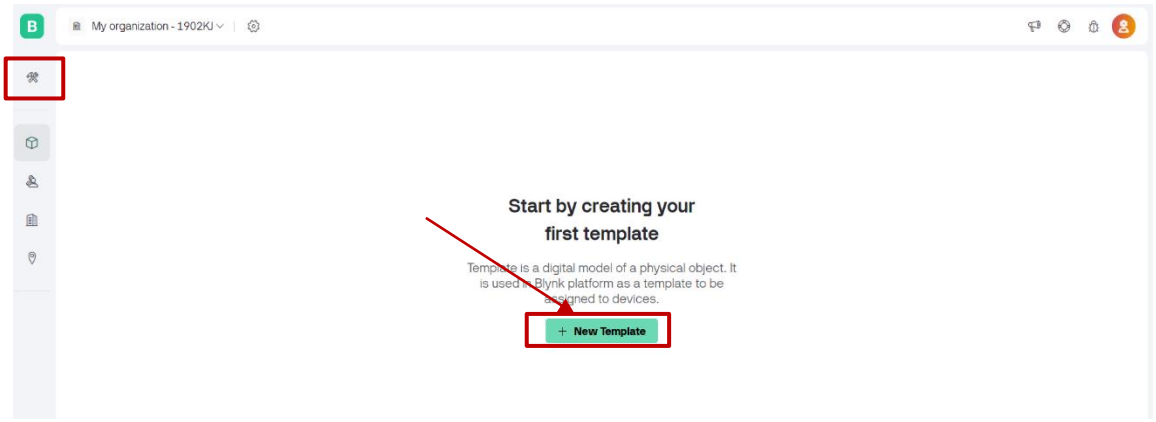
To Experience the Outcome:

1. Open the URL https://blynk.cloud/

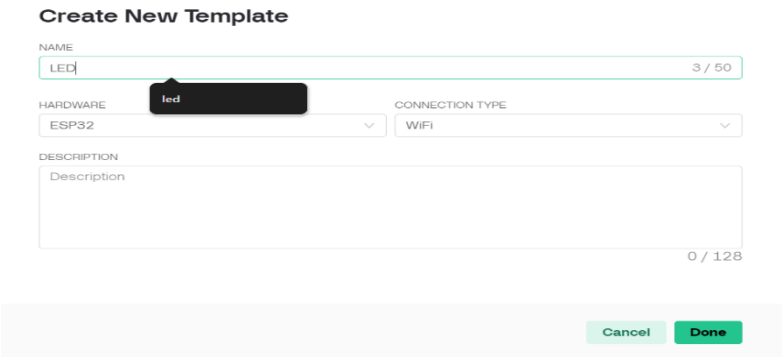
2. Signup in to the



3. Create Template in Blynk IoT Cloud

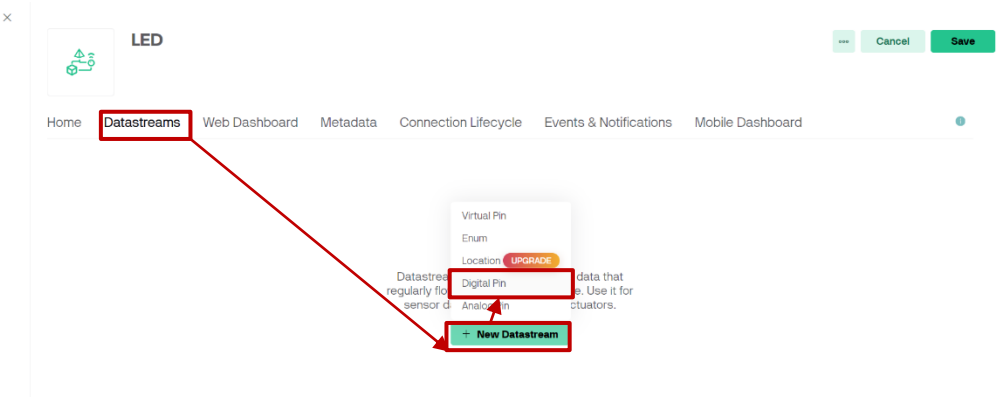


4. After login to Blynk IoT account, first you have create a template. In the dashboard click onthe Templates from the left side menu.

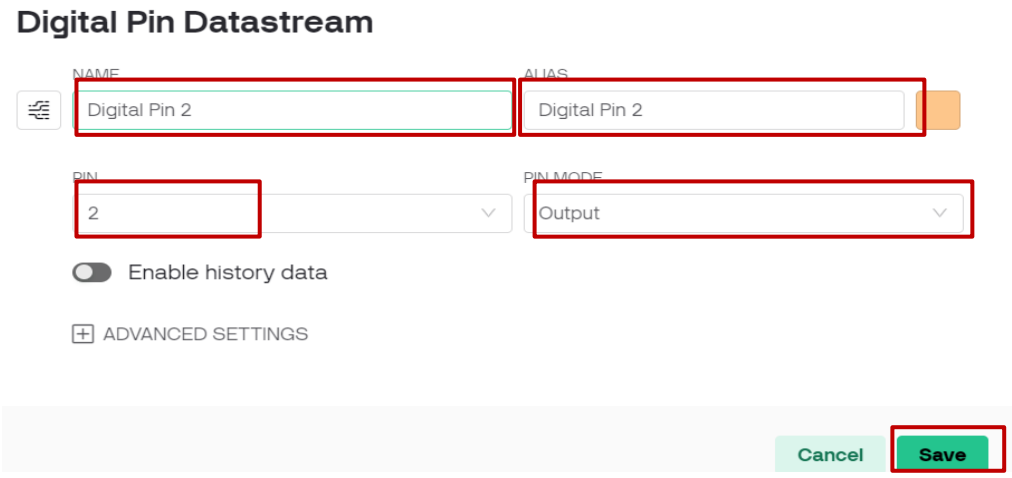


5. Then click on New Template.

6. Then, you have to enter a name for that template.
7. Select the microcontroller or board you will use in the project from the Hardware dropdown menu.
8. The connection type should be WiFi.
9. You can enter small description for the template. This field is optional.
10. Now click on Done.

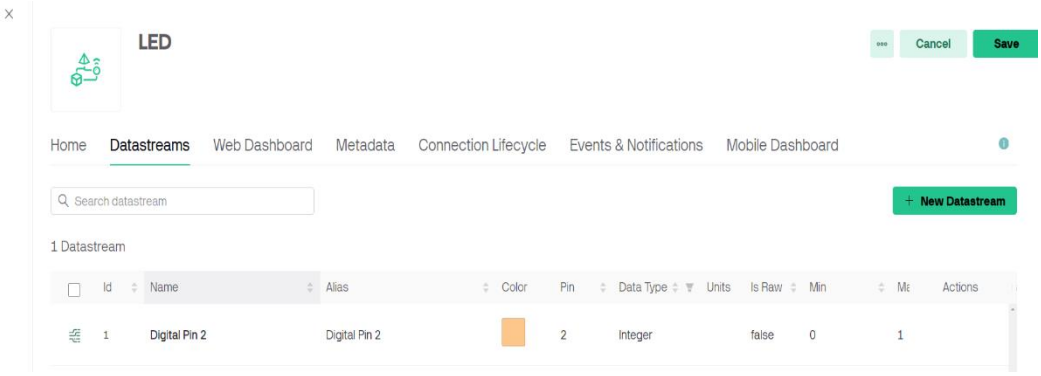


11. Create Datastreams in the Blynk IoT template
12. After creating the template, go to Datastreams tab.



13. Then click on "New Datastream" and select Digital Pin.
14. Now, you have to enter a name for this Datastream.

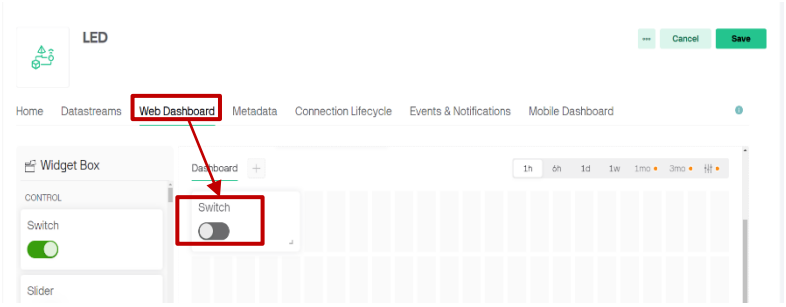
15. Select the Digital Pin and Pinmode from the dropdown menu as per the



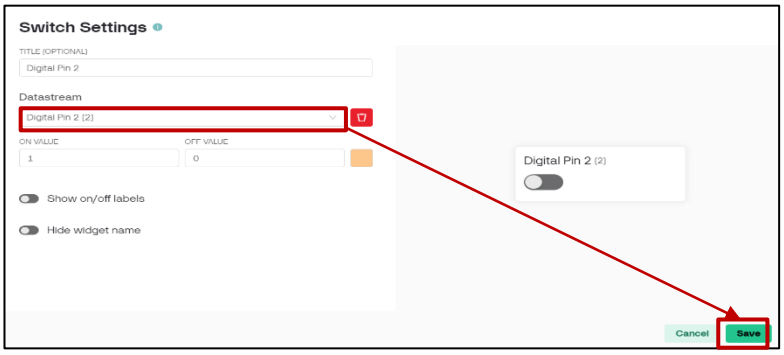
requirement.

16. After that click on Create.

Create Web Dashboard in the Blynk IoT template



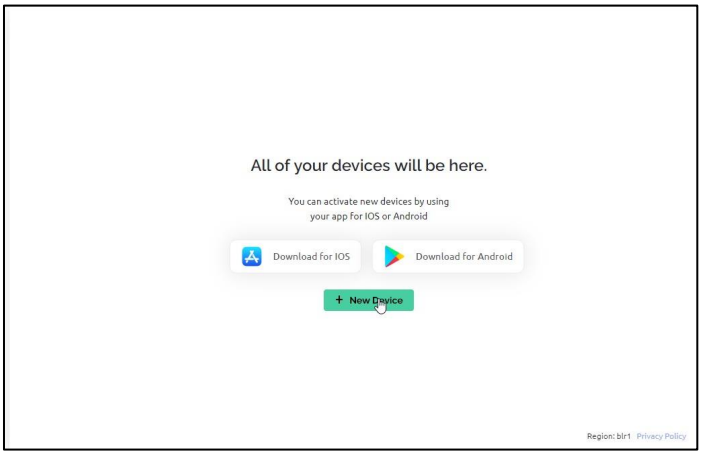
17. Now, go to the Web Dashboard tab to add widgets.
18. In the dashboard, you can easily drag and drop switch widget from the left side.
19. Now, if you hover over the widget, you will find a button with a gear

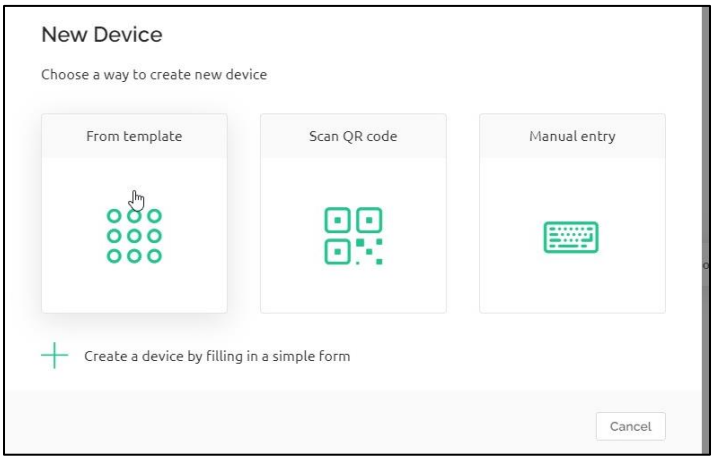


icon. Click on thebutton to go to setting.

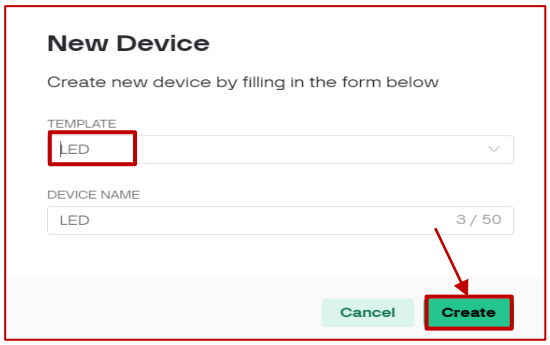
20. Click on Save.
21. Go to Organization click on “New Device”

22. Choose from Template

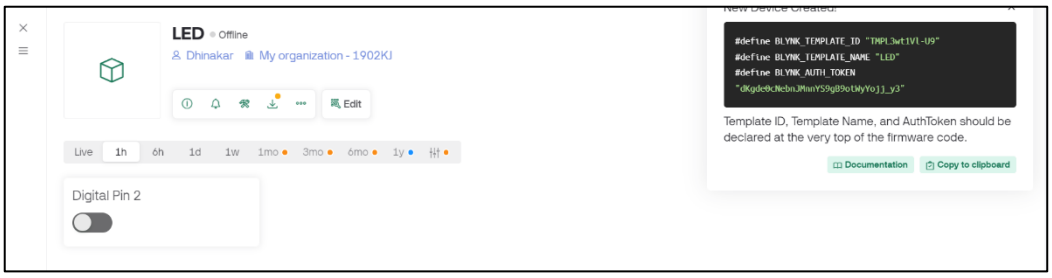




23. Select your devices



24. In the code, you just have to update the BLYNK_TEMPLATE_ID, BLYNK_DEVICE_NAME &BLYNK_AUTH_TOKEN.



15. **Program to send the data from Microcontroller to Serial Port utility application by using RS232(UART Protocol).**

PIN CONFIGURATIONS			
RS232	**ESP32**	**Arduino UNO**	**STM32**
DI	GPIO17	D1	PA9
RO	GPIO16	D0	PA10

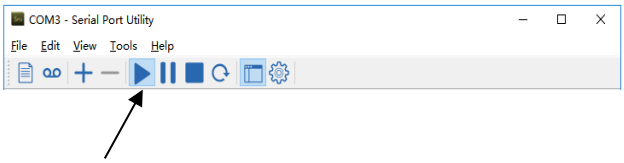
CODE

```
#define RXD2 16   // RS232 TX TO ESP32 GPIO16
#define TXD2 17   // RS232 TX TO ESP32 GPIO17
void setup() {
  // Note the format for setting a serial port is as follows: Serial2.begin(baud-rate, protocol, RX pin, TX pin);
  Serial.begin(9600);
  //Serial1.begin(9600, SERIAL_8N1, RXD2, TXD2);
  Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2);
  Serial.println("Serial Txd is on pin: "+String(TX));
  Serial.println("Serial Rxd is on pin: "+String(RX));
}
void loop() { //Choose Serial1 or Serial2 as required
  if(Serial2.available()>0) {
    Serial.print(char(Serial2.read()));

    }
    if(Serial.available()>0) {
    Serial2.print(char(Serial.read()));
    }
}
```

Procedure:

- Download “serial Port utility” from google.
  https://serialport.en.softonic.com
- After downloading install the serial port utility.
- Select the com port in the serial port utility.
- Baud rate should be same in the serial port utility as per coding.
- Click the “START” icon.



- After clicking start icon, you can send data or receive the data.

16. **Program to control LED using Serial Port utility application by using RS485(UART Protocol).**

PIN CONFIGURATIONS			
RS485	**ESP32**	**Arduino UNO**	**STM32**
DI	GPIO17	D1	PA9
RO	GPIO16	D0	PA10
RE & DE	GPIO4	D2	PA15
LED	GPIO15	D3	PA11

RS485	**CONNECTR**
A	A
B	B

CODE

```
 #include <HardwareSerial.h>

// Define the serial port and the control pin for RS485
HardwareSerial RS485Serial(3);

#define RS485ControlPin 4//  DE/RE to GPIO 4 , DI to GPIO 17 , RO to GPIO 16
#define LEDPin  15

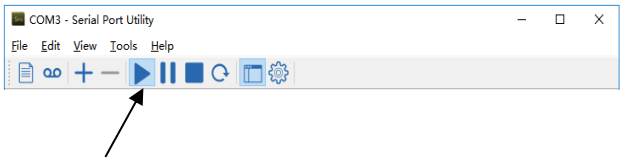
void RS485Transmit() {
  digitalWrite(RS485ControlPin, HIGH);
}

void RS485Receive() {
  digitalWrite(RS485ControlPin, LOW);
}
```

```
void setup() {
  Serial.begin(9600);
  // Initialize the RS485 control pin
  pinMode(RS485ControlPin, OUTPUT);
  RS485Receive();
  // Initialize LED pin
  pinMode(LEDPin, OUTPUT);
  // Begin serial communication at a baud rate of 9600
  RS485Serial.begin(9600, SERIAL_8N1, 16, 17);
}
void loop() {
  // Example code to send data
  RS485Transmit();
  delay(100); // Short delay
  RS485Receive();
  String message = "";
  // Example code to receive data
  if(RS485Serial.available()){
    while(RS485Serial.available()>0) {
      char receivedData = RS485Serial.read();
      if (receivedData != '\n'){
        message += String(receivedData);
      }
    }
    // Trim the message to remove any extra spaces or newline characters
    message.trim();
    // Debug print to see the exact message received
    Serial.println("Message received: '" + message + "'");
    // Processing the received message
    if (message == "1") {
      digitalWrite(LEDPin, HIGH); // Turn on LED
      Serial.println("LED Turned on");
    } else if (message == "0") {
      digitalWrite(LEDPin, LOW); // Turn off LED
      Serial.println("LED Turned off");
    }
  }
  delay(1000); // Adjust the delay as needed
}
```

Procedure:

- Download “serial Port utility” from google.
  https://serialport.en.softonic.com
- After downloading install the serial port utility.
- Select the com port in the serial port utility.
- Baud rate should be same in the serial port utility as per coding.
- Click the “START” icon.



- After clicking start icon, you can send data or receive the data.

CAN CONTROLLER CONFIGURATION:

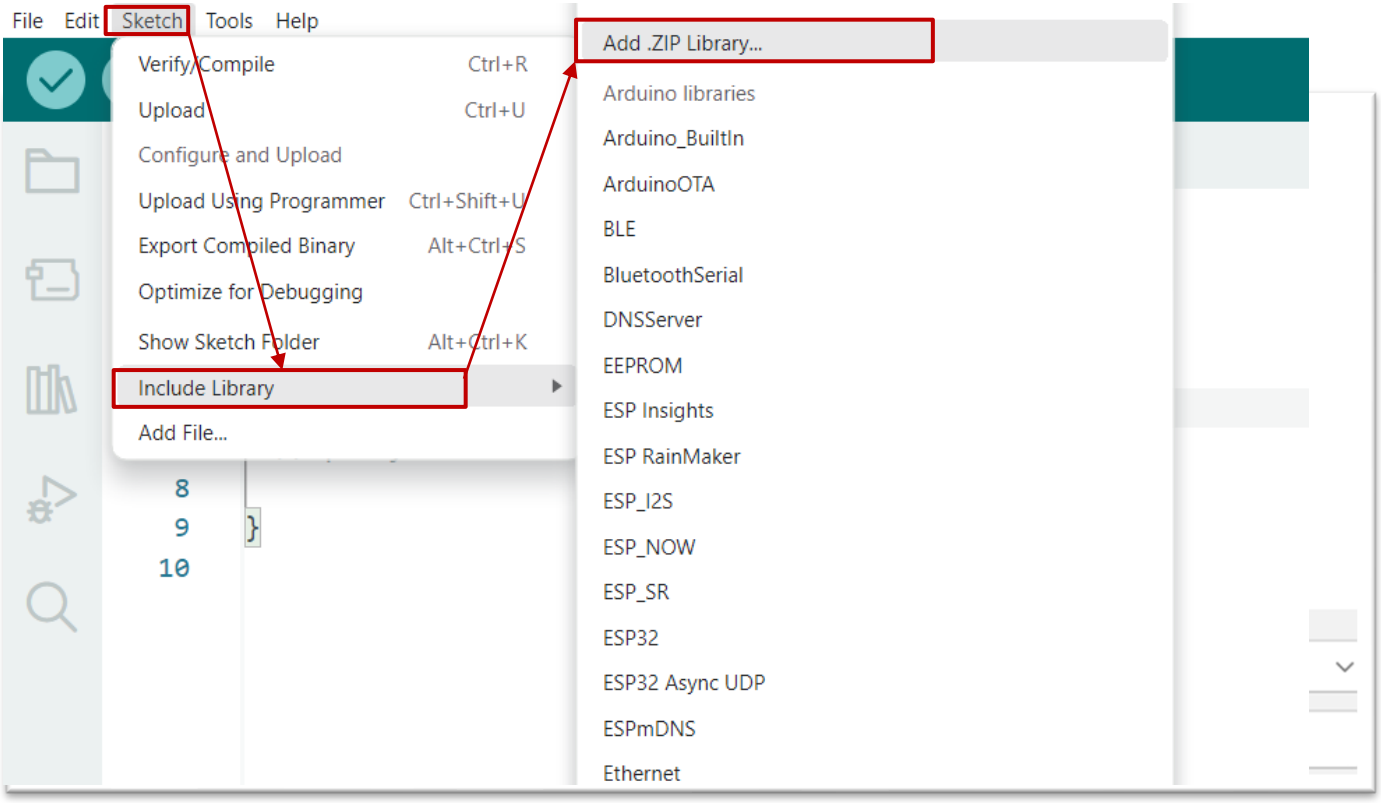
CAN Controller Tx	CAN Controller Rx
CAN H	CAN H
CAN L	CAN L

Header file for Program:

Step1: Click on the below link, scroll down and download the ZIP file.https://github.com/autowp/arduino-mcp2515

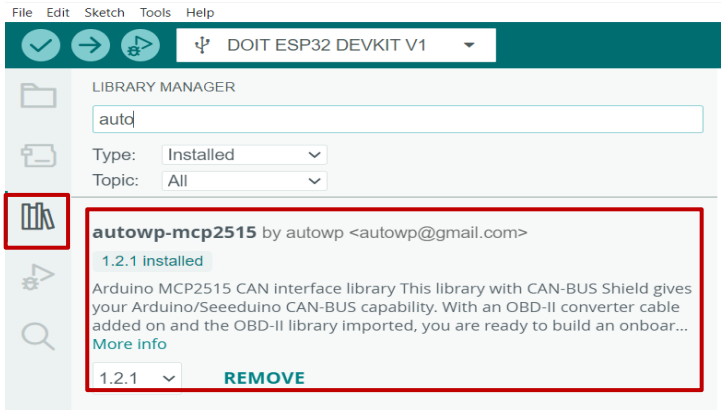


Step2: Open Arduino IDE, click on ‘Sketch’ option. In ‘Sketch’ click on ‘Include Library’option. In ‘Include Library’ click on ‘Add .ZIP Library’.



Step3: Locate the Downloaded “arduino-mcp2515-master” ZIP file and click on open.

Step4: Ater the resources been downloaded, go to library and search mcp2515 and Install the library provided by ‘autowp’.



17. Interfacing CAN Controller with BMP180 to transmit "BMP180 data" from Point-Point using Arduino Uno.

Arduino Uno	CAN Controller	TFT with Arduino UNO Rx	BMP 180 with Arduino UNO Tx
3.3V	VCC	VCC & LED	3.3V
GND	GND	GND	GND
SCL		SCL	SCL
SCK		SCK	SCK
Pin 2		A0	
Pin 3		RST	
Pin 4		CS	
Pin 10	CS		
Pin 11	SI		
Pin 12	SO		
Pin 13	SCK		

TRANSMITTER CODE

```
#include <SPI.h>
#include <mcp2515.h>    //Library for using CAN Communication
#include <Adafruit_BMP085.h>
Adafruit_BMP085 bmp;

struct can_frame canMsg;
MCP2515 mcp2515(10); // chip select CS pin 10

void setup()
{
  Serial.begin(9600);
  SPI.begin(); //Begins SPI communicationif
  (!bmp.begin()) {
        Serial.println("Could not find a valid BMP085 sensor, check wiring!");
        while (1) {}
```

```
  }
  mcp2515.reset();
  mcp2515.setBitrate(CAN_500KBPS,MCP_8MHZ); //Sets CAN at speed 500KBPS and
  mcp2515.setNormalMode();

  canMsg.can_id =0x036;//CAN id 0x036
  //CAll data length as i byte
  canMsg.can_dlc =2;
}

void loop()
  {
    float pres = bmp.readPressure(); float
    temp = bmp.readTemperature();
    Serial.println("\nCAN Transmitter    \n");
    Serial.print("Temperature: ");
    Serial.print(temp);
    Serial.println("C ");
    Serial.print("Pressure: ");
    Serial.println(pres);

    canMsg.data[0] = temp; canMsg.data[1]
    = pres;
    mcp2515.sendMessage(&canMsg);delay
    (1000);
  }
```

RECEIVER CODE

```
#include  <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_ST7735.h>
#include <SPI.h>
#include <mcp2515.h>
struct can_frame canMsg;
MCP2515 mcp2515(10);
#define TFT_CS 4
#define TFT_RST 3
 #define TFT_DC  2  //A0
```

```
#define TFT_SCLK SCL //SCK
#define TFT_MOSI  SDA  //SDA
Adafruit_ST7735 tft = Adafruit_ST7735(TFT_CS, TFT_DC, TFT_MOSI, TFT_SCLK,
TFT_RST);
void setup() {
Serial.begin(9600);
SPI.begin();
tft.initR(INITR_BLACKTAB);
 tft.setTextWrap(false);
tft.fillScreen(ST7735_BLACK);
  mcp2515.reset();
  mcp2515.setBitrate (CAN_500KBPS,MCP_8MHZ); //Sets CAll at speed 500KBPS
andClock
  mcp2515.setNormalMode(); //Sets Call at normal mode

}
void loop() {
  if (mcp2515.readMessage(&canMsg) == MCP2515:: ERROR_OK) // To receive data
(Poll Read){
     if (canMsg.can_id==0x036)
     {
       int t=canMsg.data[0];int
       p=canMsg.data[1];
       tft.fillScreen(ST7735_BLACK); /* BLACK -- Background */
       tft.setRotation(1);
       tft.setCursor(0, 30);
       tft.setTextColor(ST7735_YELLOW);
       tft.setTextSize(1.5); tft.print("Temperature
       ==> "); tft.println(t);
       tft.print("Pressure ==> ");
       tft.println(p);
       Serial.println("\nCAN Receiver.    \n");
       Serial.print("Temperature ==> ");
       Serial.println(t); Serial.print("Pressure
       ==> "); Serial.println(p);
       delay(1000);
     }
   }
  }
```

18. Program to send the DHT11 data from Microcontroller to Mobile by using 4G Module(UART Protocol).

PIN CONFIGURATIONS			
4G Module	**ESP32**	**Arduino UNO**	**STM32**
DI	GPIO17	D1	PA9
RO	GPIO16	D0	PA10

Note: Install Dht library before uploading the code.

CODE

```
#include <HardwareSerial.h>
#include <DHT.h>

// DHT11 Sensor settings
#define DHTPIN 4          // Pin where the DHT11 is connected
#define DHTTYPE DHT11     // DHT 11 sensor type
DHT dht(DHTPIN, DHTTYPE);

// Define the hardware serial port for A7670C
HardwareSerial mySerial(1);

// Define the phone number to send the message to
const char* phoneNumber = "+91YOUR_MOBILE_NUMBER";

void setup() {
  // Start serial communication for debugging
  Serial.begin(115200);

  // Start hardware serial for the A7670C module
  mySerial.begin(115200, SERIAL_8N1, 16, 17); // RX, TX pins

  // Initialize the DHT11 sensor
  dht.begin();

  // Check if the module is responding
  sendATCommand("AT", 1000); // Send AT to check communication

  // Check if the module is registered on the network
```

```
  sendATCommand("AT+CREG?", 2000); // Check network registration

  // Check signal strength
  sendATCommand("AT+CSQ", 2000); // Check signal quality

  // Set SMS mode
  sendATCommand("AT+CMGF=1", 1000); // Set SMS to text mode
}

void loop() {
  // Read humidity and temperature from the DHT11 sensor
  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();

  // Check if the readings are valid
  if (isnan(humidity) || isnan(temperature)) {
    Serial.println("Failed to read from DHT sensor!");
    return;
  }

  // Create the message with the sensor data
  String message = "DHT11 Sensor Data:\n";
  message += "Temperature: " + String(temperature) + " C\n";
  message += "Humidity: " + String(humidity) + " %";

  // Send the SMS with DHT11 data
  sendSMS(phoneNumber, message.c_str());

  // Wait for 10 minutes before sending the next SMS
  delay(10000); // 600000 ms = 10 minutes
}

void sendSMS(const char* number, const char* text) {
  // Construct the AT command to send SMS
  String command = "AT+CMGS=\"";
  command += number;
  command += "\"";

  // Send the command to the module
```

```
  sendATCommand(command.c_str(), 1000);

  // Send the actual message text
  mySerial.print(text);
  delay(100);

  // Send Ctrl+Z (ASCII 26) to indicate the end of the message
  mySerial.write(26); // Ctrl+Z
  delay(3000); // Wait for the message to send

  // Read and print the response from the module
  while (mySerial.available()) {
    Serial.write(mySerial.read());
  }
}

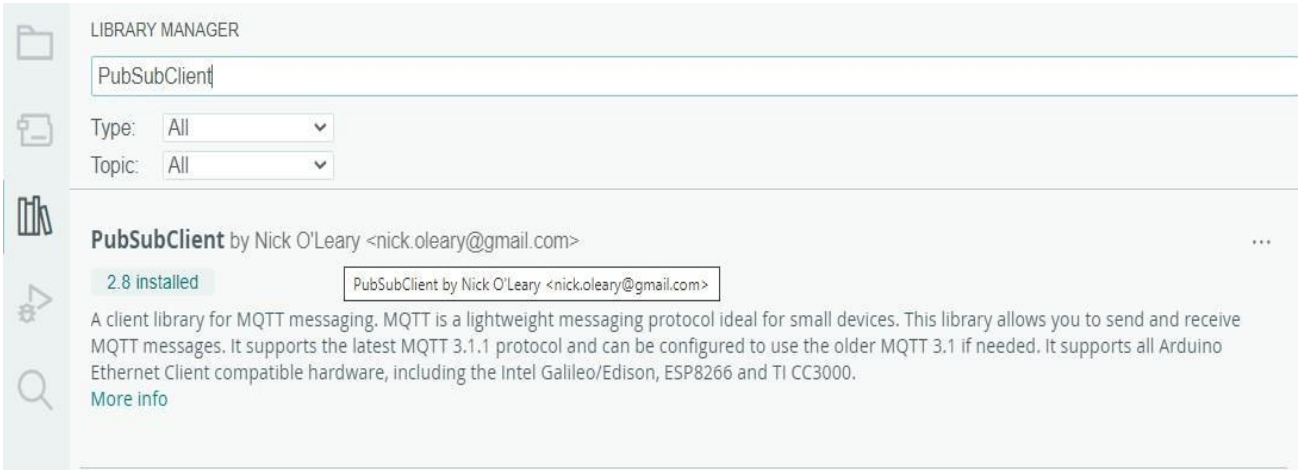
void sendATCommand(const char* cmd, unsigned long waitTime) {
  // Send the AT command to the module
  mySerial.println(cmd);
  delay(waitTime);

  // Read and print the response for debugging
  while (mySerial.available()) {
    Serial.write(mySerial.read());
  }
}
```

19.Write a program on Arduino/Raspberry Pi to publish temperature data to MQTT broker.

PIN CONFIGURATION	
	ESP 32
O/P(DHT 11)	GPIO 4
LED	GPIO 2

Install the PubSubClient library:



CODE

```
#include <Arduino.h>
#include <WiFi.h>
#include <PubSubClient.h>
#include "DHT.h"
WiFiClient wifiClient;
PubSubClient mqttClient(wifiClient);

#define DHTPIN 4  // Digital pin connected to the DHT sensor
#define DHTTYPE DHT11 // DHT 11
DHT dht(DHTPIN, DHTTYPE);
const char* ssid = "ssid";
```

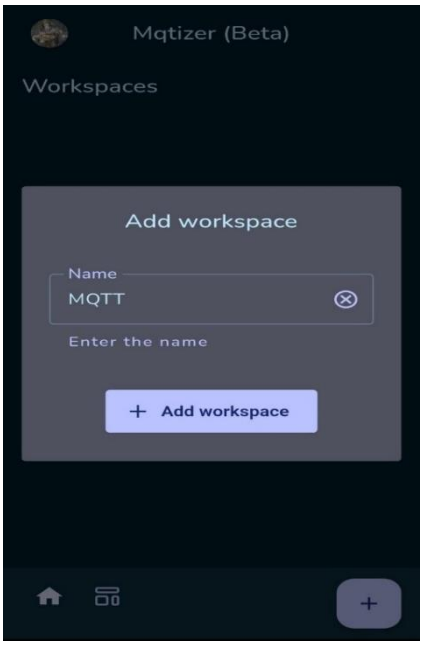
```
const char* password = "password";
//char *mqttServer = "broker.hivemq.com";
char *mqttServer = "test.mosquitto.org";
int mqttPort = 1883;
void setupMQTT() {
  mqttClient.setServer(mqttServer, mqttPort);
  mqttClient.setCallback(callback);
}
void reconnect() {
  Serial.println("Connecting to MQTT Broker...");
  while (!mqttClient.connected()) {
    Serial.println("Reconnecting to MQTT Broker..");
    String clientId = "ESP32Client-";
    clientId += String(random(0xffff), HEX);
    if (mqttClient.connect(clientId.c_str())) {
      Serial.println("Connected.");
    // subscribe to topic mqttClient.subscribe("esp32/message");
    }
  }
}
void setup() {
  Serial.begin(9600);
  Serial.println("DHT11 test!");
  dht.begin();
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.println("Connected to Wi-Fi");
  pinMode(2, OUTPUT);
  setupMQTT();
}
void loop() {
  if (!mqttClient.connected()) reconnect();
  mqttClient.loop();
  long now = millis();
  long previous_time = 0;
```

```
  if (now - previous_time > 1000) {
    previous_time = now;
    float h = dht.readHumidity();
    float t = dht.readTemperature();
    if (isnan(h) || isnan(t)) {
      Serial.println("Failed to read from DHT sensor!"); return;
    }
    char tempString[8]; dtostrf(t, 1, 2, tempString);
    Serial.print("Temperature: ");
    Serial.println(tempString);
    mqttClient.publish("esp32/temperature", tempString);
    char humString[8];
    dtostrf(h, 1, 2, humString);
    Serial.print("Humidity: ");
    Serial.println(humString);
    mqttClient.publish("esp32/humidity", humString);
    delay(2000);
  }
}

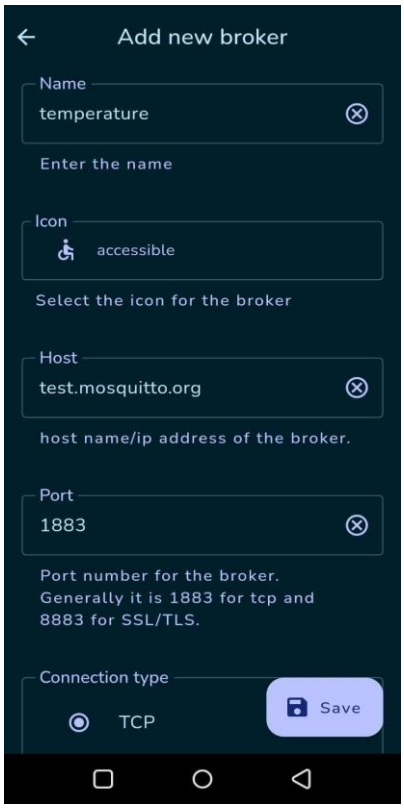
void callback(char* topic, byte* message, unsigned int length) {
  Serial.print("Callback - ");
  Serial.print("Message:"); String messageTemp;
  for (int i = 0; i < length; i++) {
    Serial.print((char)message[i]);
    messageTemp += (char)message[i];
  }
  Serial.println();
  if (String(topic) == "esp32/message") {
    Serial.print("Changing output to ");
    if(messageTemp == "on"){
      Serial.println("on");
      digitalWrite(2, HIGH);
    }
    else if(messageTemp == "off"){
      Serial.println("off");
      digitalWrite(2, LOW);
    }
  }
```

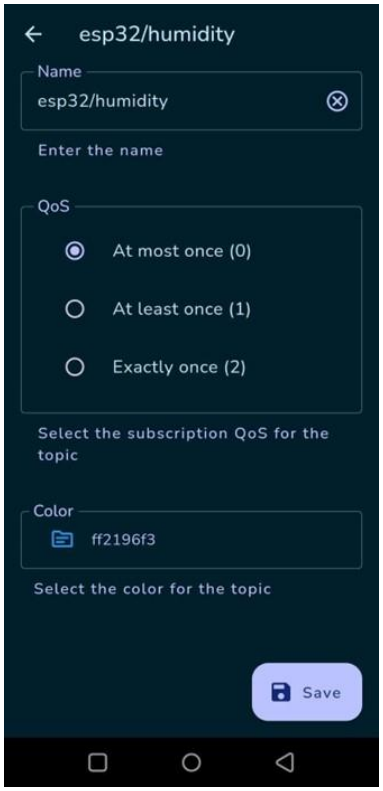
```
}
```

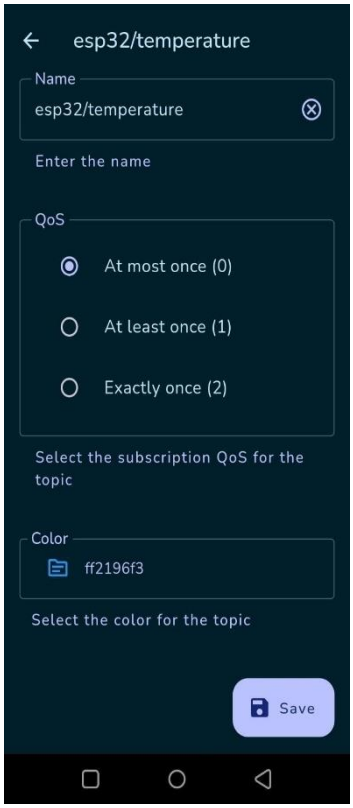
Step 1: Install the “Mqtizer” app on our mobiles.
Step 2: Create an account and add the Workspace.



Step 3: After creating Workspace add a new broker, by specifying the host and port and save it.

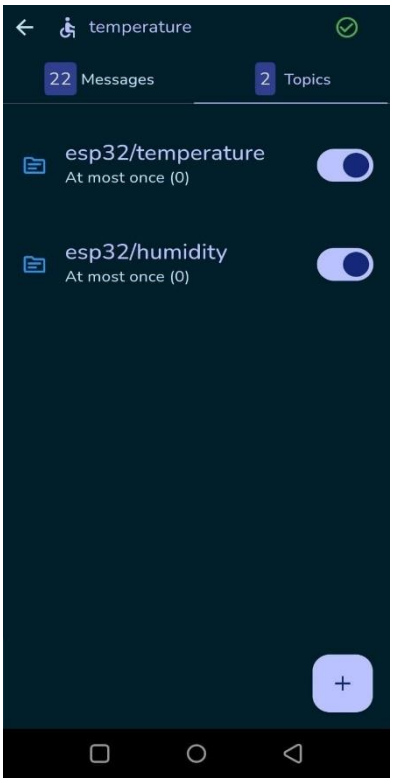






Step 4: Now we will be adding the topics that need to be subscribed.
Step 5: So, we use esp32/temperature and esp32/humidity as topics.

NOTE: Don’t give spacing after entering the topics.





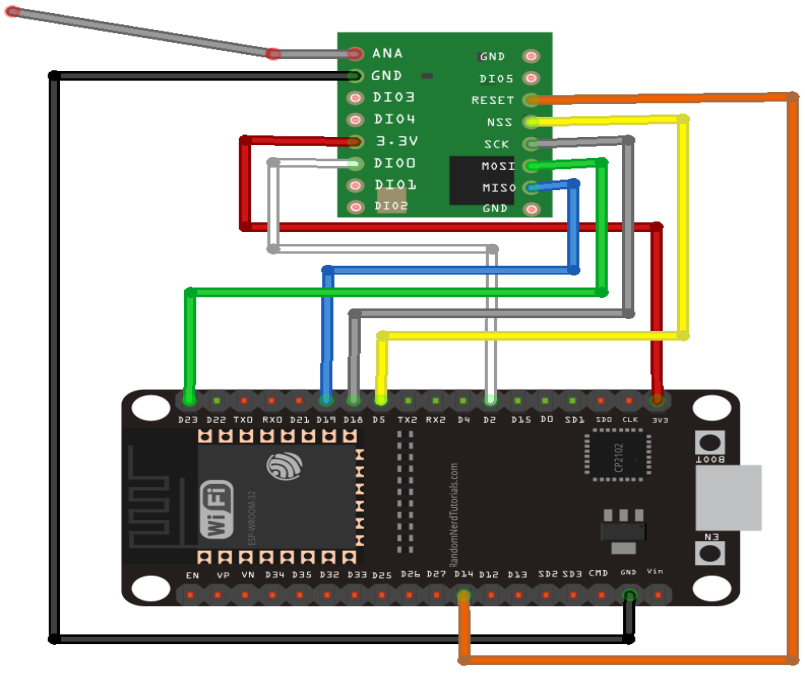
Step 6 : After subscribing the topics, now we are able to see the temperature and humidity data.

20.Interfacing LoRa module for Point-Point Communication using ESP32

PIN CONFIGURATIONS	
LoRa	**ESP32**
3.3V	3.3V
GND	GND
DIO 0	GPIO 2
SCK	GPIO 18
MOSI	GPIO 23
MISO	GPIO 19
RES	GPIO 14
NSS	GPIO 5

Example For Point-Point Communication:-

Pin configuration for both sender and receiver side as follows



Installing the LoRa Library

There are several libraries available to easily send and receive LoRa packets with the ESP32. In this example we'll be using the arduino-LoRa library by sandeep mistry.
Open your Arduino IDE, and go to **Sketch** > **Include Library** > **Manage Libraries** and search for "**LoRa**". Select the LoRa library highlighted in the figure below, and install it.

Sender Code:-

```
#include <SPI.h>
#include <LoRa.h>

//define the pins used by the transceiver module
#define nss 5
#define rst 14
#define dio0 2

void setup() {
  //initialize Serial Monitor
  Serial.begin(115200);
  while (!Serial);
  Serial.println("LoRa Sender");
  //setup LoRa transceiver module
  LoRa.setPins(nss, rst, dio0);
  //replace the LoRa.begin(---E-) argument with your location's frequency
  //433E6 for Asia
  //868E6 for Europe
  //915E6 for North America
  while (!LoRa.begin(433E6)) {
    Serial.println(".");
    delay(500);
  }
  // Change sync word (0xF3) to match the receiver
  // The sync word assures you don't get LoRa messages from other LoRa transceivers
  // ranges from 0-0xFF
  LoRa.setSyncWord(0xF3);
  Serial.println("LoRa Initializing OK!");
}
```

```
void loop() {
  Serial.print("Sending packet: ");
  //Send LoRa packet to receiver
  LoRa.beginPacket();
  LoRa.print("hello ");
  LoRa.endPacket();
  delay(10000);
}
```

Receiver Code:-

```
#include <SPI.h>
#include <LoRa.h>
//define the pins used by the transceiver module
#define ss 5
#define rst 14
#define dio0 2
void setup() {
  //initialize Serial Monitor
  Serial.begin(115200);
  while (!Serial);
  Serial.println("LoRa Receiver");
  //setup LoRa transceiver module
  LoRa.setPins(ss, rst, dio0);
  //replace the LoRa.begin(---E-) argument with your location's frequency
  //433E6 for Asia //868E6 for Europe  //915E6 for North America
  while (!LoRa.begin(433E6)) {
    Serial.println(".");
    delay(500);
  }
   // Change sync word (0xF3) to match the receiver
  // The sync word assures you don't get LoRa messages from other LoRa transceivers
  LoRa.setSyncWord(0xF3);   // ranges from 0-0xFF
  Serial.println("LoRa Initializing OK!");
```

```
}

void loop() {
  // try to parse packet
  int packetSize = LoRa.parsePacket();
  if (packetSize) {
    // received a packet
    Serial.print("Received packet : ");
    // read packet
    while (LoRa.available()) {
      String LoRaData = LoRa.readString();
      Serial.println(LoRaData);
    }
  }
}
```

21.Interfacing LoRaWAN with DHT 11 to send data to Gateway using ESP32

ESP32	LoRa	DHT 11	RGB
GPIO 15	NSS		
GPIO 12	MISO		
GPIO 14	SCK		
GPIO 13	MOSI		
GPIO 17	RESET		
GPIO 4	DIO 0		
GPIO 33	DIO 1		
GPIO 32	DIO 2		
GPIO 27		OUT	
GPIO 02			LED R

Setting Up the LoRa Environment

Library Installation

- Download LMIC Library from https://www.arduinolibraries.info/libraries/mcci-lo-ra-wan-lmic-library

Setting Up the End Nodes on the TTN Network

To transmit data to the TTN network, you need to first set up your gateway, and once that is doneyou need to set up your end nodes properly in order for the device to work properly. For that, you need to make a TTN account or log in to your existing TTN account. Once you have successfully logged in you need to go enter the TTN Console. If you have done everything correctly, you will be presented with the screen show

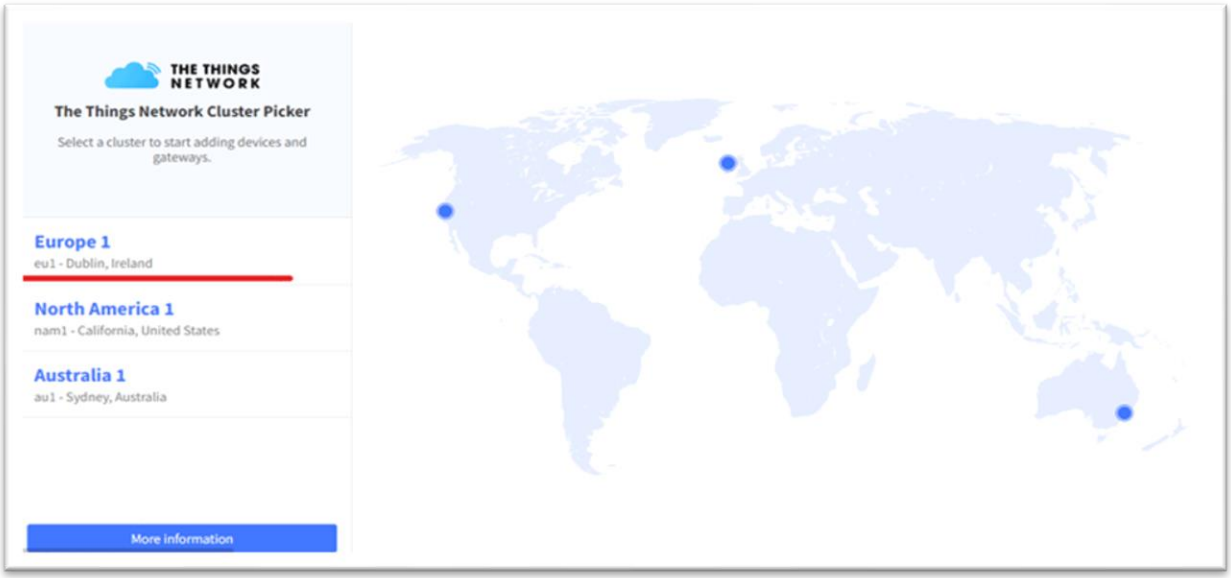
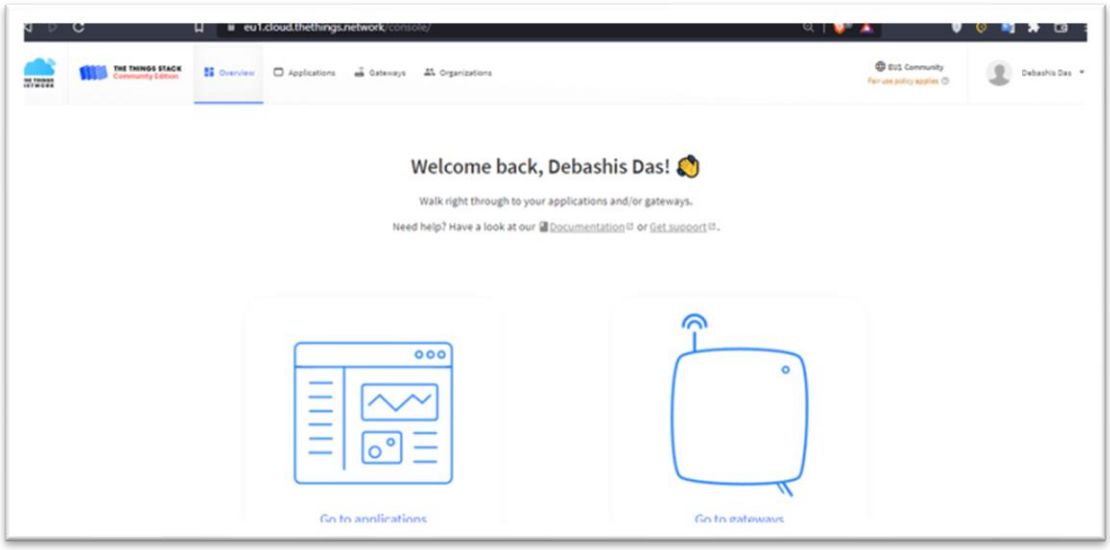
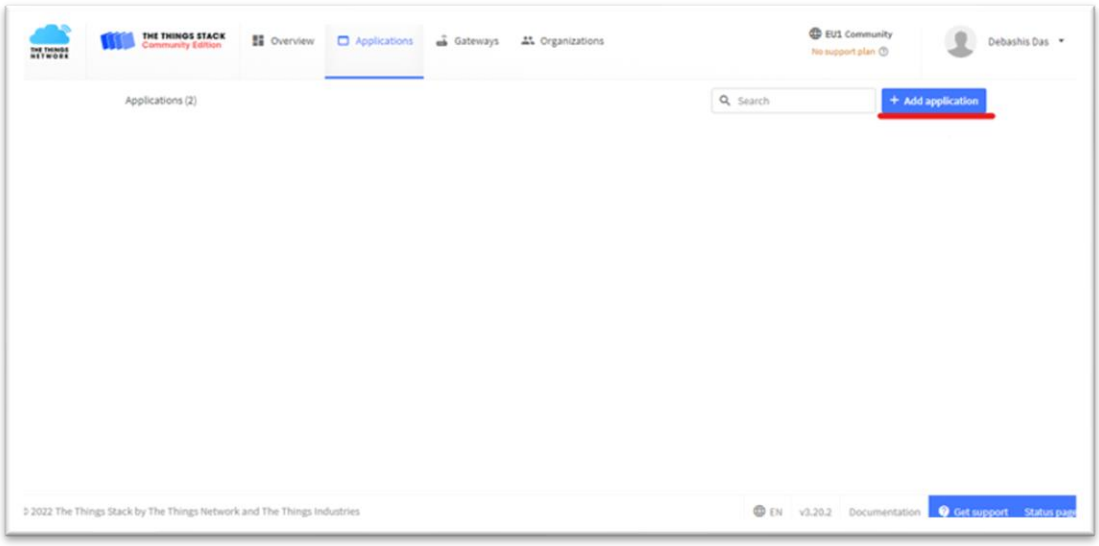


image below-

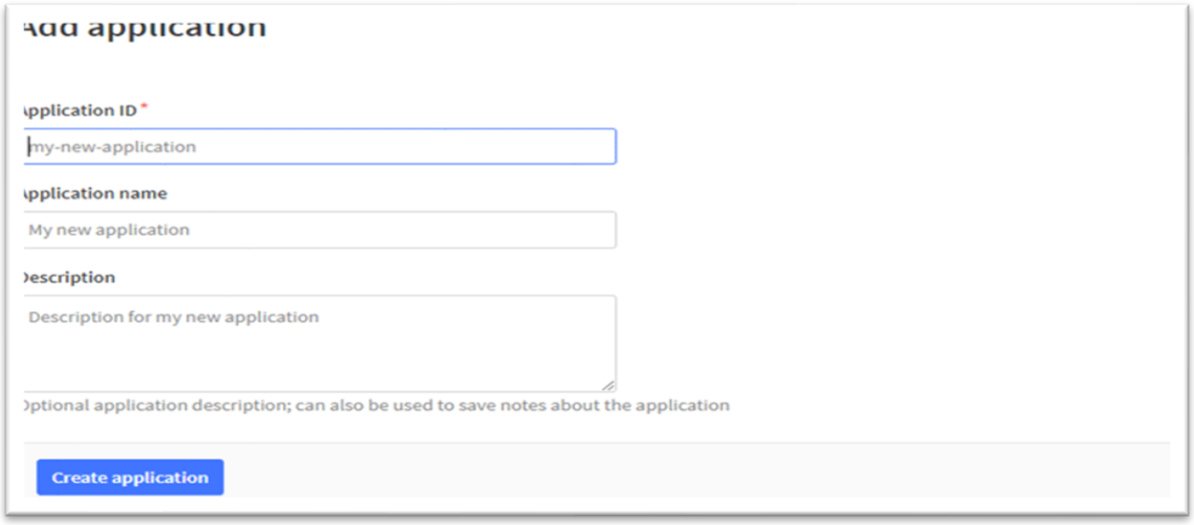
Once we are here we need to select a cluster, if you want to know more about the cluster you canclick on the more information button. For our case, we will choose europe1.



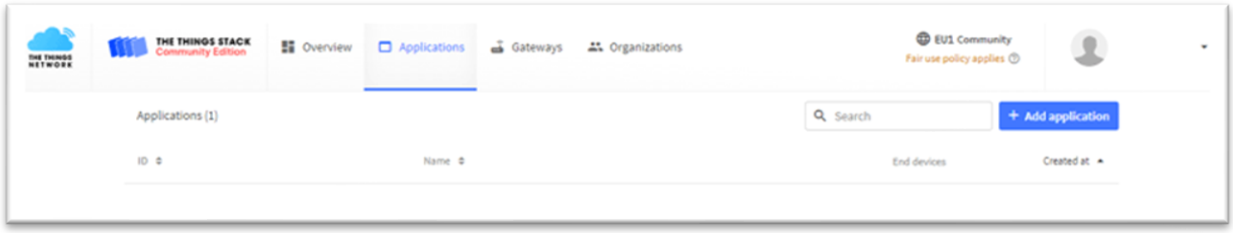
Once you do this you will be presented with the screen that is shown above. Now click on “Go to application”. You will be presented with the screen shown above, here you need to click on the Add applicationbutton to create



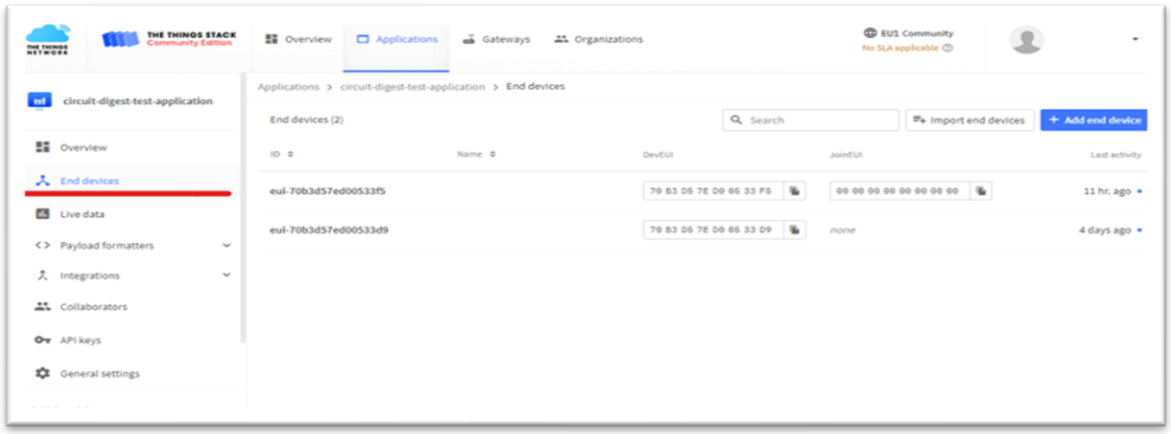
your first application.



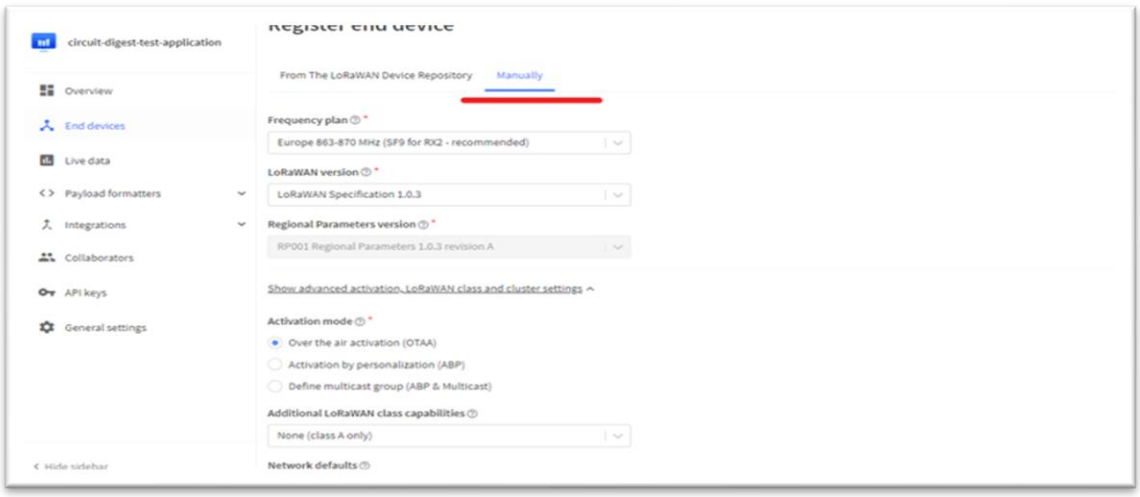
Now you need to create a new application by providing it with a unique Application ID and name. Once that is done the Description part is optional and you can click on the create application button.



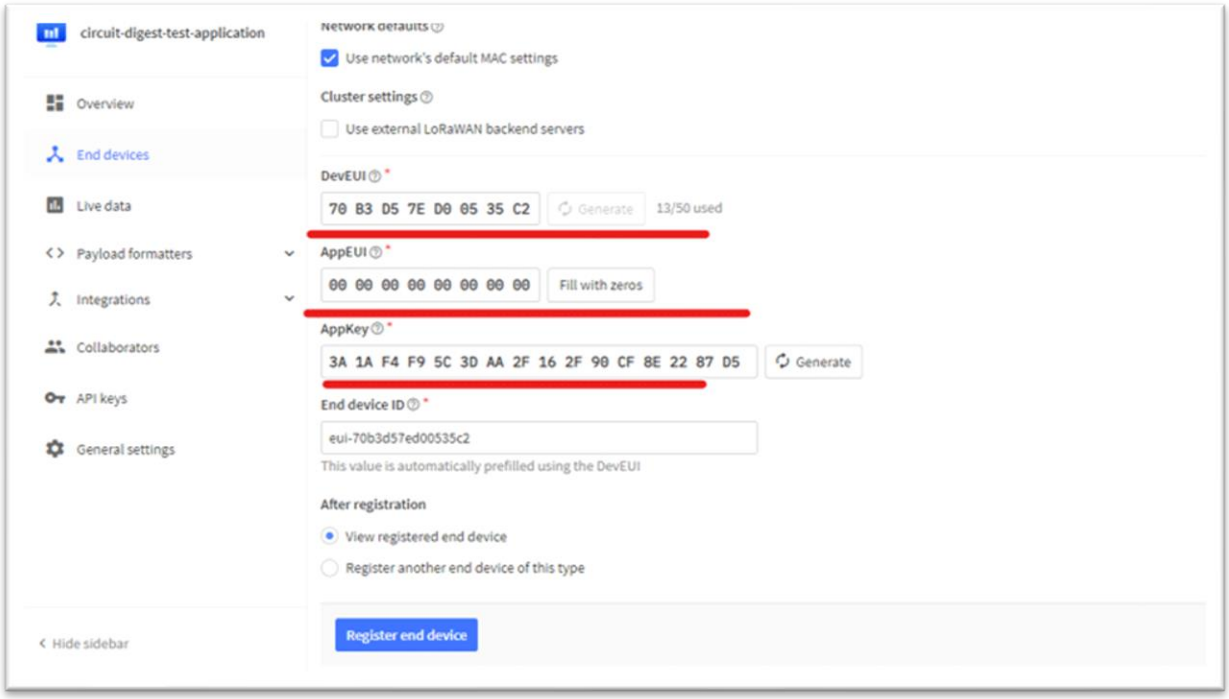
Once done, you will be presented with the screen that is shown above, now you need to create EndNodes for your application, as mentioned earlier, end nodes are the devices that send data to the application server.



Now click on the newly created Application and click on the end node button on the left and click on add application button on the right. As you can see from this demonstration we have already created two end nodes. One is to test the ABPmethod and the other oneis to testthe OTAA method.
Now once you click on the add end node button, you will be presented with the screen that is shownabove. Now click on it manually and follow along. First, we select the frequency plan, the LoRaWAN Specification, and the Activation Mode and for our first example, we are configuring it as OTAA.



Next, all you need to do is generate the keys for your end nodes and you are all done. Now click onthe Register end



device button to complete the process. Please keep in mind that your DeviceEUI,AppKey, and AppEUI are the three most important keys that you need to be careful with. Now all that is done, you can click on the Register end

device button to complete the process.

CODE

```
#include <lmic.h>
#include <hal/hal.h>
#include <SPI.h>

#include "DHT.h"

#define DHTPIN 27  // Digital pin connected to the DHT sensor
#define LED 2
#define DHTTYPE DHT11 // DHT 11
// Initialize DHT sensor.
DHT dht(DHTPIN, DHTTYPE);

// LoRaWAN NwkSKey, network session key
// This is the default Semtech key, which is used by the early prototype TTN
// network.
static const u1_t PROGMEM APPEUI[8]={ 0x63, 0x89, 0x25, 0x85, 0x69, 0x23, 0x55,
0x15 }; //LSB
void os_getArtEui (u1_t* buf) { memcpy_P(buf, APPEUI, 8);}
// LoRaWAN AppSKey, application session key
// This is the default Semtech key, which is used by the early prototype TTN
// network.
static const u1_t PROGMEM DEVEUI[8]= { 0xB2, 0x0A, 0x05, 0xD0, 0x7E, 0xD5,
0xB3, 0x70 }; //LSB
void os_getDevEui (u1_t* buf) { memcpy_P(buf, DEVEUI, 8);}
// LoRaWAN end-device address (DevAddr)
static const u1_t PROGMEM APPKEY[16] = { 0x7F, 0xF4, 0x4E, 0x49, 0x17, 0xFB,
0x85, 0x6F, 0x60, 0x61, 0x3A, 0x19, 0x98, 0x1F, 0x95, 0x03 }; //MSB
void os_getDevKey (u1_t* buf) { memcpy_P(buf, APPKEY, 16);}

uint8_t Data;
```

```
static uint8_t mydata[2] = {0x00,0x00}; static osjob_t sendjob;
// Schedule TX every this many seconds (might become longer due to duty
// cycle limitations).
const unsigned TX_INTERVAL = 30;

void dhttemp()
{
  delay(2000);
   float h = dht.readHumidity();
   float t = dht.readTemperature();
   if (isnan(h) || isnan(t)) {
    Serial.println(F("Failed to read from DHT sensor!"));
    return;
  }
  Serial.print(F("% Temperature: "));
  Serial.print(t);
  Serial.print(F("Humidity: "));
  Serial.print(h);
  Serial.println();

  mydata[0] = t;
  mydata[1] = h;

  delay(2000); //Delay of 1 second for ease of viewing
}

// Pin mapping
const lmic_pinmap lmic_pins = {
    .nss = 15,
    .rxtx = LMIC_UNUSED_PIN,
    .rst = 17,
    .dio = {4, 33, 32},
};
void onEvent (ev_t ev) {
  Serial.print(os_getTime());
  Serial.print(": ");
  switch(ev) {
    case EV_SCAN_TIMEOUT:
      Serial.println(F("EV_SCAN_TIMEOUT"));
```

```
    break;
  case EV_BEACON_FOUND:
    Serial.println(F("EV_BEACON_FOUND"));
    break;
  case EV_BEACON_MISSED:
    Serial.println(F("EV_BEACON_MISSED"));
    break;
  case EV_BEACON_TRACKED:
    Serial.println(F("EV_BEACON_TRACKED"));
    break;
  case EV_JOINING:
    Serial.println(F("EV_JOINING"));
    break;
  case EV_JOINED:
    Serial.println(F("EV_JOINED"));
    break;
  case EV_RFU1:
    Serial.println(F("EV_RFU1"));
    break;
  case EV_JOIN_FAILED:
    Serial.println(F("EV_JOIN_FAILED"));
    break;
  case EV_REJOIN_FAILED:
    Serial.println(F("EV_REJOIN_FAILED"));
    break;
  case EV_TXCOMPLETE:
    Serial.println(F("EV_TXCOMPLETE (includes waiting for RX windows)"));
    if (LMIC.txrxFlags & TXRX_ACK)
      Serial.println(F("Received ack"));
    if (LMIC.dataLen) {
      Serial.println(F("Received "));
      for (int i = 0; i < LMIC.dataLen; i++)
      {
        if (LMIC.frame[LMIC.dataBeg + i] < 0x10)
        {
          Data = (LMIC.frame[LMIC.dataBeg + i]);
        }
      }
      Serial.print("Downlink Data : ");
```

```
        Serial.println(Data);
        if (Data == 1)
        {
          digitalWrite(LED, HIGH);
          Serial.println("LED on");
        }
        else
        {
          digitalWrite(LED, LOW);
          Serial.println("LED off");
          }
      }
      os_setTimedCallback(&sendjob, os_getTime()+sec2osticks(TX_INTERVAL),
do_send);
      break;
    case EV_LOST_TSYNC:
      Serial.println(F("EV_LOST_TSYNC"));
      break;
    case EV_RESET:
      Serial.println(F("EV_RESET"));
      break;
    case EV_RXCOMPLETE:
        // data received in ping slot
      Serial.println(F("EV_RXCOMPLETE"));
      break;
    case EV_LINK_DEAD:
      Serial.println(F("EV_LINK_DEAD"));
      break;
    case EV_LINK_ALIVE:
      Serial.println(F("EV_LINK_ALIVE"));
      break;
    default:
      Serial.println(F("Unknown event"));
      break;
  }
}
void do_send(osjob_t* j){
  // Check if there is not a current TX/RX job running
  if (LMIC.opmode & OP_TXRXPEND)
```

```
  {
    Serial.println(F("OP_TXRXPEND, not sending"));
  }
  else
  {
    dhttemp();
    // Prepare upstream data transmission at the next possible time.
    LMIC_setTxData2(1, mydata, sizeof(mydata), 0);
    Serial.println(F("Packet queued"));
  }
  // Next TX is scheduled after TX_COMPLETE event.
}
void setup() {
  pinMode(LED, OUTPUT);
  Serial.begin(9600);
  dht.begin();
  Serial.println(F("Starting"));

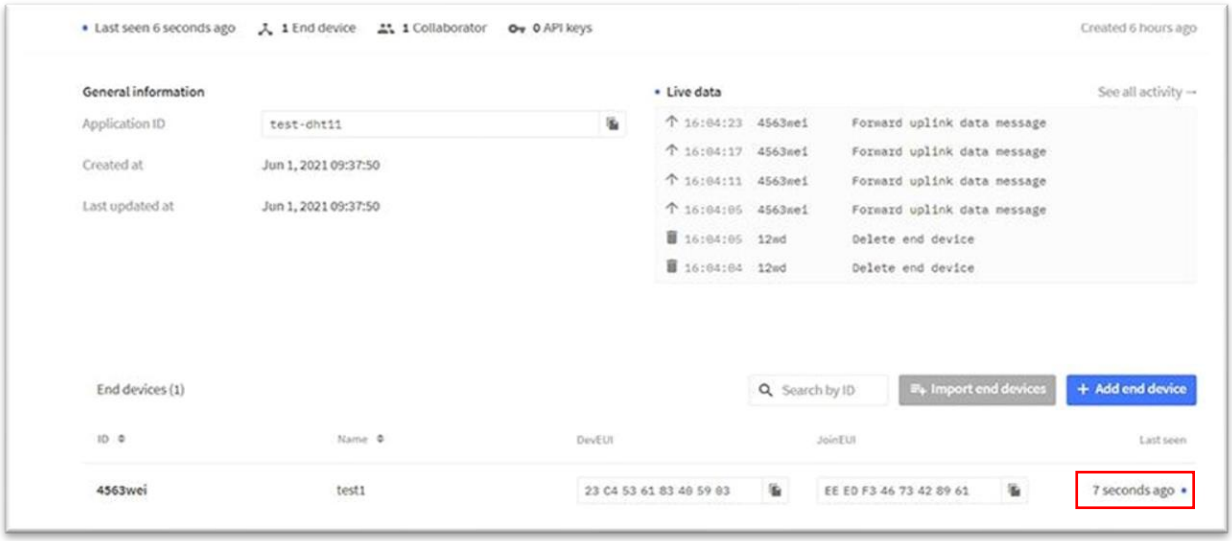
  #ifdef VCC_ENABLE
  // For Pinoccio Scout boards
  pinMode(VCC_ENABLE, OUTPUT);
  digitalWrite(VCC_ENABLE, HIGH);
  delay(1000);
  #endif
  // LMIC init
  os_init();
  // Reset the MAC state. Session and pending data transfers will be discarded.
  LMIC_reset();

     // Disable link check validation
  LMIC_setLinkCheckMode(0);
  LMIC.dn2Dr = DR_SF9;     // TTN uses SF9 for its RX2 window.
  // Set data rate and transmit power for uplink (note: txpow seems to be ignored by the
library)

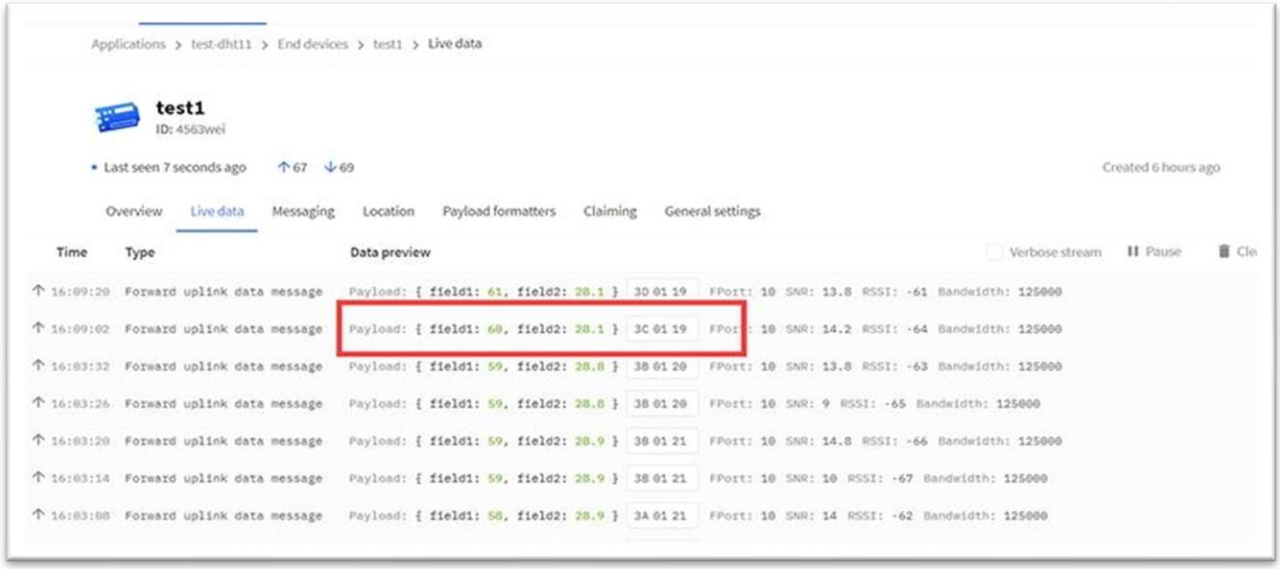
  LMIC_setDrTxpow(DR_SF7,14);
  do_send(&sendjob);
}
```

```
void loop() {
  os_runloop_once();
}
```

To Experience the OutcomeUplink



As the picture, the original payload is 3C0119, the date after payload formatter decoded are 60 and 28.1, it mean that the temperature is 28.1 °C and humidity is 60%.



To Decode this data write a Payload in Payload Formatter in End Device section

```
function Decoder(bytes,port){
  var temp=bytes[0];
  var hum=bytes[1];
  return{
    TEMPERATURE:temp,
```

- Go to payload formatter in the end device ,then go to uplink.
- Then select formatter type as “Custom javascript formatter”.
- Then paste the above code in the given box then save it .

Raspberry pi PICO W Codes:

1. **Program to Blink ONBOARD LED.**

PIN CONFIGURATION	
PICO W	**LED**
GP15	ANY LED PINS

CODE: BUILTIN LED

```
from machine import Pin
import utime as time

led = Pin('LED', Pin.OUT)
#led = Pin(15 Pin.OUT)

while True:
    led.value(True)
    time.sleep(1)
    led.value(False)
    time.sleep(1)
```

2. **Interfacing Push Button to control External LED.**

PIN CONFIGURATION	
PICO W	
GP2	LED
GP15	PUSH BUTTON

CODE: LED WITH PUSH BUTTON

```
from machine import Pin

led = Pin(2, Pin.OUT)  # LED connected to GPIO 2
button = Pin(15, Pin.IN, Pin.PULL_UP)  # Button connected to GPIO 15 with
pull-up resistor

while True:
    if not button.value():  # Button pressed (value is LOW)
```

```
    led.on()  # Turn LED on
  else:
    led.off()  # Turn LED off
```

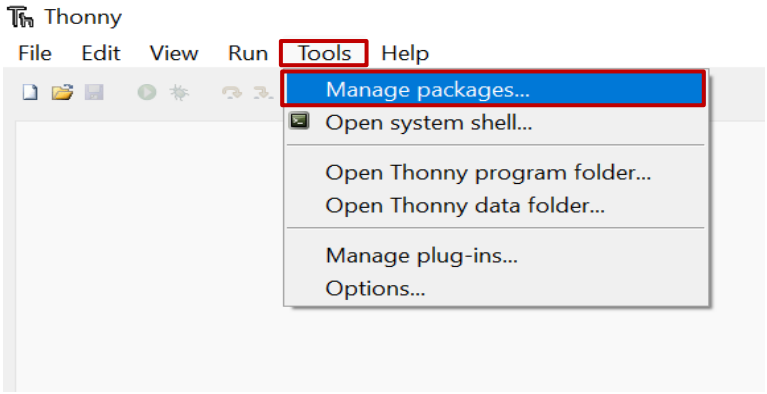
3. Program to print DHT11 data in shell using Pico W.

PIN CONFIGURATION	
PICO W	**DHT11**
GP16	DATA

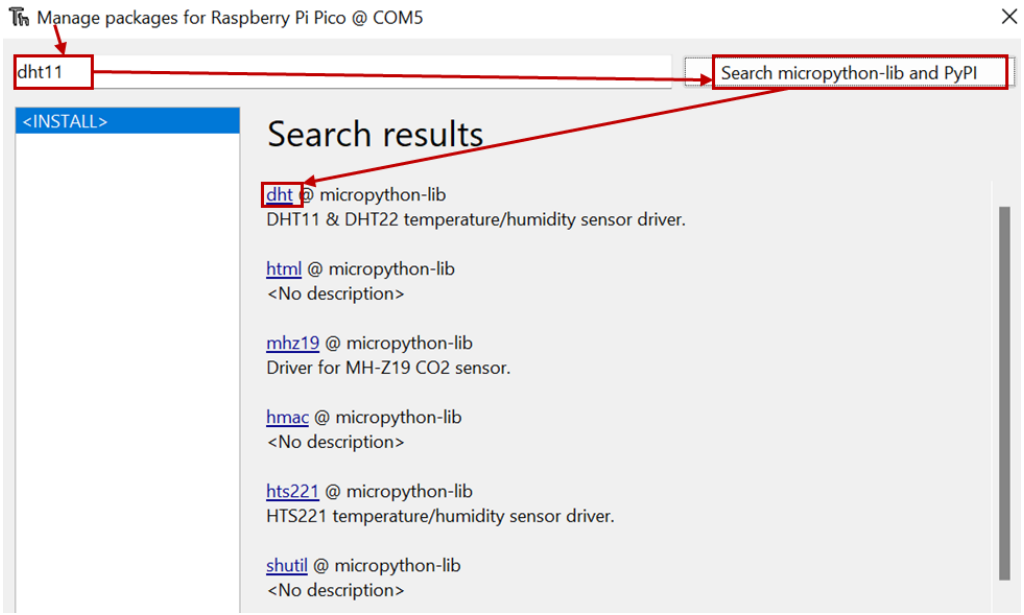
LIBRARY INATALLATION:

Step 1: Select board and port.

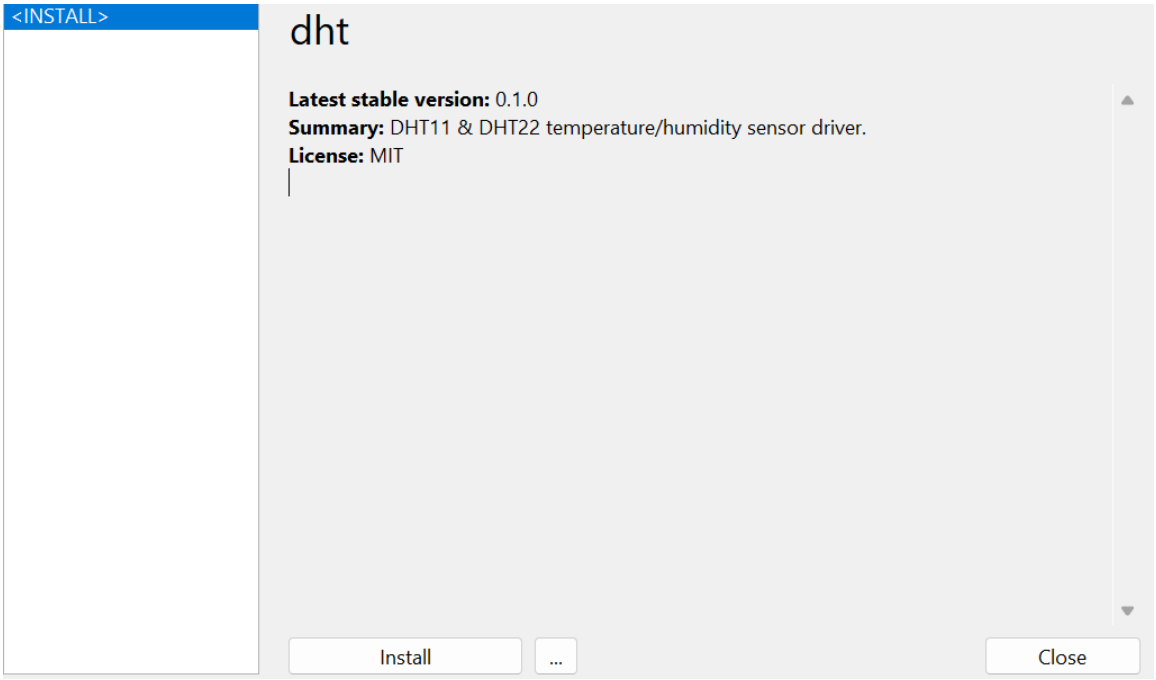
Step 2: Click on ‘Tools’ > ‘Manage Packages’.



Step 3: A Menu Bar will appear. In Search option, search as ‘dht11’ and click on ‘Search micropython-lib and PyPI’ and libraries will appear.



Step 4: Click on Install.



CODE: DHT11

```
from machine import Pin
from time import sleep
import dht
#sensor = dht.DHT22(Pin(16))
sensor = dht.DHT11(Pin(16))

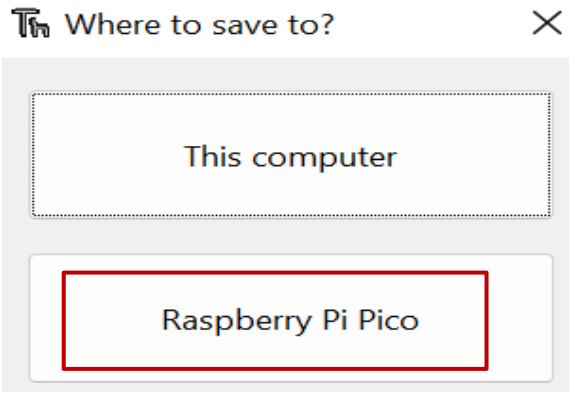
while True:
  try:
    sleep(2)
    sensor.measure()
    temp = sensor.temperature()
    hum = sensor.humidity()

    print('Temperature: %3.1f C' %temp)
    print('Humidity: %3.1f %%' %hum)
    print("")
  except OSError as e:
    print('Failed to read sensor.')
```

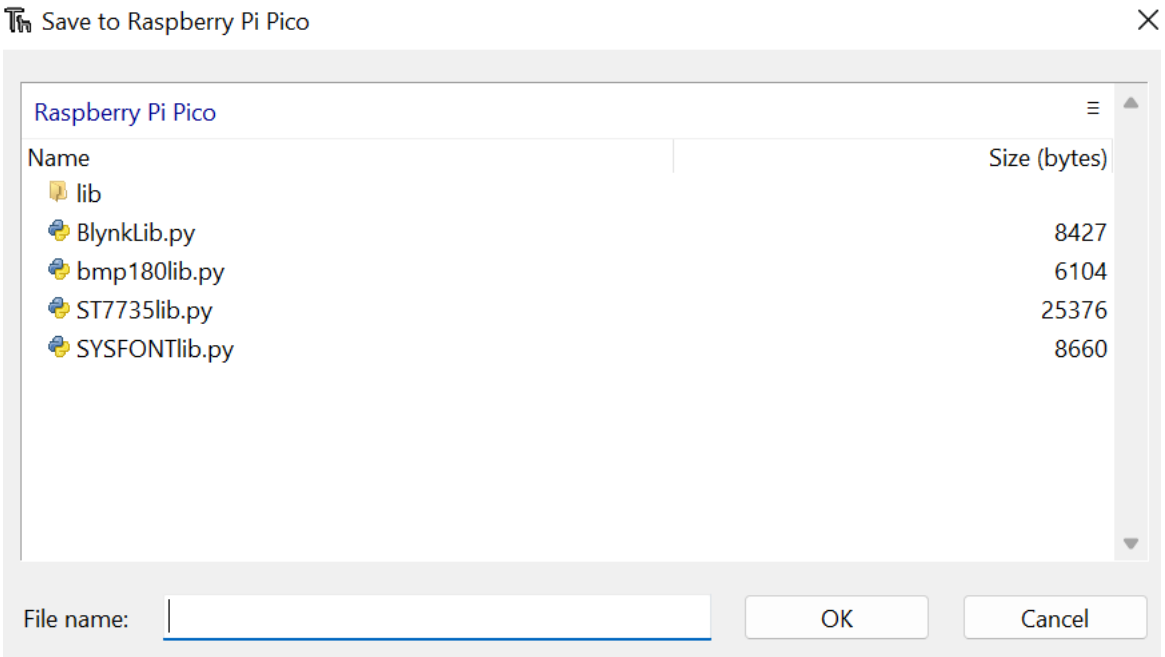
4. **Program print "Hello world" in TFT display using Pico W.**

PIN CONFIGURATION	
PICO W	**TFT**
3.3V	LED
GP2	SCK
GP3	SDA/MOSI
GP15	A0
GP4	RESET
GP5	CS

Step1: Copy these following libraries and save in Pico Storage.



Step2: Once code library code is saved, then copy the main code and run the program.



Libraries:

ST7735lib.py
Driver for Sainsmart 1.8" TFT display ST7735 import machine import time

```
from math import sqrt

# TFTRotations and TFTRGB are bits to set
# on MADCTL to control display rotation/color layout
TFTRotations = [0x00, 0x60, 0xC0, 0xA0]
TFTBGR = 0x08  # When set color is BGR else RGB.
TFTRGB = 0x00

# Helper functions
def clamp(aValue, aMin, aMax):
    return max(aMin, min(aMax, aValue))

def TFTColor(aR, aG, aB):
    '''Create a 16-bit RGB value from the given R, G, B from 0-255.'''
    return ((aR & 0xF8) << 8) | ((aG & 0xFC) << 3) | (aB >> 3)

ScreenSize = (128, 160)

class TFT(object):
    """Sainsmart TFT 7735 display driver."""

    # Command constants
    NOP = 0x0
    SWRESET = 0x01
    RDDID = 0x04
    RDDST = 0x09
    SLPIN = 0x10
    SLPOUT = 0x11
    PTLON = 0x12
    NORON = 0x13
    INVOFF = 0x20
    INVON = 0x21
    DISPOFF = 0x28
    DISPON = 0x29
    CASET = 0x2A
    RASET = 0x2B
    RAMWR = 0x2C
    RAMRD = 0x2E
    VSCRDEF = 0x33
```

```
    VSCSAD = 0x37
    COLMOD = 0x3A
    MADCTL = 0x36
    FRMCTR1 = 0xB1
    FRMCTR2 = 0xB2
    FRMCTR3 = 0xB3
    INVCTR = 0xB4
    DISSET5 = 0xB6
    PWCTR1 = 0xC0
    PWCTR2 = 0xC1
    PWCTR3 = 0xC2
    PWCTR4 = 0xC3
    PWCTR5 = 0xC4
    VMCTR1 = 0xC5
    RDID1 = 0xDA
    RDID2 = 0xDB
    RDID3 = 0xDC
    RDID4 = 0xDD
    PWCTR6 = 0xFC
    GMCTRP1 = 0xE0
    GMCTRN1 = 0xE1

    # Colors
    BLACK = 0
    RED = TFTColor(0xFF, 0x00, 0x00)
    GREEN = TFTColor(0x00, 0xFF, 0x00)
    BLUE = TFTColor(0x00, 0x00, 0xFF)
    CYAN = TFTColor(0x00, 0xFF, 0xFF)
    YELLOW = TFTColor(0xFF, 0xFF, 0x00)
    PURPLE = TFTColor(0xFF, 0x00, 0xFF)
    WHITE = TFTColor(0xFF, 0xFF, 0xFF)

    def __init__(self, spi, aDC, aReset, aCS):
        """Initialize the display with the given SPI and pins."""
        self._size = ScreenSize
        self._offset = bytearray([0, 0])
        self.rotate = 0  # Vertical with top toward pins.
        self._rgb = True  # Color order of RGB.
        self.tfa = 0  # Top fixed area
```

```
        self.bfa = 0  # Bottom fixed area
        self.dc = machine.Pin(aDC, machine.Pin.OUT)
        self.reset = machine.Pin(aReset, machine.Pin.OUT)
        self.cs = machine.Pin(aCS, machine.Pin.OUT)
        self.cs(1)
        self.spi = spi
        self.colorData = bytearray(2)
        self.windowLocData = bytearray(4)

    def on(self, aTF=True):
        """Turn display on or off."""
        self._writecommand(TFT.DISPON if aTF else TFT.DISPOFF)

    def invertcolor(self, aBool):
        """Invert the color data (Black = White)."""
        self._writecommand(TFT.INVON if aBool else TFT.INVOFF)

    def rgb(self, aTF=True):
        """Set RGB (True) or BGR (False)."""
        self._rgb = aTF
        self._setMADCTL()

    def rotation(self, aRot):
        """Set the display rotation (0-3)."""
        if 0 <= aRot < 4:
            rotchange = self.rotate ^ aRot
            self.rotate = aRot
            if rotchange & 1:
                self._size = (self._size[1], self._size[0])
            self._setMADCTL()

    def fill(self, aColor=BLACK):
        """Fill the screen with the given color."""
        self.fillrect((0, 0), self._size, aColor)

    def fillrect(self, aStart, aSize, aColor):
        """Draw a filled rectangle."""
        start = (clamp(aStart[0], 0, self._size[0]), clamp(aStart[1], 0, self._size[1]))
        end = (
```

```
            clamp(start[0] + aSize[0] - 1, 0, self._size[0]),
            clamp(start[1] + aSize[1] - 1, 0, self._size[1]),
        )
        self._setwindowloc(start, end)
        numPixels = (end[0] - start[0] + 1) * (end[1] - start[1] + 1)
        self._setColor(aColor)
        self._draw(numPixels)

    def text(self, aPos, aString, aColor, aFont, aSize=1, nowrap=False):
        """Draw text at the given position."""
        if aFont is None:
            return

        wh = (aSize, aSize) if isinstance(aSize, int) else aSize
        px, py = aPos
        width = wh[0] * aFont["Width"] + 1

        for c in aString:
            self.char((px, py), c, aColor, aFont, wh)
            px += width
            if px + width > self._size[0]:
                if nowrap:
                    break
                else:
                    py += aFont["Height"] * wh[1] + 1
                    px = aPos[0]

    def char(self, aPos, aChar, aColor, aFont, aSizes):
        """Draw a character."""
        if aFont is None:
            return

        startchar = aFont['Start']
        endchar = aFont['End']

        ci = ord(aChar)
        if startchar <= ci <= endchar:
            fontw = aFont['Width']
            fonth = aFont['Height']
```

```
            ci = (ci - startchar) * fontw

            charA = aFont['Data'][ci:ci + fontw]
            px = aPos[0]
            for c in charA:
                py = aPos[1]
                for r in range(fonth):
                    if c & 0x01:
                        self.fillrect((px, py), aSizes, aColor)
                    py += aSizes[1]
                    c >>= 1
                px += aSizes[0]

    def _writecommand(self, aCommand):
        """Write a command to the display."""
        self.dc(0)
        self.cs(0)
        self.spi.write(bytearray([aCommand]))
        self.cs(1)

    def _writedata(self, aData):
        """Write data to the display."""
        self.dc(1)
        self.cs(0)
        self.spi.write(aData)
        self.cs(1)

    def _setMADCTL(self):
        """Set the screen rotation and RGB/BGR format."""
        self._writecommand(TFT.MADCTL)
        rgb = TFTRGB if self._rgb else TFTBGR
        self._writedata(bytearray([TFTRotations[self.rotate] | rgb]))

    def _setwindowloc(self, aPos0, aPos1):
        """Set a rectangular area for drawing."""
        self._writecommand(TFT.CASET)  # Column address set
        self.windowLocData[0] = 0x00
        self.windowLocData[1] = int(aPos0[0])
        self.windowLocData[2] = 0x00
```

```
        self.windowLocData[3] = int(aPos1[0])
        self._writedata(self.windowLocData)

        self._writecommand(TFT.RASET)  # Row address set
        self.windowLocData[1] = int(aPos0[1])
        self.windowLocData[3] = int(aPos1[1])
        self._writedata(self.windowLocData)

        self._writecommand(TFT.RAMWR)  # Write to RAM

    def _setColor(self, aColor):
        self.colorData[0] = aColor >> 8
        self.colorData[1] = aColor

    def _draw(self, aPixels):
        self.dc(1)
        self.cs(0)
        for _ in range(aPixels):
            self.spi.write(self.colorData)
        self.cs(1)

    #   @micropython.native
    def _setwindowpoint( self, aPos ) :
        '"Set a single point for drawing a color to.'"
        x = self._offset[0] + int(aPos[0])
        y = self._offset[1] + int(aPos[1])
        self._writecommand(TFT.CASET)           #Column address set.
        self.windowLocData[0] = self._offset[0]
        self.windowLocData[1] = x
        self.windowLocData[2] = self._offset[0]
        self.windowLocData[3] = x
        self._writedata(self.windowLocData)

        self._writecommand(TFT.RASET)           #Row address set.
        self.windowLocData[0] = self._offset[1]
        self.windowLocData[1] = y
        self.windowLocData[2] = self._offset[1]
        self.windowLocData[3] = y
        self._writedata(self.windowLocData)
```

```
    self._writecommand(TFT.RAMWR)           #Write to RAM.

#   @micropython.native
def _setwindowloc( self, aPos0, aPos1 ) :
    '''Set a rectangular area for drawing a color to.'''
    self._writecommand(TFT.CASET)           #Column address set.
    self.windowLocData[0] = self._offset[0]
    self.windowLocData[1] = self._offset[0] + int(aPos0[0])
    self.windowLocData[2] = self._offset[0]
    self.windowLocData[3] = self._offset[0] + int(aPos1[0])
    self._writedata(self.windowLocData)

    self._writecommand(TFT.RASET)           #Row address set.
    self.windowLocData[0] = self._offset[1]
    self.windowLocData[1] = self._offset[1] + int(aPos0[1])
    self.windowLocData[2] = self._offset[1]
    self.windowLocData[3] = self._offset[1] + int(aPos1[1])
    self._writedata(self.windowLocData)

    self._writecommand(TFT.RAMWR)           #Write to RAM.

#@micropython.native
def _writecommand( self, aCommand ) :
    '''Write given command to the device.'''
    self.dc(0)
    self.cs(0)
    self.spi.write(bytearray([aCommand]))
    self.cs(1)

#@micropython.native
def _writedata( self, aData ) :
    '''Write given data to the device.  This may be
    either a single int or a bytearray of values.'''
    self.dc(1)
    self.cs(0)
    self.spi.write(aData)
    self.cs(1)

#@micropython.native
```

```
    def _pushcolor( self, aColor ) :
        '''Push given color to the device.'''
        self.colorData[0] = aColor >> 8
        self.colorData[1] = aColor
        self._writedata(self.colorData)

    #@micropython.native
    def _setMADCTL( self ) :
        '''Set screen rotation and RGB/BGR format.'''
        self._writecommand(TFT.MADCTL)
        rgb = TFTRGB if self._rgb else TFTBGR
        self._writedata(bytearray([TFTRotations[self.rotate] | rgb]))

    #@micropython.native
    def _reset( self ) :
        '''Reset the device.'''
        self.dc(0)
        self.reset(1)
        time.sleep_us(500)
        self.reset(0)
        time.sleep_us(500)
        self.reset(1)
        time.sleep_us(500)

    def initb( self ) :
        '''Initialize blue tab version.'''
        self._size = (ScreenSize[0] + 2, ScreenSize[1] + 1)
        self._reset()
        self._writecommand(TFT.SWRESET)             #Software reset.
        time.sleep_us(50)
        self._writecommand(TFT.SLPOUT)              #out of sleep mode.
        time.sleep_us(500)

        data1 = bytearray(1)
        self._writecommand(TFT.COLMOD)              #Set color mode.
        data1[0] = 0x05                         #16 bit color.
        self._writedata(data1)
        time.sleep_us(10)
```

```
    data3 = bytearray([0x00, 0x06, 0x03])       #fastest refresh, 6 lines front, 3 lines
back.
    self._writecommand(TFT.FRMCTR1)             #Frame rate control.
    self._writedata(data3)
    time.sleep_us(10)

    self._writecommand(TFT.MADCTL)
    data1[0] = 0x08                     #row address/col address, bottom to top refresh
    self._writedata(data1)

    data2 = bytearray(2)
    self._writecommand(TFT.DISSET5)             #Display settings
    data2[0] = 0x15                     #1 clock cycle nonoverlap, 2 cycle gate rise, 3
cycle oscil, equalize
    data2[1] = 0x02                     #fix on VTL
    self._writedata(data2)

    self._writecommand(TFT.INVCTR)              #Display inversion control
    data1[0] = 0x00                     #Line inversion.
    self._writedata(data1)

    self._writecommand(TFT.PWCTR1)              #Power control
    data2[0] = 0x02   #GVDD = 4.7V
    data2[1] = 0x70   #1.0uA
    self._writedata(data2)
    time.sleep_us(10)

    self._writecommand(TFT.PWCTR2)              #Power control
    data1[0] = 0x05                     #VGH = 14.7V, VGL = -7.35V
    self._writedata(data1)

    self._writecommand(TFT.PWCTR3)          #Power control
    data2[0] = 0x01   #Opamp current small
    data2[1] = 0x02   #Boost frequency
    self._writedata(data2)

    self._writecommand(TFT.VMCTR1)              #Power control
    data2[0] = 0x3C   #VCOMH = 4V
    data2[1] = 0x38   #VCOML = -1.1V
```

```
        self._writedata(data2)
        time.sleep_us(10)

        self._writecommand(TFT.PWCTR6)          #Power control
        data2[0] = 0x11
        data2[1] = 0x15
        self._writedata(data2)

        #These different values don't seem to make a difference.
        #   dataGMCTRP = bytearray([0x0f, 0x1a, 0x0f, 0x18, 0x2f, 0x28, 0x20, 0x22,
0x1f,
        #                           0x1b, 0x23, 0x37, 0x00, 0x07, 0x02, 0x10])
        dataGMCTRP = bytearray([0x02, 0x1c, 0x07, 0x12, 0x37, 0x32, 0x29, 0x2d, 0x29,
        0x25, 0x2b, 0x39, 0x00, 0x01, 0x03, 0x10])
        self._writecommand(TFT.GMCTRP1)
        self._writedata(dataGMCTRP)

        #   dataGMCTRN = bytearray([0x0f, 0x1b, 0x0f, 0x17, 0x33, 0x2c, 0x29, 0x2e,
0x30,
        #                           0x30, 0x39, 0x3f, 0x00, 0x07, 0x03, 0x10])
        dataGMCTRN = bytearray([0x03, 0x1d, 0x07, 0x06, 0x2e, 0x2c, 0x29, 0x2d, 0x2e,
        0x2e, 0x37, 0x3f, 0x00, 0x00, 0x02, 0x10])
        self._writecommand(TFT.GMCTRN1)
        self._writedata(dataGMCTRN)
        time.sleep_us(10)

        self._writecommand(TFT.CASET)           #Column address set.
        self.windowLocData[0] = 0x00
        self.windowLocData[1] = 2               #Start at column 2
        self.windowLocData[2] = 0x00
        self.windowLocData[3] = self._size[0] - 1
        self._writedata(self.windowLocData)

        self._writecommand(TFT.RASET)           #Row address set.
        self.windowLocData[1] = 1               #Start at row 2.
        self.windowLocData[3] = self._size[1] - 1
        self._writedata(self.windowLocData)

        self._writecommand(TFT.NORON)           #Normal display on.
```

```
        time.sleep_us(10)

        self._writecommand(TFT.RAMWR)
        time.sleep_us(500)

        self._writecommand(TFT.DISPON)
        self.cs(1)
        time.sleep_us(500)

    def initr( self ) :
        '''Initialize a red tab version.'''
        self._reset()

        self._writecommand(TFT.SWRESET)             #Software reset.
        time.sleep_us(150)
        self._writecommand(TFT.SLPOUT)              #out of sleep mode.
        time.sleep_us(500)

        data3 = bytearray([0x01, 0x2C, 0x2D])       #fastest refresh, 6 lines front, 3 lines
back.
        self._writecommand(TFT.FRMCTR1)             #Frame rate control.
        self._writedata(data3)

        self._writecommand(TFT.FRMCTR2)             #Frame rate control.
        self._writedata(data3)

        data6 = bytearray([0x01, 0x2c, 0x2d, 0x01, 0x2c, 0x2d])
        self._writecommand(TFT.FRMCTR3)             #Frame rate control.
        self._writedata(data6)
        time.sleep_us(10)

        data1 = bytearray(1)
        self._writecommand(TFT.INVCTR)              #Display inversion control
        data1[0] = 0x07                             #Line inversion.
        self._writedata(data1)

        self._writecommand(TFT.PWCTR1)              #Power control
        data3[0] = 0xA2
        data3[1] = 0x02
```

```
    data3[2] = 0x84
    self._writedata(data3)

    self._writecommand(TFT.PWCTR2)          #Power control
    data1[0] = 0xC5   #VGH = 14.7V, VGL = -7.35V
    self._writedata(data1)

    data2 = bytearray(2)
    self._writecommand(TFT.PWCTR3)          #Power control
    data2[0] = 0x0A   #Opamp current small
    data2[1] = 0x00   #Boost frequency
    self._writedata(data2)

    self._writecommand(TFT.PWCTR4)          #Power control
    data2[0] = 0x8A   #Opamp current small
    data2[1] = 0x2A   #Boost frequency
    self._writedata(data2)

    self._writecommand(TFT.PWCTR5)          #Power control
    data2[0] = 0x8A   #Opamp current small
    data2[1] = 0xEE   #Boost frequency
    self._writedata(data2)

    self._writecommand(TFT.VMCTR1)          #Power control
    data1[0] = 0x0E
    self._writedata(data1)

    self._writecommand(TFT.INVOFF)

    self._writecommand(TFT.MADCTL)          #Power control
    data1[0] = 0xC8
    self._writedata(data1)

    self._writecommand(TFT.COLMOD)
    data1[0] = 0x05
    self._writedata(data1)

    self._writecommand(TFT.CASET)           #Column address set.
    self.windowLocData[0] = 0x00
```

```
        self.windowLocData[1] = 0x00
        self.windowLocData[2] = 0x00
        self.windowLocData[3] = self._size[0] - 1
        self._writedata(self.windowLocData)

        self._writecommand(TFT.RASET)               #Row address set.
        self.windowLocData[3] = self._size[1] - 1
        self._writedata(self.windowLocData)

        dataGMCTRP = bytearray([0x0f, 0x1a, 0x0f, 0x18, 0x2f, 0x28, 0x20, 0x22, 0x1f,
        0x1b, 0x23, 0x37, 0x00, 0x07, 0x02, 0x10])
        self._writecommand(TFT.GMCTRP1)
        self._writedata(dataGMCTRP)

        dataGMCTRN = bytearray([0x0f, 0x1b, 0x0f, 0x17, 0x33, 0x2c, 0x29, 0x2e, 0x30,
        0x30, 0x39, 0x3f, 0x00, 0x07, 0x03, 0x10])
        self._writecommand(TFT.GMCTRN1)
        self._writedata(dataGMCTRN)
        time.sleep_us(10)

        self._writecommand(TFT.DISPON)
        time.sleep_us(100)

        self._writecommand(TFT.NORON)               #Normal display on.
        time.sleep_us(10)

        self.cs(1)

    def initb2( self ) :
        '''Initialize another blue tab version.'''
        self._size = (ScreenSize[0] + 2, ScreenSize[1] + 1)
        self._offset[0] = 2
        self._offset[1] = 1
        self._reset()
        self._writecommand(TFT.SWRESET)             #Software reset.
        time.sleep_us(50)
        self._writecommand(TFT.SLPOUT)              #out of sleep mode.
        time.sleep_us(500)
```

```
    data3 = bytearray([0x01, 0x2C, 0x2D])       #
    self._writecommand(TFT.FRMCTR1)             #Frame rate control.
    self._writedata(data3)
    time.sleep_us(10)

    self._writecommand(TFT.FRMCTR2)             #Frame rate control.
    self._writedata(data3)
    time.sleep_us(10)

    self._writecommand(TFT.FRMCTR3)             #Frame rate control.
    self._writedata(data3)
    time.sleep_us(10)

    self._writecommand(TFT.INVCTR)              #Display inversion control
    data1 = bytearray(1)                        #
    data1[0] = 0x07
    self._writedata(data1)

    self._writecommand(TFT.PWCTR1)              #Power control
    data3[0] = 0xA2   #
    data3[1] = 0x02   #
    data3[2] = 0x84   #
    self._writedata(data3)
    time.sleep_us(10)

    self._writecommand(TFT.PWCTR2)              #Power control
    data1[0] = 0xC5                             #
    self._writedata(data1)

    self._writecommand(TFT.PWCTR3)              #Power control
    data2 = bytearray(2)
    data2[0] = 0x0A   #
    data2[1] = 0x00   #
    self._writedata(data2)

    self._writecommand(TFT.PWCTR4)              #Power control
    data2[0] = 0x8A   #
    data2[1] = 0x2A   #
    self._writedata(data2)
```

```
        self._writecommand(TFT.PWCTR5)          #Power control
        data2[0] = 0x8A   #
        data2[1] = 0xEE   #
        self._writedata(data2)

        self._writecommand(TFT.VMCTR1)              #Power control
        data1[0] = 0x0E   #
        self._writedata(data1)
        time.sleep_us(10)

        self._writecommand(TFT.MADCTL)
        data1[0] = 0xC8                         #row address/col address, bottom to top refresh
        self._writedata(data1)

        #These different values don't seem to make a difference.
        #     dataGMCTRP = bytearray([0x0f, 0x1a, 0x0f, 0x18, 0x2f, 0x28, 0x20, 0x22,
0x1f,
        #                         0x1b, 0x23, 0x37, 0x00, 0x07, 0x02, 0x10])
        dataGMCTRP = bytearray([0x02, 0x1c, 0x07, 0x12, 0x37, 0x32, 0x29, 0x2d, 0x29,
        0x25, 0x2b, 0x39, 0x00, 0x01, 0x03, 0x10])
        self._writecommand(TFT.GMCTRP1)
        self._writedata(dataGMCTRP)

        #dataGMCTRN = bytearray([0x0f, 0x1b, 0x0f, 0x17, 0x33, 0x2c, 0x29, 0x2e,
0x30,
        #                         0x30, 0x39, 0x3f, 0x00, 0x07, 0x03, 0x10])
        dataGMCTRN = bytearray([0x03, 0x1d, 0x07, 0x06, 0x2e, 0x2c, 0x29, 0x2d, 0x2e,
        0x2e, 0x37, 0x3f, 0x00, 0x00, 0x02, 0x10])
        self._writecommand(TFT.GMCTRN1)
        self._writedata(dataGMCTRN)
        time.sleep_us(10)

        self._writecommand(TFT.CASET)               #Column address set.
        self.windowLocData[0] = 0x00
        self.windowLocData[1] = 0x02                #Start at column 2
        self.windowLocData[2] = 0x00
        self.windowLocData[3] = self._size[0] - 1
        self._writedata(self.windowLocData)
```

```
    self._writecommand(TFT.RASET)              #Row address set.
    self.windowLocData[1] = 0x01               #Start at row 2.
    self.windowLocData[3] = self._size[1] - 1
    self._writedata(self.windowLocData)

    data1 = bytearray(1)
    self._writecommand(TFT.COLMOD)             #Set color mode.
    data1[0] = 0x05                        #16 bit color.
    self._writedata(data1)
    time.sleep_us(10)

    self._writecommand(TFT.NORON)              #Normal display on.
    time.sleep_us(10)

    self._writecommand(TFT.RAMWR)
    time.sleep_us(500)

    self._writecommand(TFT.DISPON)
    self.cs(1)
    time.sleep_us(500)

#@micropython.native
def initg( self ) :
    '''Initialize a green tab version.'''
    self._reset()

    self._writecommand(TFT.SWRESET)            #Software reset.
    time.sleep_us(150)
    self._writecommand(TFT.SLPOUT)             #out of sleep mode.
    time.sleep_us(255)

    data3 = bytearray([0x01, 0x2C, 0x2D])   #fastest refresh, 6 lines front, 3 lines back.
    self._writecommand(TFT.FRMCTR1)            #Frame rate control.
    self._writedata(data3)

    self._writecommand(TFT.FRMCTR2)            #Frame rate control.
    self._writedata(data3)
```

```
data6 = bytearray([0x01, 0x2c, 0x2d, 0x01, 0x2c, 0x2d])
self._writecommand(TFT.FRMCTR3)           #Frame rate control.
self._writedata(data6)
time.sleep_us(10)

self._writecommand(TFT.INVCTR)           #Display inversion control
self._writedata(bytearray([0x07]))
self._writecommand(TFT.PWCTR1)           #Power control
data3[0] = 0xA2
data3[1] = 0x02
data3[2] = 0x84
self._writedata(data3)

self._writecommand(TFT.PWCTR2)           #Power control
self._writedata(bytearray([0xC5]))

data2 = bytearray(2)
self._writecommand(TFT.PWCTR3)           #Power control
data2[0] = 0x0A   #Opamp current small
data2[1] = 0x00   #Boost frequency
self._writedata(data2)

self._writecommand(TFT.PWCTR4)           #Power control
data2[0] = 0x8A   #Opamp current small
data2[1] = 0x2A   #Boost frequency
self._writedata(data2)

self._writecommand(TFT.PWCTR5)           #Power control
data2[0] = 0x8A   #Opamp current small
data2[1] = 0xEE   #Boost frequency
self._writedata(data2)

self._writecommand(TFT.VMCTR1)           #Power control
self._writedata(bytearray([0x0E]))

self._writecommand(TFT.INVOFF)

self._setMADCTL()
```

```
        self._writecommand(TFT.COLMOD)
        self._writedata(bytearray([0x05]))

        self._writecommand(TFT.CASET)           #Column address set.
        self.windowLocData[0] = 0x00
        self.windowLocData[1] = 0x01            #Start at row/column 1.
        self.windowLocData[2] = 0x00
        self.windowLocData[3] = self._size[0] - 1
        self._writedata(self.windowLocData)

        self._writecommand(TFT.RASET)           #Row address set.
        self.windowLocData[3] = self._size[1] - 1
        self._writedata(self.windowLocData)

        dataGMCTRP = bytearray([0x02, 0x1c, 0x07, 0x12, 0x37, 0x32, 0x29, 0x2d, 0x29,
        0x25, 0x2b, 0x39, 0x00, 0x01, 0x03, 0x10])
        self._writecommand(TFT.GMCTRP1)
        self._writedata(dataGMCTRP)

        dataGMCTRN = bytearray([0x03, 0x1d, 0x07, 0x06, 0x2e, 0x2c, 0x29, 0x2d, 0x2e,
        0x2e, 0x37, 0x3f, 0x00, 0x00, 0x02, 0x10])
        self._writecommand(TFT.GMCTRN1)
        self._writedata(dataGMCTRN)

        self._writecommand(TFT.NORON)           #Normal display on.
        time.sleep_us(10)

        self._writecommand(TFT.DISPON)
        time.sleep_us(100)

        self.cs(1)

    def maker(  ) :
        t = TFT(1, "X1", "X2")
        print("Initializing")
        t.initr()
        t.fill(0)
        return t
```

```
def makeb(  ) :
    t = TFT(1, "X1", "X2")
    print("Initializing")
    t.initb()
    t.fill(0)
    return t

def makeg(  ) :
    t = TFT(1, "X1", "X2")
    print("Initializing")
    t.initg()
    t.fill(0)
    return t
```

SYSFONTlib.py

```
#Font used for ST7735 display.

#Each character uses 5 bytes.
#index using ASCII value * 5.
#Each byte contains a column of pixels.
#The character may be 8 pixels high and 5 wide.

sysfont = {"Width": 5, "Height": 8, "Start": 0, "End": 254, "Data": bytearray([
  0x00, 0x00, 0x00, 0x00, 0x00,
  0x3E, 0x5B, 0x4F, 0x5B, 0x3E,
  0x3E, 0x6B, 0x4F, 0x6B, 0x3E,
  0x1C, 0x3E, 0x7C, 0x3E, 0x1C,
  0x18, 0x3C, 0x7E, 0x3C, 0x18,
  0x1C, 0x57, 0x7D, 0x57, 0x1C,
  0x1C, 0x5E, 0x7F, 0x5E, 0x1C,
  0x00, 0x18, 0x3C, 0x18, 0x00,
  0xFF, 0xE7, 0xC3, 0xE7, 0xFF,
  0x00, 0x18, 0x24, 0x18, 0x00,
  0xFF, 0xE7, 0xDB, 0xE7, 0xFF,
  0x30, 0x48, 0x3A, 0x06, 0x0E,
  0x26, 0x29, 0x79, 0x29, 0x26,
  0x40, 0x7F, 0x05, 0x05, 0x07,
```

```
0x40, 0x7F, 0x05, 0x25, 0x3F,
0x5A, 0x3C, 0xE7, 0x3C, 0x5A,
0x7F, 0x3E, 0x1C, 0x1C, 0x08,
0x08, 0x1C, 0x1C, 0x3E, 0x7F,
0x14, 0x22, 0x7F, 0x22, 0x14,
0x5F, 0x5F, 0x00, 0x5F, 0x5F,
0x06, 0x09, 0x7F, 0x01, 0x7F,
0x00, 0x66, 0x89, 0x95, 0x6A,
0x60, 0x60, 0x60, 0x60, 0x60,
0x94, 0xA2, 0xFF, 0xA2, 0x94,
0x08, 0x04, 0x7E, 0x04, 0x08,
0x10, 0x20, 0x7E, 0x20, 0x10,
0x08, 0x08, 0x2A, 0x1C, 0x08,
0x08, 0x1C, 0x2A, 0x08, 0x08,
0x1E, 0x10, 0x10, 0x10, 0x10,
0x0C, 0x1E, 0x0C, 0x1E, 0x0C,
0x30, 0x38, 0x3E, 0x38, 0x30,
0x06, 0x0E, 0x3E, 0x0E, 0x06,
0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x5F, 0x00, 0x00,
0x00, 0x07, 0x00, 0x07, 0x00,
0x14, 0x7F, 0x14, 0x7F, 0x14,
0x24, 0x2A, 0x7F, 0x2A, 0x12,
0x23, 0x13, 0x08, 0x64, 0x62,
0x36, 0x49, 0x56, 0x20, 0x50,
0x00, 0x08, 0x07, 0x03, 0x00,
0x00, 0x1C, 0x22, 0x41, 0x00,
0x00, 0x41, 0x22, 0x1C, 0x00,
0x2A, 0x1C, 0x7F, 0x1C, 0x2A,
0x08, 0x08, 0x3E, 0x08, 0x08,
0x00, 0x80, 0x70, 0x30, 0x00,
0x08, 0x08, 0x08, 0x08, 0x08,
0x00, 0x00, 0x60, 0x60, 0x00,
0x20, 0x10, 0x08, 0x04, 0x02,
0x3E, 0x51, 0x49, 0x45, 0x3E,
0x00, 0x42, 0x7F, 0x40, 0x00,
0x72, 0x49, 0x49, 0x49, 0x46,
0x21, 0x41, 0x49, 0x4D, 0x33,
0x18, 0x14, 0x12, 0x7F, 0x10,
```

```
0x27, 0x45, 0x45, 0x45, 0x39,
0x3C, 0x4A, 0x49, 0x49, 0x31,
0x41, 0x21, 0x11, 0x09, 0x07,
0x36, 0x49, 0x49, 0x49, 0x36,
0x46, 0x49, 0x49, 0x29, 0x1E,
0x00, 0x00, 0x14, 0x00, 0x00,
0x00, 0x40, 0x34, 0x00, 0x00,
0x00, 0x08, 0x14, 0x22, 0x41,
0x14, 0x14, 0x14, 0x14, 0x14,
0x00, 0x41, 0x22, 0x14, 0x08,
0x02, 0x01, 0x59, 0x09, 0x06,
0x3E, 0x41, 0x5D, 0x59, 0x4E,
0x7C, 0x12, 0x11, 0x12, 0x7C,
0x7F, 0x49, 0x49, 0x49, 0x36,
0x3E, 0x41, 0x41, 0x41, 0x22,
0x7F, 0x41, 0x41, 0x41, 0x3E,
0x7F, 0x49, 0x49, 0x49, 0x41,
0x7F, 0x09, 0x09, 0x09, 0x01,
0x3E, 0x41, 0x41, 0x51, 0x73,
0x7F, 0x08, 0x08, 0x08, 0x7F,
0x00, 0x41, 0x7F, 0x41, 0x00,
0x20, 0x40, 0x41, 0x3F, 0x01,
0x7F, 0x08, 0x14, 0x22, 0x41,
0x7F, 0x40, 0x40, 0x40, 0x40,
0x7F, 0x02, 0x1C, 0x02, 0x7F,
0x7F, 0x04, 0x08, 0x10, 0x7F,
0x3E, 0x41, 0x41, 0x41, 0x3E,
0x7F, 0x09, 0x09, 0x09, 0x06,
0x3E, 0x41, 0x51, 0x21, 0x5E,
0x7F, 0x09, 0x19, 0x29, 0x46,
0x26, 0x49, 0x49, 0x49, 0x32,
0x03, 0x01, 0x7F, 0x01, 0x03,
0x3F, 0x40, 0x40, 0x40, 0x3F,
0x1F, 0x20, 0x40, 0x20, 0x1F,
0x3F, 0x40, 0x38, 0x40, 0x3F,
0x63, 0x14, 0x08, 0x14, 0x63,
0x03, 0x04, 0x78, 0x04, 0x03,
0x61, 0x59, 0x49, 0x4D, 0x43,
0x00, 0x7F, 0x41, 0x41, 0x41,
```

```
0x02, 0x04, 0x08, 0x10, 0x20,
0x00, 0x41, 0x41, 0x41, 0x7F,
0x04, 0x02, 0x01, 0x02, 0x04,
0x40, 0x40, 0x40, 0x40, 0x40,
0x00, 0x03, 0x07, 0x08, 0x00,
0x20, 0x54, 0x54, 0x78, 0x40,
0x7F, 0x28, 0x44, 0x44, 0x38,
0x38, 0x44, 0x44, 0x44, 0x28,
0x38, 0x44, 0x44, 0x28, 0x7F,
0x38, 0x54, 0x54, 0x54, 0x18,
0x00, 0x08, 0x7E, 0x09, 0x02,
0x18, 0xA4, 0xA4, 0x9C, 0x78,
0x7F, 0x08, 0x04, 0x04, 0x78,
0x00, 0x44, 0x7D, 0x40, 0x00,
0x20, 0x40, 0x40, 0x3D, 0x00,
0x7F, 0x10, 0x28, 0x44, 0x00,
0x00, 0x41, 0x7F, 0x40, 0x00,
0x7C, 0x04, 0x78, 0x04, 0x78,
0x7C, 0x08, 0x04, 0x04, 0x78,
0x38, 0x44, 0x44, 0x44, 0x38,
0xFC, 0x18, 0x24, 0x24, 0x18,
0x18, 0x24, 0x24, 0x18, 0xFC,
0x7C, 0x08, 0x04, 0x04, 0x08,
0x48, 0x54, 0x54, 0x54, 0x24,
0x04, 0x04, 0x3F, 0x44, 0x24,
0x3C, 0x40, 0x40, 0x20, 0x7C,
0x1C, 0x20, 0x40, 0x20, 0x1C,
0x3C, 0x40, 0x30, 0x40, 0x3C,
0x44, 0x28, 0x10, 0x28, 0x44,
0x4C, 0x90, 0x90, 0x90, 0x7C,
0x44, 0x64, 0x54, 0x4C, 0x44,
0x00, 0x08, 0x36, 0x41, 0x00,
0x00, 0x00, 0x77, 0x00, 0x00,
0x00, 0x41, 0x36, 0x08, 0x00,
0x02, 0x01, 0x02, 0x04, 0x02,
0x3C, 0x26, 0x23, 0x26, 0x3C,
0x1E, 0xA1, 0xA1, 0x61, 0x12,
0x3A, 0x40, 0x40, 0x20, 0x7A,
0x38, 0x54, 0x54, 0x55, 0x59,
```

```
0x21, 0x55, 0x55, 0x79, 0x41,
0x21, 0x54, 0x54, 0x78, 0x41,
0x21, 0x55, 0x54, 0x78, 0x40,
0x20, 0x54, 0x55, 0x79, 0x40,
0x0C, 0x1E, 0x52, 0x72, 0x12,
0x39, 0x55, 0x55, 0x55, 0x59,
0x39, 0x54, 0x54, 0x54, 0x59,
0x39, 0x55, 0x54, 0x54, 0x58,
0x00, 0x00, 0x45, 0x7C, 0x41,
0x00, 0x02, 0x45, 0x7D, 0x42,
0x00, 0x01, 0x45, 0x7C, 0x40,
0xF0, 0x29, 0x24, 0x29, 0xF0,
0xF0, 0x28, 0x25, 0x28, 0xF0,
0x7C, 0x54, 0x55, 0x45, 0x00,
0x20, 0x54, 0x54, 0x7C, 0x54,
0x7C, 0x0A, 0x09, 0x7F, 0x49,
0x32, 0x49, 0x49, 0x49, 0x32,
0x32, 0x48, 0x48, 0x48, 0x32,
0x32, 0x4A, 0x48, 0x48, 0x30,
0x3A, 0x41, 0x41, 0x21, 0x7A,
0x3A, 0x42, 0x40, 0x20, 0x78,
0x00, 0x9D, 0xA0, 0xA0, 0x7D,
0x39, 0x44, 0x44, 0x44, 0x39,
0x3D, 0x40, 0x40, 0x40, 0x3D,
0x3C, 0x24, 0xFF, 0x24, 0x24,
0x48, 0x7E, 0x49, 0x43, 0x66,
0x2B, 0x2F, 0xFC, 0x2F, 0x2B,
0xFF, 0x09, 0x29, 0xF6, 0x20,
0xC0, 0x88, 0x7E, 0x09, 0x03,
0x20, 0x54, 0x54, 0x79, 0x41,
0x00, 0x00, 0x44, 0x7D, 0x41,
0x30, 0x48, 0x48, 0x4A, 0x32,
0x38, 0x40, 0x40, 0x22, 0x7A,
0x00, 0x7A, 0x0A, 0x0A, 0x72,
0x7D, 0x0D, 0x19, 0x31, 0x7D,
0x26, 0x29, 0x29, 0x2F, 0x28,
0x26, 0x29, 0x29, 0x29, 0x26,
0x30, 0x48, 0x4D, 0x40, 0x20,
0x38, 0x08, 0x08, 0x08, 0x08,
```

```
0x08, 0x08, 0x08, 0x08, 0x38,
0x2F, 0x10, 0xC8, 0xAC, 0xBA,
0x2F, 0x10, 0x28, 0x34, 0xFA,
0x00, 0x00, 0x7B, 0x00, 0x00,
0x08, 0x14, 0x2A, 0x14, 0x22,
0x22, 0x14, 0x2A, 0x14, 0x08,
0xAA, 0x00, 0x55, 0x00, 0xAA,
0xAA, 0x55, 0xAA, 0x55, 0xAA,
0x00, 0x00, 0x00, 0xFF, 0x00,
0x10, 0x10, 0x10, 0xFF, 0x00,
0x14, 0x14, 0x14, 0xFF, 0x00,
0x10, 0x10, 0xFF, 0x00, 0xFF,
0x10, 0x10, 0xF0, 0x10, 0xF0,
0x14, 0x14, 0x14, 0xFC, 0x00,
0x14, 0x14, 0xF7, 0x00, 0xFF,
0x00, 0x00, 0xFF, 0x00, 0xFF,
0x14, 0x14, 0xF4, 0x04, 0xFC,
0x14, 0x14, 0x17, 0x10, 0x1F,
0x10, 0x10, 0x1F, 0x10, 0x1F,
0x14, 0x14, 0x14, 0x1F, 0x00,
0x10, 0x10, 0x10, 0xF0, 0x00,
0x00, 0x00, 0x00, 0x1F, 0x10,
0x10, 0x10, 0x10, 0x1F, 0x10,
0x10, 0x10, 0x10, 0xF0, 0x10,
0x00, 0x00, 0x00, 0xFF, 0x10,
0x10, 0x10, 0x10, 0x10, 0x10,
0x10, 0x10, 0x10, 0xFF, 0x10,
0x00, 0x00, 0x00, 0xFF, 0x14,
0x00, 0x00, 0xFF, 0x00, 0xFF,
0x00, 0x00, 0x1F, 0x10, 0x17,
0x00, 0x00, 0xFC, 0x04, 0xF4,
0x14, 0x14, 0x17, 0x10, 0x17,
0x14, 0x14, 0xF4, 0x04, 0xF4,
0x00, 0x00, 0xFF, 0x00, 0xF7,
0x14, 0x14, 0x14, 0x14, 0x14,
0x14, 0x14, 0xF7, 0x00, 0xF7,
0x14, 0x14, 0x14, 0x17, 0x14,
0x10, 0x10, 0x1F, 0x10, 0x1F,
0x14, 0x14, 0x14, 0xF4, 0x14,
```

```
0x10, 0x10, 0xF0, 0x10, 0xF0,
0x00, 0x00, 0x1F, 0x10, 0x1F,
0x00, 0x00, 0x00, 0x1F, 0x14,
0x00, 0x00, 0x00, 0xFC, 0x14,
0x00, 0x00, 0xF0, 0x10, 0xF0,
0x10, 0x10, 0xFF, 0x10, 0xFF,
0x14, 0x14, 0x14, 0xFF, 0x14,
0x10, 0x10, 0x10, 0x1F, 0x00,
0x00, 0x00, 0x00, 0xF0, 0x10,
0xFF, 0xFF, 0xFF, 0xFF, 0xFF,
0xF0, 0xF0, 0xF0, 0xF0, 0xF0,
0xFF, 0xFF, 0xFF, 0x00, 0x00,
0x00, 0x00, 0x00, 0xFF, 0xFF,
0x0F, 0x0F, 0x0F, 0x0F, 0x0F,
0x38, 0x44, 0x44, 0x38, 0x44,
0x7C, 0x2A, 0x2A, 0x3E, 0x14,
0x7E, 0x02, 0x02, 0x06, 0x06,
0x02, 0x7E, 0x02, 0x7E, 0x02,
0x63, 0x55, 0x49, 0x41, 0x63,
0x38, 0x44, 0x44, 0x3C, 0x04,
0x40, 0x7E, 0x20, 0x1E, 0x20,
0x06, 0x02, 0x7E, 0x02, 0x02,
0x99, 0xA5, 0xE7, 0xA5, 0x99,
0x1C, 0x2A, 0x49, 0x2A, 0x1C,
0x4C, 0x72, 0x01, 0x72, 0x4C,
0x30, 0x4A, 0x4D, 0x4D, 0x30,
0x30, 0x48, 0x78, 0x48, 0x30,
0xBC, 0x62, 0x5A, 0x46, 0x3D,
0x3E, 0x49, 0x49, 0x49, 0x00,
0x7E, 0x01, 0x01, 0x01, 0x7E,
0x2A, 0x2A, 0x2A, 0x2A, 0x2A,
0x44, 0x44, 0x5F, 0x44, 0x44,
0x40, 0x51, 0x4A, 0x44, 0x40,
0x40, 0x44, 0x4A, 0x51, 0x40,
0x00, 0x00, 0xFF, 0x01, 0x03,
0xE0, 0x80, 0xFF, 0x00, 0x00,
0x08, 0x08, 0x6B, 0x6B, 0x08,
0x36, 0x12, 0x36, 0x24, 0x36,
0x06, 0x0F, 0x09, 0x0F, 0x06,
```

```
  0x00, 0x00, 0x18, 0x18, 0x00,
  0x00, 0x00, 0x10, 0x10, 0x00,
  0x30, 0x40, 0xFF, 0x01, 0x01,
  0x00, 0x1F, 0x01, 0x01, 0x1E,
  0x00, 0x19, 0x1D, 0x17, 0x12,
  0x00, 0x3C, 0x3C, 0x3C, 0x3C
])}
```

Main Code:

CODE: Main.py

```
from ST7735lib import TFT
from SYSFONTlib import sysfont
from machine import SPI, Pin
import time

# Configure SPI
spi = SPI(0, baudrate=20000000, polarity=0, phase=0, sck=Pin(2), mosi=Pin(3),
miso=None)

# Initialize TFT with appropriate pins
tft = TFT(spi, 15, 4, 5)  # A0=15, RESET=4, CS=5
tft.initr()  # Initialize the TFT
tft.rgb(True)  # Use RGB color format
tft.rotation(3)
# Define functions for the display
def tftprinttest():
    tft.fill(TFT.BLACK)  # Clear the screen with black
    v = 30
    # Display text with different colors and sizes
    tft.text((0, v), "Hello World!", TFT.RED, sysfont, 1, nowrap=True)
    v += sysfont["Height"]
    tft.text((0, v), "Hello World!", TFT.YELLOW, sysfont, 2, nowrap=True)
    v += sysfont["Height"] * 2
    tft.text((0, v), "Hello World!", TFT.GREEN, sysfont, 3, nowrap=True)
    v += sysfont["Height"] * 3
    tft.text((0, v), str(1234.567), TFT.BLUE, sysfont, 4, nowrap=True)
```

```
    time.sleep_ms(1500)

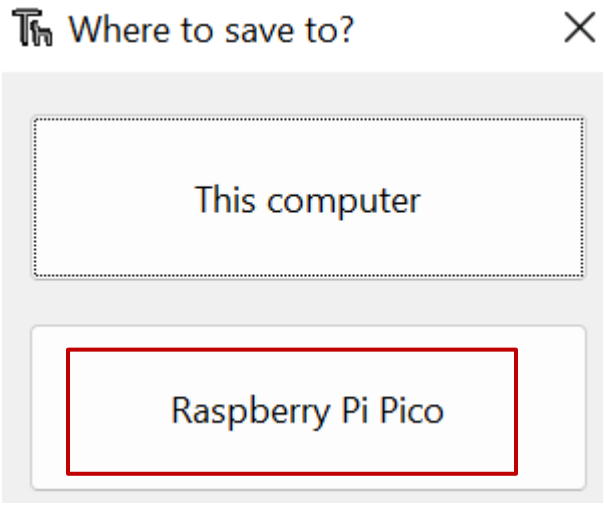
def test_main():
    tft.fill(TFT.BLACK)  # Clear the screen
    tft.text((0, 0), "IoBiT Solutions Welcomes You", TFT.WHITE, sysfont, 1.5)
    time.sleep(3)
    tftprinttest()
    time.sleep(5)

# Run the main function
test_main()
```

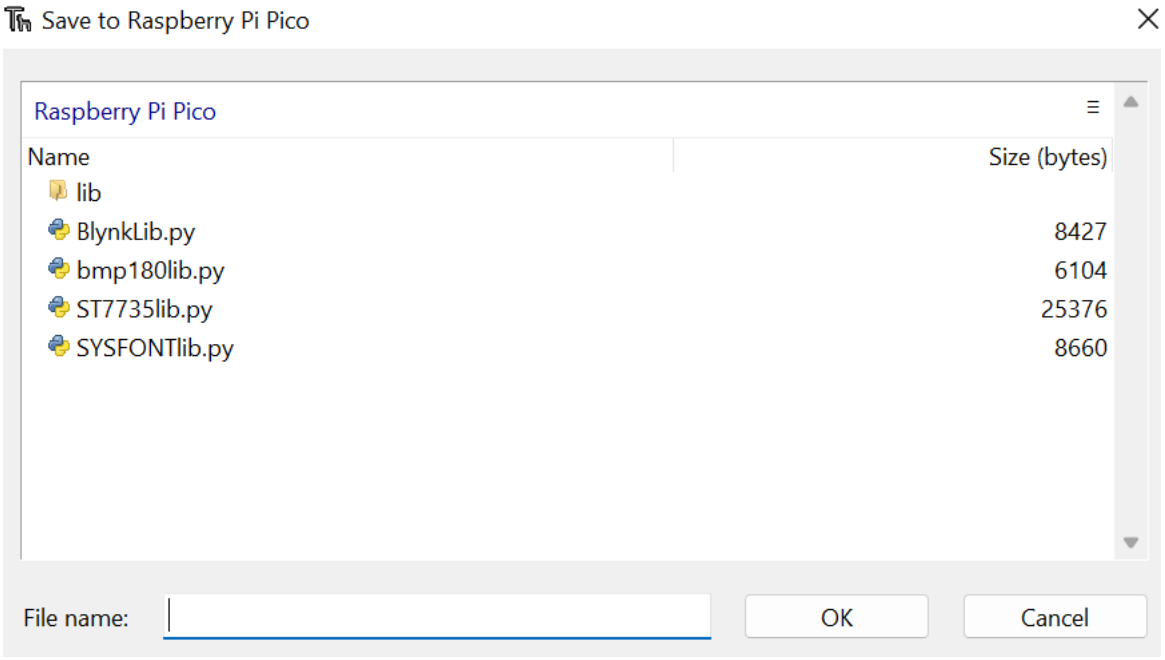
5. **Program to print BMP180 data in Shell.**

PIN CONFIGURATION	
PICO W	**BMP180**
GP9	SCL
GP8	SDA

Step1: Copy these following libraries and save in Pico Storage.



Step2: Once code library code is saved, then copy the main code and run the program.



Library:

bmp180lib.py

```
from ustruct import unpack as unp
from machine import I2C, Pin
import math
import time

# BMP180 class
class BMP180():
    '''
    Module for the BMP180 pressure sensor.
    '''

    _bmp_addr = 119          # Address of BMP180 is hardcoded on the sensor
```

```
    # Init
    def __init__(self, i2c_bus):
        self._bmp_i2c = i2c_bus
        self._bmp_addr = self._bmp_addr

        try:
            # Check chip ID
            chip_id_data = self._bmp_i2c.readfrom_mem(self._bmp_addr, 0xD0, 1)
            self.chip_id = chip_id_data[0]
            if self.chip_id != 0x55:  # BMP180 chip ID is 0x55
                raise ValueError("Invalid BMP180 chip ID")
        except OSError as e:
            raise RuntimeError("Failed to initialize BMP180: Check wiring or I2C
bus") from e

        # Read calibration data from EEPROM
        self._AC1 = unp('>h', self._bmp_i2c.readfrom_mem(self._bmp_addr, 0xAA,
2))[0]
        self._AC2 = unp('>h', self._bmp_i2c.readfrom_mem(self._bmp_addr, 0xAC,
2))[0]
        self._AC3 = unp('>h', self._bmp_i2c.readfrom_mem(self._bmp_addr, 0xAE,
2))[0]
        self._AC4 = unp('>H', self._bmp_i2c.readfrom_mem(self._bmp_addr, 0xB0,
2))[0]
        self._AC5 = unp('>H', self._bmp_i2c.readfrom_mem(self._bmp_addr, 0xB2,
2))[0]
        self._AC6 = unp('>H', self._bmp_i2c.readfrom_mem(self._bmp_addr, 0xB4,
2))[0]
        self._B1 = unp('>h', self._bmp_i2c.readfrom_mem(self._bmp_addr, 0xB6,
2))[0]
        self._B2 = unp('>h', self._bmp_i2c.readfrom_mem(self._bmp_addr, 0xB8,
2))[0]
        self._MB = unp('>h', self._bmp_i2c.readfrom_mem(self._bmp_addr, 0xBA,
2))[0]
        self._MC = unp('>h', self._bmp_i2c.readfrom_mem(self._bmp_addr, 0xBC,
2))[0]
        self._MD = unp('>h', self._bmp_i2c.readfrom_mem(self._bmp_addr, 0xBE,
2))[0]
```

```
        # Settings to be adjusted by user
        self.oversample_setting = 3
        self.baseline = 101325.0

        # Raw output
        self.UT_raw = None
        self.B5_raw = None
        self.MSB_raw = None
        self.LSB_raw = None
        self.XLSB_raw = None
        self.gauge = self.makegauge()  # Generator instance
        for _ in range(128):
            next(self.gauge)
            time.sleep_ms(1)

    def compvaldump(self):
        '''
        Returns a list of all compensation values
        '''
        return [self._AC1, self._AC2, self._AC3, self._AC4, self._AC5, self._AC6,
                self._B1, self._B2, self._MB, self._MC, self._MD,
self.oversample_setting]

    # Gauge raw
    def makegauge(self):
        '''
        Generator refreshing the raw measurements.
        '''
        delays = (5, 8, 14, 25)
        while True:
            self._bmp_i2c.writeto_mem(self._bmp_addr, 0xF4, bytearray([0x2E]))
            t_start = time.ticks_ms()
            while (time.ticks_ms() - t_start) <= 5:  # 5ms delay
                yield None
            try:
                self.UT_raw = self._bmp_i2c.readfrom_mem(self._bmp_addr, 0xF6, 2)
            except:
                yield None
```

```
            self._bmp_i2c.writeto_mem(self._bmp_addr, 0xF4, bytearray([0x34 +
(self.oversample_setting << 6)]))
            t_pressure_ready = delays[self.oversample_setting]
            t_start = time.ticks_ms()
            while (time.ticks_ms() - t_start) <= t_pressure_ready:
                yield None
            try:
                self.MSB_raw = self._bmp_i2c.readfrom_mem(self._bmp_addr, 0xF6,
1)
                self.LSB_raw = self._bmp_i2c.readfrom_mem(self._bmp_addr, 0xF7, 1)
                self.XLSB_raw = self._bmp_i2c.readfrom_mem(self._bmp_addr, 0xF8,
1)
            except:
                yield None
            yield True

    def blocking_read(self):
        if next(self.gauge) is not None:  # Discard old data
            pass
        while next(self.gauge) is None:
            pass

    @property
    def oversample_sett(self):
        return self.oversample_setting

    @oversample_sett.setter
    def oversample_sett(self, value):
        if value in range(4):
            self.oversample_setting = value
        else:
            print('oversample_sett can only be 0, 1, 2 or 3, using 3 instead')
            self.oversample_setting = 3

    @property
    def temperature(self):
        '''
        Temperature in degree C.
        '''
```

```
        next(self.gauge)
        try:
            UT = unp('>H', self.UT_raw)[0]
        except:
            return 0.0
        X1 = (UT - self._AC6) * self._AC5 / 2**15
        X2 = self._MC * 2**11 / (X1 + self._MD)
        self.B5_raw = X1 + X2
        return (((X1 + X2) + 8) / 2**4) / 10

    @property
    def pressure(self):
        '''
        Pressure in mbar.
        '''
        next(self.gauge)
        self.temperature  # Populate self.B5_raw
        try:
            MSB = unp('B', self.MSB_raw)[0]
            LSB = unp('B', self.LSB_raw)[0]
            XLSB = unp('B', self.XLSB_raw)[0]
        except:
            return 0.0
        UP = ((MSB << 16) + (LSB << 8) + XLSB) >> (8 - self.oversample_setting)
        B6 = self.B5_raw - 4000
        X1 = (self._B2 * (B6 ** 2 / 2**12)) / 2**11
        X2 = self._AC2 * B6 / 2**11
        X3 = X1 + X2
        B3 = ((int((self._AC1 * 4 + X3)) << self.oversample_setting) + 2) / 4
        X1 = self._AC3 * B6 / 2**13
        X2 = (self._B1 * (B6 ** 2 / 2**12)) / 2**16
        X3 = ((X1 + X2) + 2) / 2**2
        B4 = abs(self._AC4) * (X3 + 32768) / 2**15
        B7 = (abs(UP) - B3) * (50000 >> self.oversample_setting)
        if B7 < 0x80000000:
            pressure = (B7 * 2) / B4
        else:
            pressure = (B7 / B4) * 2
        X1 = (pressure / 2**8) ** 2
```

```
    X1 = (X1 * 3038) / 2**16
    X2 = (-7357 * pressure) / 2**16
    return pressure + (X1 + X2 + 3791) / 2**4

  @property
  def altitude(self):
    '''
    Altitude in meters.
    '''
    try:
      p = -7990.0 * math.log(self.pressure / self.baseline)
    except:
      p = 0.0
    return p
```

Main Code:

CODE: BMP_MAIN.py

```
from machine import I2C, Pin
from bmp180 import BMP180
import time

# Initialize I2C
i2c = I2C(0, scl=Pin(9), sda=Pin(8), freq=100000)  # I2C0 with SCL on GPIO 9 and SDA on GPIO 8

# Initialize BMP180
bmp = BMP180(i2c)
bmp.oversample_sett = 2  # Set oversampling for better accuracy
bmp.sealevel = 1013.25  # Set sea-level pressure in hPa (standard value)

while True:
    temp = bmp.temperature  # Get temperature in Celsius
    pressure = bmp.pressure  # Get pressure in hPa
    altitude = bmp.altitude  # Calculate altitude based on pressure

    print(f"Temperature: {temp:.2f} °C")
```

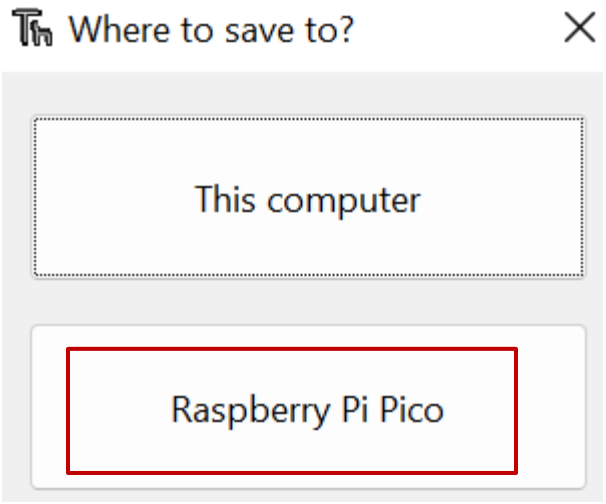
```
    print(f"Pressure: {pressure:.2f} hPa")
    print(f"Altitude: {altitude:.2f} m")
    print("-" * 30)

    time.sleep(2)  # Wait for 2 seconds before the next reading
```

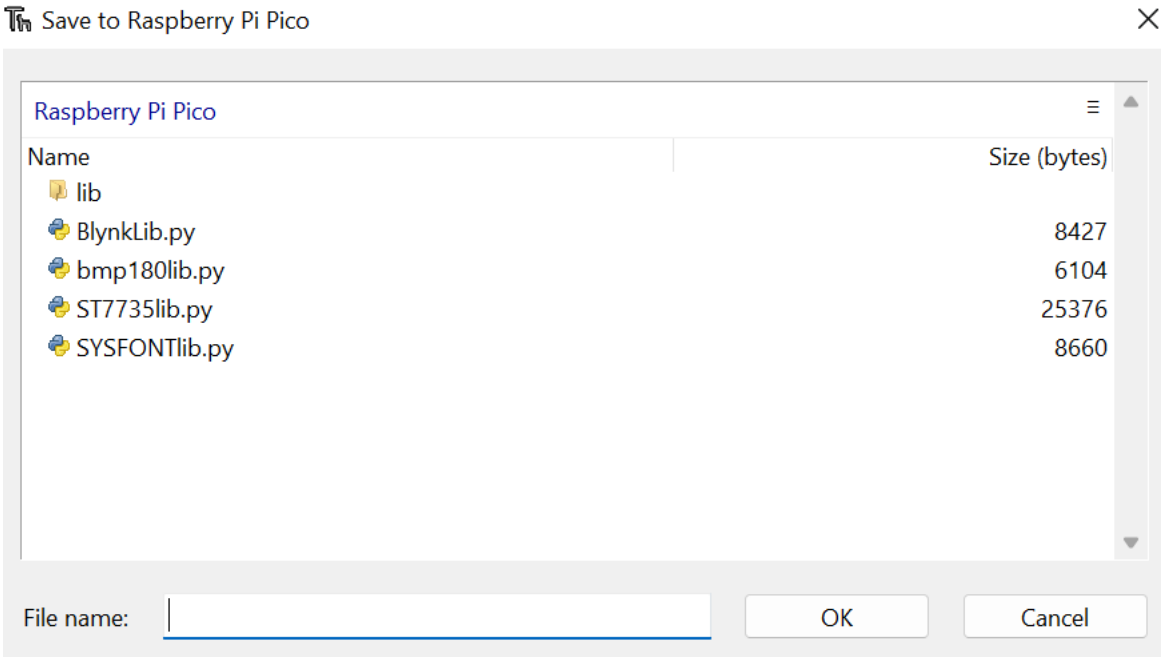
6. Program to print MPU6050 data in Shell.

PIN CONFIGURATION	
PICO W	**MPU6050**
GP5	SCL
GP4	SDA

Step1: Copy these following libraries and save in Pico Storage.



Step2: Once code library code is saved, then copy the main code and run the program.



Library:

MPU6050lib.py

```
from machine import Pin, I2C
import utime
PWR_MGMT_1 = 0x6B
SMPLRT_DIV = 0x19
CONFIG = 0x1A
GYRO_CONFIG = 0x1B
ACCEL_CONFIG = 0x1C
TEMP_OUT_H = 0x41
ACCEL_XOUT_H = 0x3B
GYRO_XOUT_H = 0x43

def init_mpu6050(i2c, address=0x68):
    i2c.writeto_mem(address, PWR_MGMT_1, b'\x00')
```

```
    utime.sleep_ms(100)
    i2c.writeto_mem(address, SMPLRT_DIV, b'\x07')
    i2c.writeto_mem(address, CONFIG, b'\x00')
    i2c.writeto_mem(address, GYRO_CONFIG, b'\x00')
    i2c.writeto_mem(address, ACCEL_CONFIG, b'\x00')

def read_raw_data(i2c, addr, address=0x68):
    high = i2c.readfrom_mem(address, addr, 1)[0]
    low = i2c.readfrom_mem(address, addr + 1, 1)[0]
    value = high << 8 | low
    if value > 32768:
        value = value - 65536
    return value

def get_mpu6050_data(i2c):
    temp = read_raw_data(i2c, TEMP_OUT_H) / 340.0 + 36.53
    accel_x = read_raw_data(i2c, ACCEL_XOUT_H) / 16384.0
    accel_y = read_raw_data(i2c, ACCEL_XOUT_H + 2) / 16384.0
    accel_z = read_raw_data(i2c, ACCEL_XOUT_H + 4) / 16384.0
    gyro_x = read_raw_data(i2c, GYRO_XOUT_H) / 131.0
    gyro_y = read_raw_data(i2c, GYRO_XOUT_H + 2) / 131.0
    gyro_z = read_raw_data(i2c, GYRO_XOUT_H + 4) / 131.0

    return {
        'temp': temp,
        'accel': {
            'x': accel_x,
            'y': accel_y,
            'z': accel_z,
        },
        'gyro': {
            'x': gyro_x,
            'y': gyro_y,
            'z': gyro_z,
        }
    }
    utime.sleep(0.2)
```

Main Code:

Code: MPU6050.py

```
from machine import Pin, I2C
import utime
from MPU6050lib import init_mpu6050, get_mpu6050_data

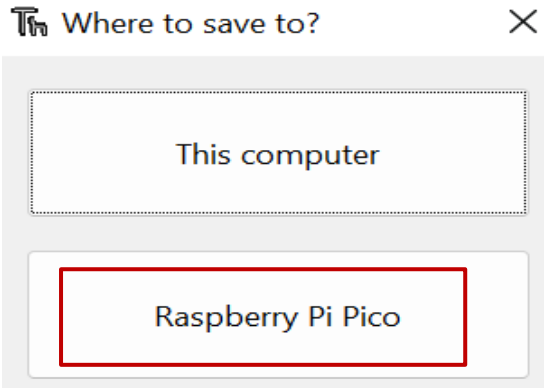
i2c = I2C(0, scl=Pin(5), sda=Pin(4), freq=400000)
init_mpu6050(i2c)

while True:
    data = get_mpu6050_data(i2c)
    print("Temperature: {:.2f} °C".format(data['temp']))
    print("Acceleration: X: {:.2f}, Y: {:.2f}, Z: {:.2f} g".format(data['accel']['x'],
data['accel']['y'], data['accel']['z']))
    print("Gyroscope: X: {:.2f}, Y: {:.2f}, Z: {:.2f}
°/s".format(data['gyro']['x'],data['gyro']['y'], data['gyro']['z']))
    utime.sleep(0.5)
```

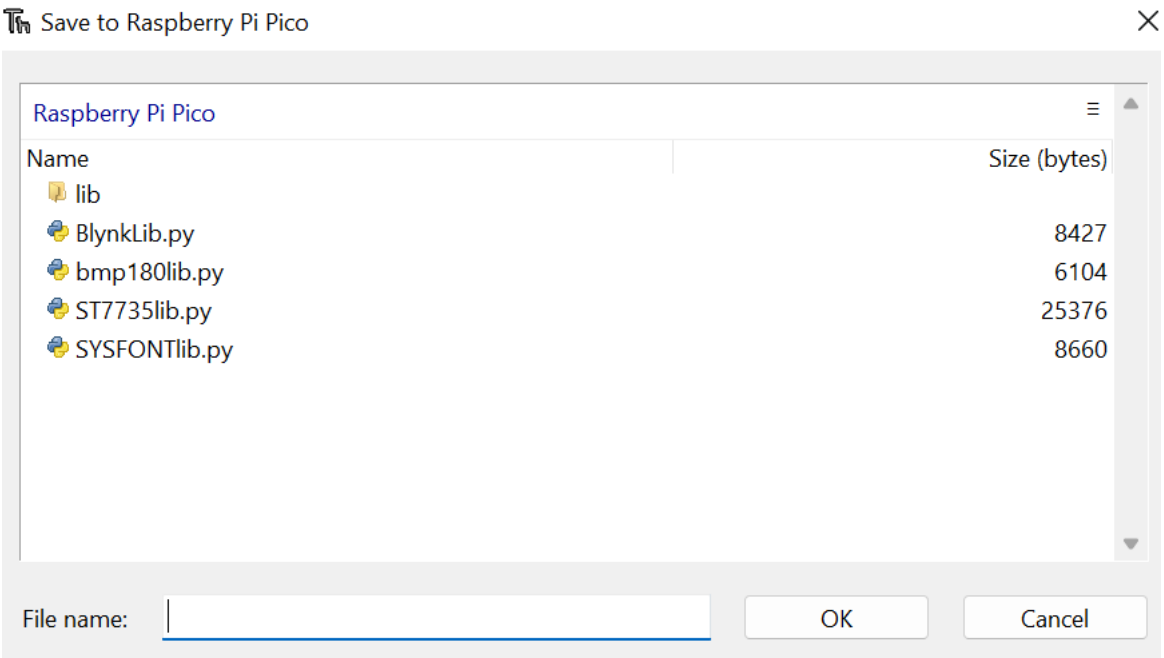
7. Program to control LED using BLYNK IOT platform.

PIN CONFIGURATION	
PICO W	**LED**
GP1	CONNECT TO ANY LED PINS
GP2	CONNECT TO ANY LED PINS

Step1: Copy these following libraries and save in Pico Storage.



Step2: Once code library code is saved, then copy the main code and run the program.



Library:

BlinkLib.py

```
__version__ = "1.0.0"

import struct
import time
import sys
import os

try:
    import machine
    gettime = lambda: time.ticks_ms()
    SOCK_TIMEOUT = 0
except ImportError:
    const = lambda x: x
    gettime = lambda: int(time.time() * 1000)
    SOCK_TIMEOUT = 0.05

def dummy(*args):
    pass

MSG_RSP = const(0)
MSG_LOGIN = const(2)
MSG_PING  = const(6)

MSG_TWEET = const(12)
MSG_NOTIFY = const(14)
MSG_BRIDGE = const(15)
MSG_HW_SYNC = const(16)
MSG_INTERNAL = const(17)
MSG_PROPERTY = const(19)
MSG_HW = const(20)
MSG_HW_LOGIN = const(29)
MSG_EVENT_LOG = const(64)
```

```
MSG_REDIRECT  = const(41)  # TODO: not implemented
MSG_DBG_PRINT  = const(55) # TODO: not implemented

STA_SUCCESS = const(200)
STA_INVALID_TOKEN = const(9)

DISCONNECTED = const(0)
CONNECTING = const(1)
CONNECTED = const(2)

print("""
    ___  __          __
   / _ )/ /_ _____  / /__
  / _  / / // / _ \\/  '_/
 /____/_/\\_, /_//_/_/\\_\\
        /___/ for Python v""" + __version__ + " (" + sys.platform + ")\n")

class EventEmitter:
    def __init__(self):
        self._cbks = {}

    def on(self, evt, f=None):
        if f:
            self._cbks[evt] = f
        else:
            def D(f):
                self._cbks[evt] = f
                return f
            return D

    def emit(self, evt, *a, **kv):
        if evt in self._cbks:
            self._cbks[evt](*a, **kv)


class BlynkProtocol(EventEmitter):
    def __init__(self, auth, tmpl_id=None, fw_ver=None, heartbeat=50,
buffin=1024, log=None):
        EventEmitter.__init__(self)
```

```
        self.heartbeat = heartbeat*1000
        self.buffin = buffin
        self.log = log or dummy
        self.auth = auth
        self.tmpl_id = tmpl_id
        self.fw_ver = fw_ver
        self.state = DISCONNECTED
        self.connect()

    def virtual_write(self, pin, *val):
        self._send(MSG_HW, 'vw', pin, *val)

    def send_internal(self, pin, *val):
        self._send(MSG_INTERNAL,  pin, *val)

    def set_property(self, pin, prop, *val):
        self._send(MSG_PROPERTY, pin, prop, *val)

    def sync_virtual(self, *pins):
        self._send(MSG_HW_SYNC, 'vr', *pins)

    def log_event(self, *val):
        self._send(MSG_EVENT_LOG, *val)

    def _send(self, cmd, *args, **kwargs):
        if 'id' in kwargs:
            id = kwargs.get('id')
        else:
            id = self.msg_id
            self.msg_id += 1
            if self.msg_id > 0xFFFF:
                self.msg_id = 1

        if cmd == MSG_RSP:
            data = b''
            dlen = args[0]
        else:
            data = ('\0'.join(map(str, args))).encode('utf8')
            dlen = len(data)
```

```
        self.log('<', cmd, id, '|', *args)
        msg = struct.pack("!BHH", cmd, id, dlen) + data
        self.lastSend = gettime()
        self._write(msg)

    def connect(self):
        if self.state != DISCONNECTED: return
        self.msg_id = 1
        (self.lastRecv, self.lastSend, self.lastPing) = (gettime(), 0, 0)
        self.bin = b""
        self.state = CONNECTING
        self._send(MSG_HW_LOGIN, self.auth)

    def disconnect(self):
        if self.state == DISCONNECTED: return
        self.bin = b""
        self.state = DISCONNECTED
        self.emit('disconnected')

    def process(self, data=None):
        if not (self.state == CONNECTING or self.state == CONNECTED): return
        now = gettime()
        if now - self.lastRecv > self.heartbeat+(self.heartbeat//2):
            return self.disconnect()
        if (now - self.lastPing > self.heartbeat//10 and
            (now - self.lastSend > self.heartbeat or
             now - self.lastRecv > self.heartbeat)):
            self._send(MSG_PING)
            self.lastPing = now

        if data != None and len(data):
            self.bin += data

        while True:
            if len(self.bin) < 5:
                break

            cmd, i, dlen = struct.unpack("!BHH", self.bin[:5])
```

```
        if i == 0: return self.disconnect()

        self.lastRecv = now
        if cmd == MSG_RSP:
            self.bin = self.bin[5:]

            self.log('>', cmd, i, '|', dlen)
            if self.state == CONNECTING and i == 1:
                if dlen == STA_SUCCESS:
                    self.state = CONNECTED
                    dt = now - self.lastSend
                    info = ['ver', __version__, 'h-beat', self.heartbeat//1000, 'buff-in',
self.buffin, 'dev', sys.platform+'-py']
                    if self.tmpl_id:
                        info.extend(['tmpl', self.tmpl_id])
                        info.extend(['fw-type', self.tmpl_id])
                    if self.fw_ver:
                        info.extend(['fw', self.fw_ver])
                    self._send(MSG_INTERNAL, *info)
                    try:
                        self.emit('connected', ping=dt)
                    except TypeError:
                        self.emit('connected')
                else:
                    if dlen == STA_INVALID_TOKEN:
                        self.emit("invalid_auth")
                        print("Invalid auth token")
                    return self.disconnect()
        else:
            if dlen >= self.buffin:
                print("Cmd too big: ", dlen)
                return self.disconnect()

            if len(self.bin) < 5+dlen:
                break

            data = self.bin[5:5+dlen]
            self.bin = self.bin[5+dlen:]
```

```
            args = list(map(lambda x: x.decode('utf8'), data.split(b'\0')))

            self.log('>', cmd, i, '|', ','.join(args))
            if cmd == MSG_PING:
                self._send(MSG_RSP, STA_SUCCESS, id=i)
            elif cmd == MSG_HW or cmd == MSG_BRIDGE:
                if args[0] == 'vw':
                    self.emit("V"+args[1], args[2:])
                    self.emit("V*", args[1], args[2:])
            elif cmd == MSG_INTERNAL:
                self.emit("internal:"+args[0], args[1:])
            elif cmd == MSG_REDIRECT:
                self.emit("redirect", args[0], int(args[1]))
            else:
                print("Unexpected command: ", cmd)
                return self.disconnect()

import socket

class Blynk(BlynkProtocol):
    def __init__(self, auth, **kwargs):
        self.insecure = kwargs.pop('insecure', False)
        self.server = kwargs.pop('server', 'blynk.cloud')
        self.port = kwargs.pop('port', 80 if self.insecure else 443)
        BlynkProtocol.__init__(self, auth, **kwargs)
        self.on('redirect', self.redirect)

    def redirect(self, server, port):
        self.server = server
        self.port = port
        self.disconnect()
        self.connect()

    def connect(self):
        print('Connecting to %s:%d...' % (self.server, self.port))
        s = socket.socket()
        s.connect(socket.getaddrinfo(self.server, self.port)[0][-1])
        try:
            s.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
```

```
        except:
            pass
        if self.insecure:
            self.conn = s
        else:
            try:
                import ussl
                ssl_context = ussl
            except ImportError:
                import ssl
                ssl_context = ssl.create_default_context()
            self.conn = ssl_context.wrap_socket(s, server_hostname=self.server)
        try:
            self.conn.settimeout(SOCK_TIMEOUT)
        except:
            s.settimeout(SOCK_TIMEOUT)
        BlynkProtocol.connect(self)

    def _write(self, data):
        #print('<', data)
        self.conn.write(data)
        # TODO: handle disconnect

    def run(self):
        data = b''
        try:
            data = self.conn.read(self.buffin)
            #print('>', data)
        except KeyboardInterrupt:
            raise
        except socket.timeout:
            # No data received, call process to send ping messages when needed
            pass
        except: # TODO: handle disconnect
            return
        self.process(data)
```

Main Code:

CODE: LED_CONTROL.py

```
import time
import network
from machine import Pin
import BlynkLib

led1=Pin(1, Pin.OUT)
led2=Pin(2, Pin.OUT)

wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect("HOTSPOT-NAME","HOTSPOT-PASSWORD")
 #define BLYNK_TEMPLATE_ID "TMPL3X4LhI-DN"
#define BLYNK_TEMPLATE_NAME "PicoW LED"
#define BLYNK_AUTH_TOKEN "jTNSZ4d1EaEaGre-
67LMQWwWf7Xa9lob"
BLYNK_AUTH = 'jTNSZ4d1EaEaGre-67LMQWwWf7Xa9lob'

# connect the network
wait = 10
while wait > 0:
    if wlan.status() < 0 or wlan.status() >= 3:
        break
    wait -= 1
    print('waiting for connection...')
    time.sleep(1)

# Handle connection error
if wlan.status() != 3:
    raise RuntimeError('network connection failed')
else:
    print('connected')
    ip=wlan.ifconfig()[0]
    print('IP: ', ip)


"Connection to Blynk"
# Initialize Blynk
```

```
blynk = BlynkLib.Blynk(BLYNK_AUTH)

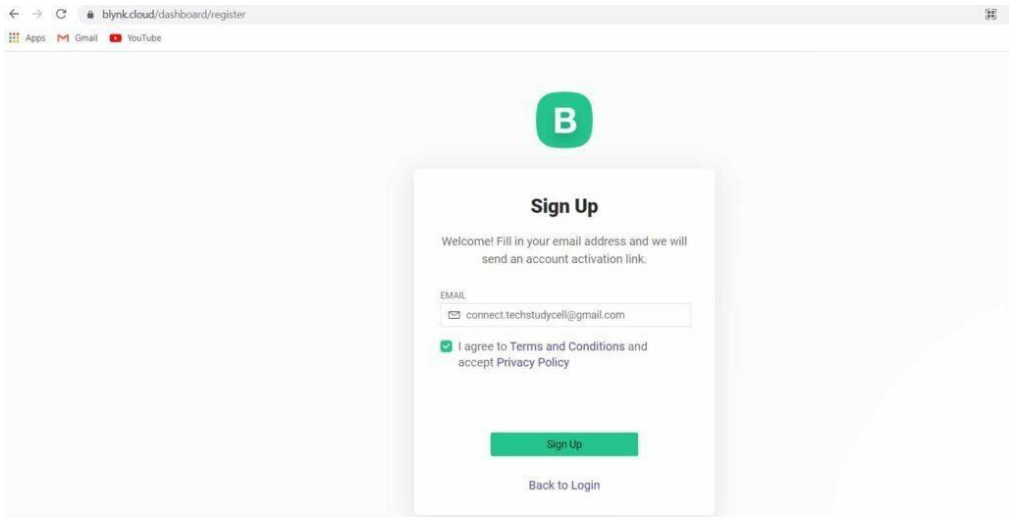
# Register virtual pin handler
@blynk.on("V0") #virtual pin V0
def v0_write_handler(value): #read the value
    if int(value[0]) == 1:
        led1.value(1) #turn the led on
    else:
        led1.value(0)    #turn the led off

@blynk.on("V1") #virtual pin V0
def v1_write_handler(value): #read the value
    if int(value[0]) == 1:
        led2.value(1) #turn the led on
    else:
        led2.value(0)    #turn the led off
while True:
    blynk.run()
```

To Experience the Outcome

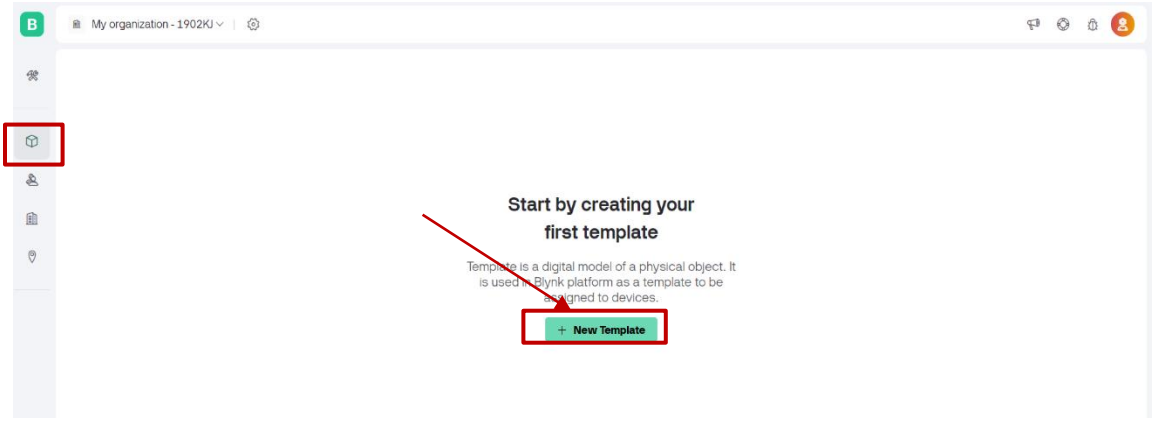
Creating a Blynk IoT Environment

1. Open the URL https://blynk.cloud/



2. Signup in to the

3. Create Template in Blynk IoT Cloud

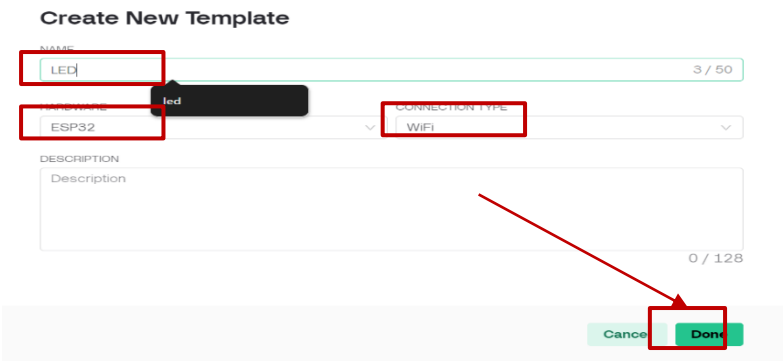


4. After login to Blynk IoT account, first you have create a template. In the dashboard click onthe Templates from the left side menu.

5. Then click on New Template.

6. Then, you have to enter a name for that template.

7. Select the microcontroller or board you will use in the project from the Hardwaredropdown menu. Here select Raspberry.



8. The connection type should be WiFi.

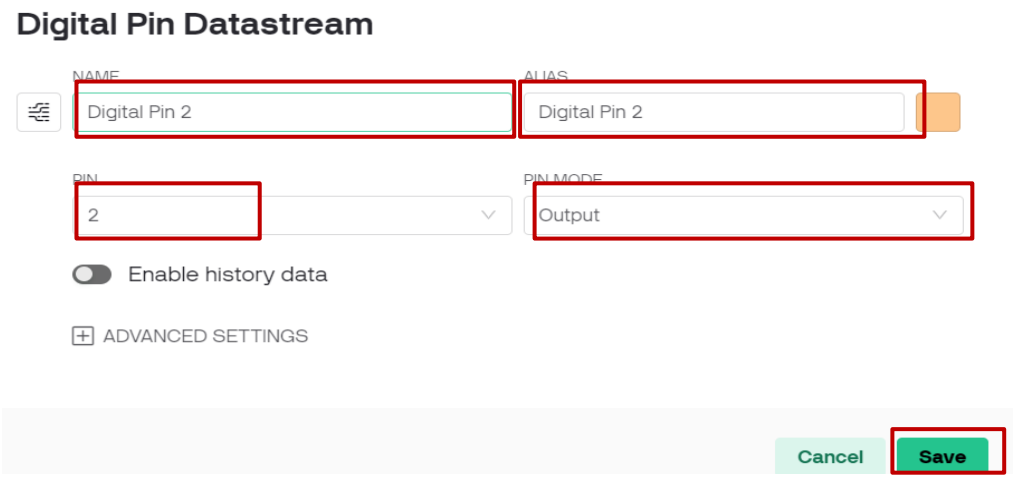
9. You can enter small description for the template. This field is optional.

10. Now click on Done.

11. Create Datastreams in the Blynk IoT template

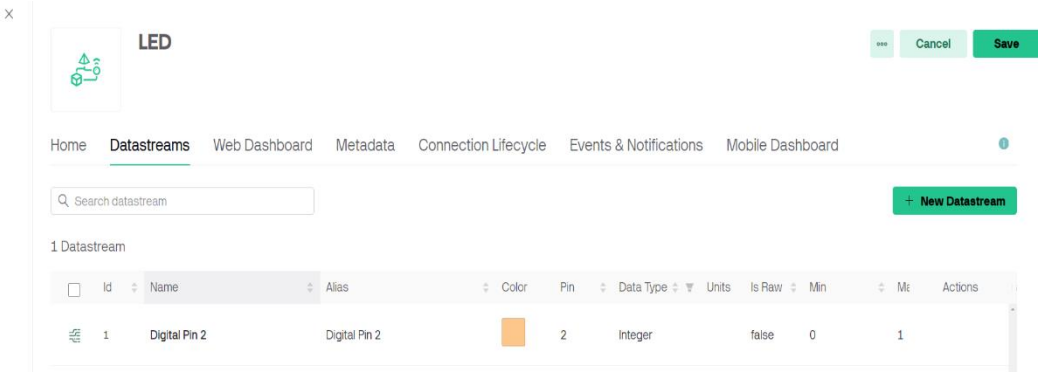


12. After creating the template, go to Datastreams tab.

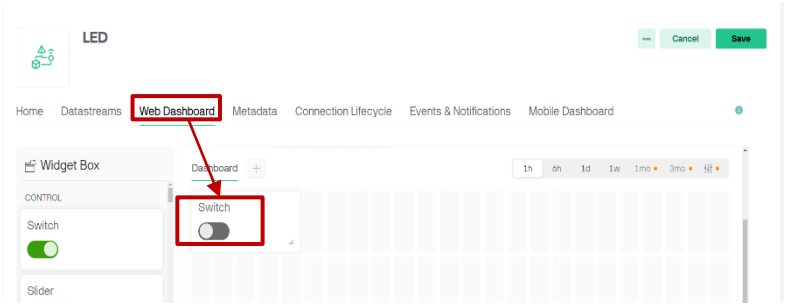


13. Then click on “New Datastream” and select Digital Pin.
14. Now, you have to enter a name for this Datastream.

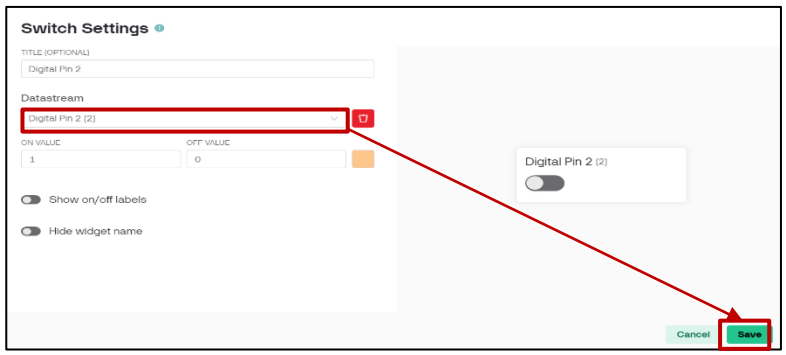
15. Select the Digital Pin and Pinmode from the dropdown menu as per the requirement.



16. After that click on Create.
    Create Web Dashboard in the Blynk IoT template

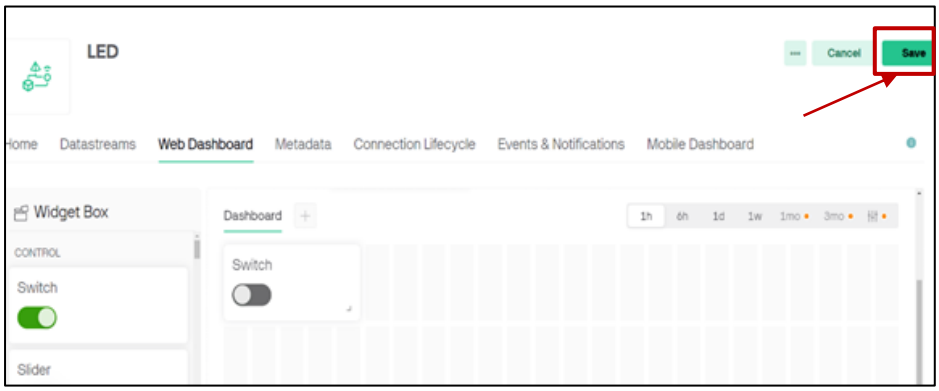


17. Now, go to the Web Dashboard tab to add widgets.
18. In the dashboard, you can easily drag and drop switch widget from the left side.
19. Now, if you hover over the widget, you will find a button with a gear icon. Click on thebutton to

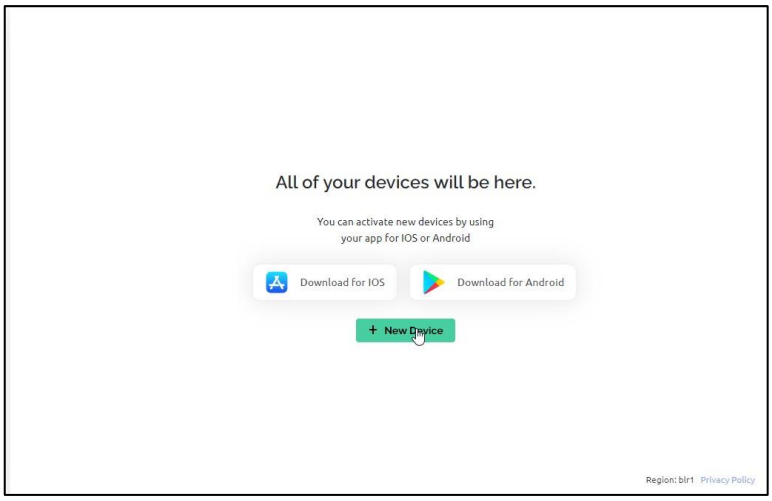


go to setting.

20. Click on Save.

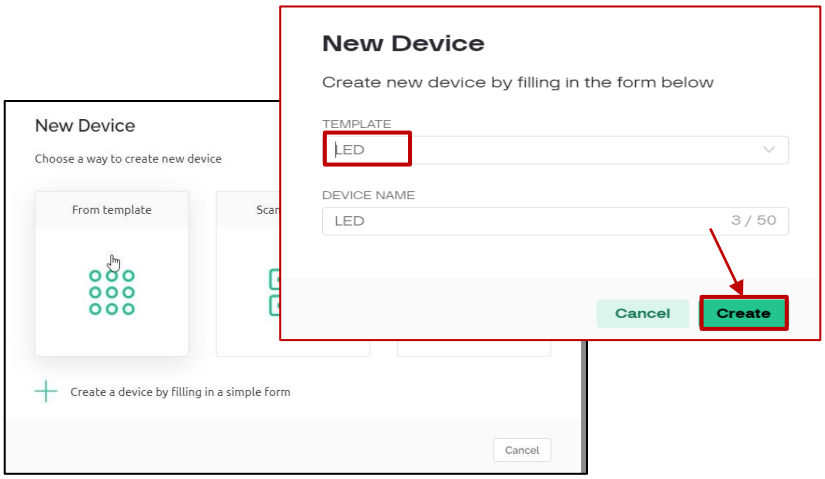


21. Go to Organization click on “New Device”

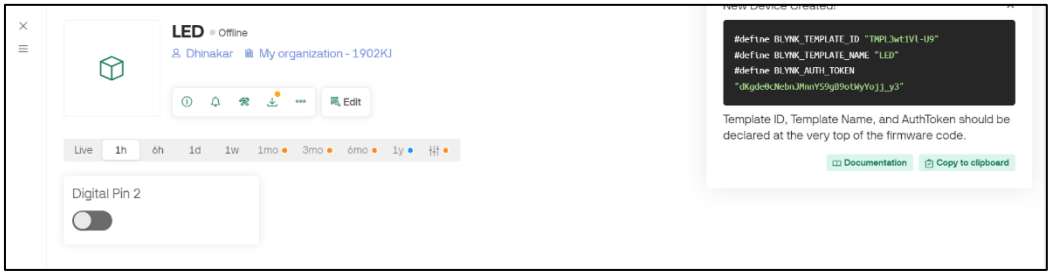


22. Choose from Template

23. Select your devices



24. In the code, you just have to update the BLYNK_TEMPLATE_ID, BLYNK_DEVICE_NAME & BLYNK_AUTH_TOKEN.



8. Program to send DHT11 data to THINGSPEAK.

PIN CONFIGURATION	
PICO W	**DHT11**
GP15	DATA

CODE: THINGSPEAK_DHT.py

```
import network
import urequests
import random
import dht
from machine import Pin
from time import sleep

#Network Initialization
ssid = " HOTSPOT-NAME "
password = "HOTSPOT-PASSWORD"

sensor = dht.DHT11(Pin(15))
def ConnectWiFi():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    wlan.connect(ssid, password)
    while wlan.isconnected() == False:
        print('Waiting for connection...')
        sleep(1)
    ip = wlan.ifconfig()[0]
    print(f'Connected on {ip}')
    return ip

#Connect to Network
ip = ConnectWiFi()

#ThingSpeak Initialization
server = "http://api.thingspeak.com/"
apikey = "18GXM50CZ7DZV3SW"
field = 1
```

```
#Main Program
while True:
    sensor.measure()
    temp = sensor.temperature()
    hum = sensor.humidity()

    print('Temperature: %3.1f C' %temp)
    print('Humidity: %3.1f %%' %hum)
    print("")

    url = f"{server}/update?api_key={apikey}&field{field}={temp}"
    request = urequests.post(url)
    request.close()

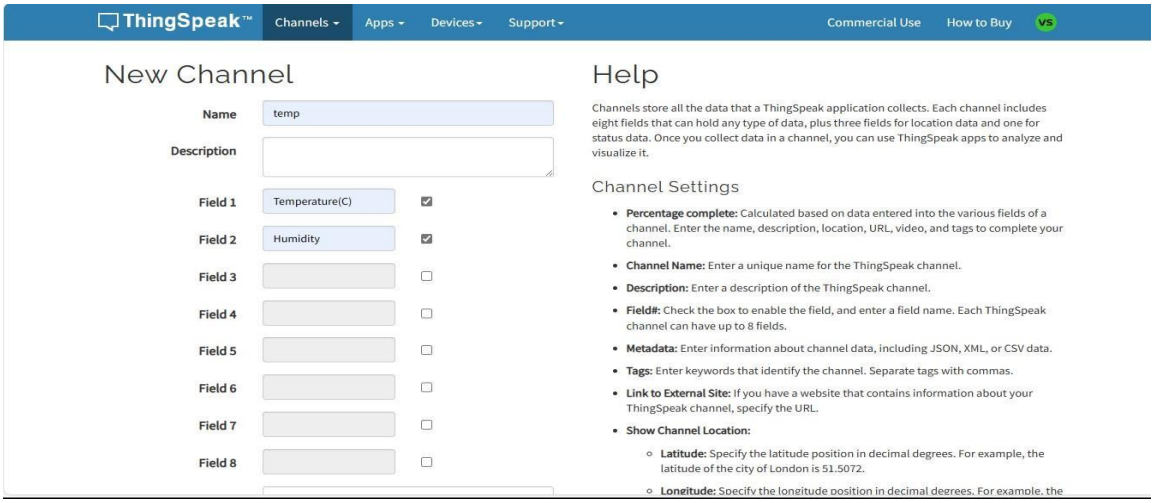
    sleep(20)
```

Procedure:-

Step 1: Go to https://thingspeak.com.

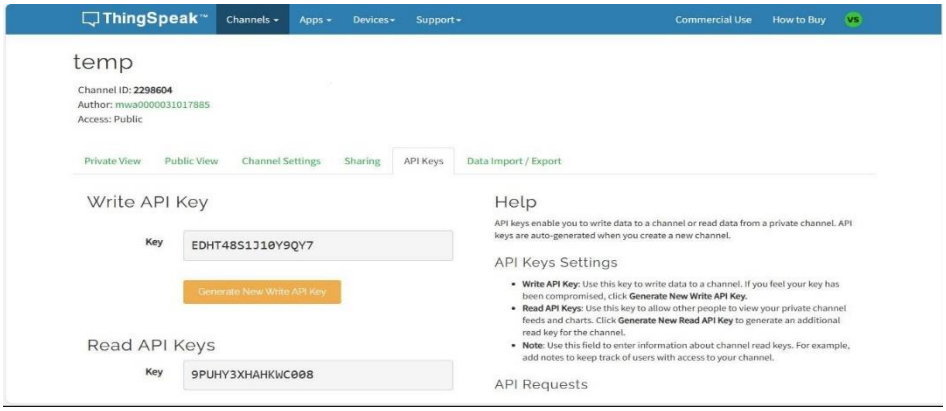
Step 2: Create an account using Email-id.

Step 3: Now, create a channel.



Step 4:After the creation of channel ,go to the API Keys and copy the Write API Key.

Step 5: And now paste the Write API Key in your code and your channel number in place of “myChannelNumber” and “myWriteAPIKey”.



Step 6: After pasting the API key and Channel number, now you can upload the code to ESP32.

Step 7: Now you can view the temperature and humidity data in the form of graph in your channel i.e. in the private view.



